SAM 2 선행연구

SAM 2 성능 개선을 위한 선행연구 조사와 과제연구 설계안

조사 기준일: 2026-08-20
중심 범위: SAM 2/SAM 2.1 기반의 프롬프트형 비디오 객체 분할(PVOS), 반지도 비디오 객체 분할(VOS), 객체 추적(VOT)
핵심 질문: **SAM 2**의 평균 정확도를 재는 데서 끝나지 않고**,** 어떤 실패 원인을 어떻게 고쳐야 실제 성능이 좋아지는가**?**

0. 먼저 읽을 결론

1. 실험 기준선은 원본 SAM 2가 아니라 공식 **SAM 2.1**로 잡는 것이 맞다. 공식 저장소에는 개선된 체크포인트와 다중 객체별 독립 추론이 포함되어 있다. (공식 코드·체크포인트)
2. 장기/단기 이중 메모리, 고정 신뢰도 임계값으로 메모리 선택, 칼만 필터 추가, 가림 시 메모리 동결만으로는 신규성이 약하다. SAM2Long, DAM4SAM, SAMURAI, UAMP, HiM2SAM, OASAMT, Lean-SAM2, 그리고 2026-08-19 공개된 SAM2Dual까지 이미 이 공간을 상당 부분 차지했다.
3. 가장 추천하는 주제는 다음이다.

> 교차 데이터셋 신뢰도 보정에 기반한 **SAM 2.1**의 선택적 메모리 갱신과 오류 누적 억제

이 주제의 차별점은 “신뢰도 값이 높으면 저장한다”가 아니다. **SAM 2**의 내부 점수가 실제 오류 확률과 얼마나 맞는지 영상 단위·데이터셋 간에 검증·보정하고, 보정된 실패 위험에 따라 메모리 쓰기와 보류를 결정하는 것이다. 상태 제어와 재탐색은 이 가설을 시험하는 응용 모듈이지 신규성의 중심으로 주장하지 않는다. 최종 J&F뿐 아니라 실패 검출 AUROC, calibration error, 잘못된 메모리 기록률, 가림 후 회복 시간까지 평가한다.

4. 두 번째로 좋은 선택은 소형·얇은 객체를 위한 시간적으로 일관된 경계 보정이다. SAM2Refiner가 공간적 세부 복원을 이미 다루므로, 단순 고해상도 decoder가 아니라 **motion-aligned temporal boundary consistency**로 범위를 좁혀야 한다.
5. 세 번째 선택은 저조도·안개·비·모션블러에서 불확실성 기반 메모리 보호다. CVPR 2026 RobustPVOS가 이 문제를 정식 벤치마크로 만들었으므로, 공개 데이터와 명확한 실패 조건을 사용할 수 있다.

---

1. 먼저 연구 범위를 정확히 고정하기

SAM 2 관련 논문은 겉으로 비슷해도 실제 과제가 다르다. 논문 성능을 비교할 때 아래 구분을 섞으면 안 된다.

구분	입력	출력/평가	이 문서에서의 역할
반지도 VOS	보통 첫 프레임 정답 마스크	이후 프레임의 객체 마스크, J/F/J&F	주 연구 범위
Promptable VOS	점·박스·마스크 및 중간 수정	마스크와 사용자 상호작용 횟수	주 연구 범위

구분	입력	출력/평가	이 문서에서의 역할
VOT	첫 프레임 박스/마스크	단일 객체 위치·마스크, AUC/AO/Q	메모리·가림 연구 참고
MOT	검출된 여러 객체 또는 질의	궤적·ID, HOTA/IDF1	다중 객체 확장 참고
RVOS/VRS	텍스트 또는 추론 문장	언어가 가리키는 객체 마스크	의미·재탐색 확장 참고
정적 이미지/의료 3D	점·박스·support mask 등	한 장 또는 slice sequence의 mask	경계·도메인 적응 참고

본 과제는 우선 첫 프레임 마스크가 주어진 장기 **VOS**로 고정하는 것이 가장 깔끔하다. 텍스트, 외부 검출기, 사용자 재클릭을 처음부터 모두 넣으면 무엇이 개선을 만들었는지 분리하기 어렵다.

문헌 상태 표기

- **[정식]** 학회 본회의 또는 저널 게재
- **[워크숍]** 학회 워크숍 게재
- **[프리프린트]** arXiv 등 공개 원고이며 동료심사 확정 전
- **[공식 구현]** 논문 게재와 별개인 공식 코드/체크포인트 공개

프리프린트는 최신 아이디어 중복 여부를 확인하는 데는 중요하지만, 확립된 결과처럼 인용하면 안 된다.

---

2. SAM 2가 실패하는 구조적 이유

SAM 2 원 논문은 Hiera 이미지 인코더, 메모리 어텐션, 프롬프트/마스크 디코더, 메모리 인코더와 스트리밍 메모리를 결합한다. 이전 프레임의 예측이 다음 프레임의 조건으로 다시 들어가는 자기회귀적 구조이므로, 한 번의 실수가 이후 판단 자료까지 오염시킬 수 있다.

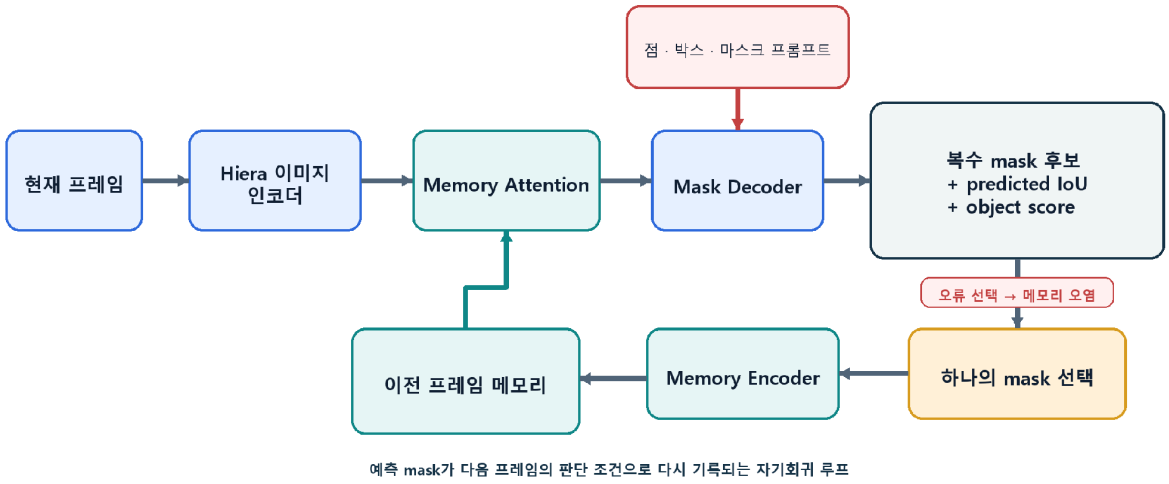


그림 ***1. SAM 2***의 스트리밍 메모리 기반 추론과 오류 전파 지점

핵심 실패 지점은 다섯 가지다.

1. 탐욕적 후보 선택: 모호한 프레임에서도 predicted IoU가 가장 높은 후보 하나를 고른다. 내부 점수가 틀리면 잘못된 mask가 바로 확정된다.
2. 오류의 메모리 기록: 잘못 선택한 mask가 메모리로 들어가 다음 프레임을 오염시킨다. 단발성 오류가 drift나 identity switch로 변한다.
3. **FIFO**와 짧은 시간 범위: 최근 프레임은 현재 모습 적응에 좋지만, 긴 가림 뒤 원래 정체성을 되찾는 anchor로는 약하다.
4. 명시적 운동·상태 모델 부족: 빠른 이동, 화면 밖 이탈, 가림/재등장 상태를 별도 동역학으로 다루지 않는다.
5. 객체 간 상호작용 부족: 기본 구조는 여러 객체를 대체로 독립 처리한다. 비슷한 객체가 겹치면 mask 소유권과 identity가 충돌한다.

세부적으로는 작은 객체, 얇은 구조, 낮은 대비, 모션블러, 악천후에서 이미지 특징과 경계가 먼저 약해지고, 그 오류가 다시 시간축으로 증폭된다.

---

3. 가장 직접적인 선행연구 지도

3.1 장기 메모리, 오류 누적, 가림과 재등장

연구	상태	무엇을 바꿨나	핵심 성과/의미	남은 한계와 중복 주의
SAM2Long	[정식] ICCV 2025	여러 mask 경로를 유지하는 제한적 tree search, object-aware memory filtering/modulation	12개 직접 비교 평균 +3.7점, 장기 benchmark 최대 +5.3 J&F. 탐욕 선택이 오류 누적의 원인임을 직접 공략	여러 경로만큼 연산·메모리 증가, 주로 단일 객체. “multi-hypothesis만 추가”는 이미 선행연구가 있음
DAM4SAM	[정식] CVPR 2025	Recent Appearance Memory와 Distractor Resolving Memory, introspection 기반 update	유사 방해물에 특화한 DiDi dataset 제안, 7개 benchmark 중 6개 SOTA	VOT 중심. “negative/distractor memory” 자체는 이미 선행됨
SAMURAI	[정식] IEEE TIP 2026	칼만 운동 단서와 motion-aware mask/memory selection	LaSOT-ext AUC +7.1%, GOT-10k AO +3.5% 보고	선형 운동 가정. “SAM 2 + Kalman”은 독립 주제로는 이미 완료된 아이디어
SAM2Plus	[정식] Sensors 2025	Kalman motion, 동적 품질 threshold, motion·appearance 다중 기준과 제한된 adaptive memory	SA-V·LVOS의 24개 직접 비교 평균 +1.0 J&F, 최대 +2.3 보고	motion+appearance heuristic gate도 이미 선행됨. 각 신호의 독립 기여와 threshold 전이성이 불명확
UAMP	[정식] PLOS ONE 2026	appearance uncertainty, Kalman motion uncertainty, 장·단기 메모리 결합	5개 VOS benchmark에서 SAM 2 개선, 최대 +5.6 J&F 보고	uncertainty와 dual memory 결합도 이미 존재. 확률 보정과 dataset 간 전이는 별도 검증되지 않음
OASAMT	[정식] Pattern Recognition 2026	TCN 기반 가림 상태 추론과 dual-memory bank	mask IoU, 면적 변화, confidence의 시간 패턴으로 가림을 명시적으로 모델링	가림 분류기+dual memory라는 조합도 선점됨
Prompt Self-Correction	[정식] Electronics 2025	object pointer 변화로 오류를 감지하고 particle-filter box와 negative points로 재추론	학습 없이 자동 재프롬프트 가능성을 보임	보고 향상폭은 작음. 단일 heuristic과 고정 임계값의 전이성 문제가 남음
MA-SAM2	[정식] MICCAI 2025	context-aware memory와 occlusion-resilient memory	EndoVis 2017/2018에서 SAM 2 대비 +4.36/+6.1%	수술 영상 특화. 일반 VOS와 같은 조건의 검증이 필요
HiM2SAM	[정식] PRCV 2025 proceedings	Kalman coarse motion + 선택적 CoTracker3 비선형 refinement, 장·단기 메모리	LaSOT/LaSOT-ext에서 SAM 2 대비 상대 AUC +9.6/+7.2% 보고	offline point tracker와 여러 gate가 필요하고 VOT 중심. SAMURAI·UAMP와 큰 축이 겹침

연구	상태	무엇을 바꿨나	핵심 성과/의미	남은 한계와 중복 주의
SAMITE	[프리프린트] 2025	feature/position correctness 기반 prototype memory와 위치 프롬프트 생성	가림·방해물 프레임을 걸러 오류 전파를 차단	“prototype score + position reprompt”도 이미 제안됨. 제목의 calibrated는 확률 calibration과 다름
SAM2RL	[프리프린트/워크숍] 2025	메모리 update를 순차 의사결정으로 보고 강화학습으로 제어	hand-crafted rule을 넘을 가능성 제시	현재 핵심 실험은 영상마다 별도 agent를 과적합한 설정. 영상 간 일반화 policy는 열려 있음
SENTRY	[프리프린트] 2026	mask 후보를 짧은 tracklet으로 되감아 cycle consistency를 확인한 뒤 memory에 기록	5개 SAM2 계열 baseline·9개 benchmark에서 일관된 향상 보고	“refine-before-write”와 temporal consistency 검증까지 선점. 최신 원고라 독립 재현 필요
Lean-SAM2	[프리프린트] 2026	target anchor 기반 memory pruning, insurance memory, 위험 시 full-frame refresh	LVOSv2에서 1.41× 속도 향상과 J&F +5.0/+3.6 보고	risk-aware fallback과 anchor memory가 이미 포함됨. 효율·정확도 수치의 외부 검증 전
SAM2Dual	[프리프린트] 2026-08-19	short/long dual memory와 early-frame text cue 기반 memory reweighting	MOSEv2 J&F 49.33→50.65, LVOSv2 82.1→83.1 보고	고정 주기 장기 memory write가 part-level target에서 오염됨. 논문 자체가 confidence/consistency/event-triggered update를 후속 과제로 제시
IMTrack	[정식] EAAI 2026	신뢰 keyframe으로 재탐색용 implicit memory를 준비하고 loss 감지 시 local tracking에서 global re-detection으로 전환	LaSOT success 최대 +3.0%, LaSOT-ext 최대 +8.8% 보고	box recovery 중심이며 loss detector와 keyframe 품질에 의존. “전역 재탐색 추가” 자체는 이미 선행됨
StreamDAM	[프리프린트] 2026-08-04	학습된 presence signal 하나로 memory write, read horizon, 출력 보류, 재탐색을 함께 제어하고 deadline-aware streaming으로 평가	4개 benchmark·5개 baseline에서 offline 성능 대부분을 회복했다고 보고	매우 최신 단독 저자 원고이며 외부 검증 전. 상태기계+재탐색 통합 자체도 더는 빈 공간이 아님

여기서 얻는 결론

- 단순 FIFO 교체, 장·단기 bank, Kalman filter, confidence threshold, 가림 때 freeze, distractor bank, multi-hypothesis는 각각 이미 논문이 있다.
- 남은 핵심은 어떤 신호를 믿을 것인가, 그 신호가 데이터셋이 바뀌어도 믿을 만한가, 언제 비싼 복구를 실행해야 하는가다.
- 대부분의 방법은 자체 점수 또는 손으로 정한 임계값을 쓰고 최종 평균 J&F로 끝난다. 실패 위험의 **calibration,** 메모리 오염률**,** 가림 사건별 회복 과정은 상대적으로 덜 정리되어 있다.

3.2 프롬프트**,** 언어**,** 의미 기반 재집중

연구	상태	핵심 아이디어	본 과제와의 관계
SAMWISE	[정식] CVPR 2025	frozen SAM 2에 5M 미만 adapter로 언어·시간 정보를 주입하고 tracking bias를 감지해 refocus	의미 단서가 잘못된 객체 고착을 풀 수 있음을 보임. 단, RVOS 과제
MPG-SAM 2	[정식] ICCV 2025	text-video encoder, dense mask prior, global-historical aggregator	전역 문맥과 텍스트를 prompt/memory 양쪽에 넣는 큰 모델. 단순 text prompt 추가는 신규성이 약함
SAM2-LOVE	[정식] CVPR 2025	텍스트와 오디오가 가리키는 객체를 계속 분할	다중 모달 대상 지정의 참고 사례이며 일반 VOS 성능 개선과는 구분해야 함
EntitySAM	[정식] CVPR 2025	영상 속 모든 entity를 자동 발견하고 SAM 2로 추적	자동 프롬프트·새 객체 등장 처리에 참고

연구	상태	핵심 아이디어	본 과제와의 관계
Segment Any Motion	[정식] CVPR 2025	장거리 trajectory와 DINO semantic feature를 SAM 2의 반복 point prompt로 변환	움직임 자체로 prompt를 만들 수 있으나 '주어진 대상 추적'과 '모든 움직이는 객체 분할'은 다른 과제
DC-SAM	[정식] IEEE TPAMI 2026	positive/negative visual prompt와 cycle-consistent cross-attention으로 one-shot/in-context segmentation	첫 프레임 visual exemplar를 재탐색 단서로 사용하는 설계에 참고
SeC	[정식] ICLR 2026	LVLM이 장면을 지나며 object-centric concept prior를 갱신하고 장면 전환 때 의미 특징을 주입	SeCVOS에서 SAM2.1 58.2→70.0 J&F 보고. 강력하지만 4B급 의미 모델의 학습·추론 비용과 동일 범주 instance 구분 한계가 있음
LIT-LoRA	[정식] CVPR 2026	사용자 수정 때 online LoRA를 약 0.5초 학습해 같은 오류의 재발을 줄임	수정으로부터 학습하는 축은 이미 등장. 자동 실패 감지나 '어느 프레임을 수정할지'는 별도 연구 가능
SAM 3	[정식] ICLR 2026	텍스트·이미지 exemplar로 concept를 지정하는 open-vocabulary 검출+분할+추적	SAM 2 연구의 범위 경계. "텍스트를 붙여 zero-shot 대상을 찾는다"만으로는 최신 기준 신규성이 부족함

3.3 다중 객체, **identity**와 객체 간 충돌

연구	상태	핵심 아이디어	남은 연구 공간
SAM2MOT	[정식] AAAI 2026	segmentation 중심 trajectory manager와 cross-object tracking logic	DanceTrack에서 +2.1 HOTA, +4.5 IDF1 보고. 여러 객체를 독립 처리하는 한계를 직접 다룸
SAMURAI++	[정식] CVPR Findings 2026	tracking-by-detection/query 결합, bipartite matching, short-motion/long-instance hierarchical memory	다중 객체 identity 보존과 detection miss 복구까지 포함
SAMIDARE	[워크숍/프리프린트] 2026	밀집 스포츠에서 density-aware mask 재생성, 선택적 memory update, state-aware association	특정 밀집 장면의 mask 충돌 처리 참고

객체 간 상호작용은 여전히 의미 있는 연구축이지만, 이제는 "SAM 2를 여러 객체에 반복 실행"이 아니라 겹침 영역의 소유권, 가림 순서, 동일 외형 객체 사이 **identity** 배타성처럼 더 구체적인 문제를 다뤄야 한다.

3.4 경계, 작은 객체와 세부 구조

연구	상태	핵심 아이디어와 결과	남은 한계
SAM2Refiner	[정식] ICCV 2025	global-local cross-attention, prompt retargeting, 다중 scale cascade로 256→1024 mask refinement. 정적 세부 dataset 평균 SAM2-L 78.5/71.5 mIoU/mBIoU→93.1/88.9	crop·고해상도 head 비용, 시간축 경계 flicker를 직접 최적화하지 않음
HQ-SAM 2	[공식 구현/베타]	기존 HQ-SAM 학습 전략을 SAM 2에 확장. 공개 표에서 DAVIS는 개선되지만 MOSE 향상은 작음	논문 수준의 완결된 검증과 장기·가림 분석이 부족
Quantifying the Limits of Segmentation Foundation Models	[정식] WACV 2026	tree-like 구조와 낮은 texture separability가 SAM/SAM 2/HQ-SAM의 성능을 체계적으로 낮춘다고 분석	진단 논문이며 해결법은 아님. 구조 난이도별 개선 검증의 근거가 됨

연구	상태	핵심 아이디어와 결과	남은 한계
SAM2-WaveUNet	[정식] ESWA 2026	wavelet과 frequency fusion으로 해양 생물의 세부 경계를 복원	정적 이미지·Hiera-L 중심. temporal consistency가 없음

3.5 도메인 적응과 실제 환경 강건성

연구	상태	핵심 아이디어	해석 시 주의점
Biomedical SAM-2	[정식] NeurIPS 2024	Hiera와 mask decoder를 생의학 영상에 미세조정, 2D/3D/video 통합	대규모 domain fine-tuning이며 경량 개선은 아님
SAM2-Adapter	[워크숍] ICLR 2025	frozen Hiera의 각 stage에 adapter를 넣어 위장·그림자·의료 등 prompt-free binary segmentation	raw SAM 2를 prompt 없이 쓴 baseline의 큰 향상폭을 표준 interactive 성능 향상으로 오해하면 안 됨
SAM2-UNet	[정식/프리프린트 계열]	Hiera encoder+adapter+U-Net decoder	memory와 prompt encoder를 제거한 정적 semantic model이므로 SAM 2 비디오 추적 개선은 아님
Medical SAM 2	[프리프린트] 2024	confidence·cosine diversity·현재 frame relevance로 self-sorting memory	대규모 학습비용. memory scoring 아이디어는 참고 가능
SAMed-2	[정식] MICCAI 2025	temporal adapter와 confidence/similarity 기반 memory retrieval/replacement	공식 심사에서도 baseline 공정성과 inference 설명 문제가 지적됨
OFL-SAM2	[정식] AAAI 2026	frozen SAM 2와 online few-shot learner를 adaptive fusion	소수 mask label과 test-time update가 필요
CamSAM2	[정식] NeurIPS 2025	frozen SAM 2에 object-aware fusion과 prototype을 넣어 camouflage video를 분할	위장 객체 특화지만 낮은 대비·identity 단서 연구에 유용
M4-SAM	[정식] CVPR 2026	RGB-D MoE-LoRA, 다중 scale fusion, prompt-free pseudo initialization	깊이 센서가 필요한 별도 task
RobustPVOS / MoGA	[정식] CVPR 2026	안개·비·눈·야간·저조도·모션블러를 포함하는 RobustPVOS benchmark와 object-memory-conditioned adaptation	입력 손상 강건성의 최신 기준. calibration과 recovery policy 자체는 중심이 아님
RUAC	[정식] ICML 2026	SAM2의 mask-level confidence confusion을 지적하고 Bayesian decoder와 uncertainty–accuracy alignment로 domain shift의 pixel uncertainty를 개선	**calibration** 연구의 가장 가까운 인접 선행연구. 정적/개별 mask의 pixel uncertainty가 중심이며, video memory admission과 오류 전파 제어는 다루지 않음

3.6 속도·메모리 효율

연구	상태	핵심 아이디어와 결과	본 과제에 주는 힌트
EdgeTAM	[정식] CVPR 2025	RepViT와 Spatial Perceiver, feature distillation. iPhone 15 Pro Max 16 FPS 보고	대규모 재학습이 필요하므로 비교 대상으로 적합
Efficient Track Anything	[정식] ICCV 2025	lightweight ViT와 효율적 memory module	Hiera-B+ 유사 성능에서 약 1.6× 속도, 2.4× 적은 parameter 보고
Q-MiniSAM2	[정식] IJCAI 2025	post-training quantization, 계층적 video calibration, 유사 frame 간 mutual-KV	양자화 오차와 temporal redundancy를 함께 다룸

연구	상태	핵심 아이디어와 결과	본 과제에 주는 힌트
Efficient-SAM2	[정식] ICLR 2026	object-aware Sparse Window Routing과 Sparse Memory Retrieval	SAM2.1-L에서 SA-V test 정확도 약 1%p 손실로 1.68× speedup 보고
TinySAM 2	[프리프린트] 2026	memory quality 관리, 시공간 token 압축, RepViT	SAM2.1 성능의 90%를 7% memory token과 3% training data로 유지한다고 보고
Lean-SAM2	[프리프린트] 2026	효율화와 anchor/insurance memory, 위험 시 dense fallback을 함께 설계	"빠르면서도 어려운 순간에는 계산을 되돌린다"는 방향의 직접 선행연구

효율화 연구를 택한다면 단순 FPS 비교보다 위험할 때만 **dense** 경로를 쓰는 **adaptive computation**이 더 흥미롭다. 다만 Lean-SAM2와 구분하려면 위험 점수의 calibration이나 명확한 compute budget 최적화가 필요하다.

3.7 추가로 확인한 직접·인접 연구

아래 연구는 최우선 비교군은 아니지만, 주제를 좁힐 때 중복 여부를 반드시 확인해야 한다.

연구	상태·범위	확인해야 할 아이디어
Rethinking Memory Design in SAM-Based VOT	[프리프린트] 2025	memory 설계를 admission/retention/read 관점으로 통합 분석하고 short appearance/long distractor memory를 구분. 원고가 일부 결과를 finalizing 중이라고 밝힘
SurgSLOT	[프리프린트] 2025/26	수술 도구의 semantic identity와 long-term memory로 장기 부재 뒤 재식별
ReSurgSAM2	[정식] MICCAI 2025	text-referred detection으로 신뢰 초기 frame을 찾고 diversity-driven long-term memory 구성
TSMS-SAM2	[정식/공개 원문] 수술 영상	multi-temporal-scale sampling과 memory splitting/pruning으로 빠른 운동과 중복 memory 대응
SAM2Text	[정식] CVPR 2026, 장면 문자	self-prompting, 256/512/1024 decoder, short-term+top-k memory. 범용 객체가 아닌 text segmentation
AL-Ref-SAM 2	[정식] AAAI 2025, audio/language RVOS	GPT가 pivot frame/box를 고르고 Grounding DINO와 SAM 2로 전파. 초기 prompt frame 선택 연구에 참고
CoT-RVS	[정식] ICLR 2026, reasoning VOS	MLLM이 CoT로 keyframe·description을 고르고 SAM 2가 양방향 전파. training-free지만 대형 모델 필요
Refer-Agent	[정식] CVPR 2026, RVOS	여러 agent의 reasoning/reflection으로 대상 후보와 empty-mask 가능성을 검증
StreamingRVOS	[정식] CVPR 2026, RVOS	mask가 불안정할 때만 MLLM의 semantic embedding을 갱신하는 event-triggered 의미 보정
MSRTrack	[정식] WACV 2026, VOT	VLLM이 구별 속성을 추출하고 motion·semantic reasoning으로 대상 소실 복구와 distractor 거절
OpenWorldSAM	[정식] NeurIPS 2025, 이미지	frozen SAM 2와 VLM에 소형 adapter를 붙인 language-prompt universal image segmentation. temporal identity는 미검증
SAM-I2V	[정식] CVPR 2025, **SAM 1** 확장	memory filtering과 memory-as-prompt를 적은 학습비용으로 구현. SAM 2 직접 개조로 분류하면 안 됨

연구	상태·범위	확인해야 할 아이디어
SAMEO	[정식] CVPR 2025, 정적 amodal segmentation	가려져 보이지 않는 부분까지 형태를 예측. 시간 memory나 재등장 tracker는 아님
UncertainSAM	[정식] ICML 2025, SAM 계열 uncertainty	prompt·model·task ambiguity를 분리한 경량 post-hoc uncertainty. video memory action으로 연결하는 참고점

이 목록의 역할은 모델을 모두 구현하라는 뜻이 아니다. 문제 정의가 겹치는지 확인하고, 직접 비교할 **3~5**개 방법을 고르는 용도다.

---

4. 선행연구가 이미 차지한 영역과 남은 공백

아이디어	포화도	대표 선행연구	그대로 하면 약한 이유	아직 가능한 좁은 질문
FIFO를 장·단기 메모리로 변경	매우 높음	UAMP, HiM2SAM, OASAMT, SAMURAI++, SAM2Dual	구조 자체가 반복됨	고정 주기가 아닌 확률적으로 보정된 **write** 정책
칼만 필터/운동 점수	매우 높음	SAMURAI, UAMP, HiM2SAM	구현이 단순해 이미 여러 논문이 사용	비선형 운동에서 언제 motion을 버려야 하는지 calibration
predicted IoU가 높을 때만 저장	높음	SAM2Long, DAM4SAM, MA-SAM2, SAMITE	내부 점수가 실제 정확도와 맞는다는 가정	dataset shift에서 confidence–accuracy 불일치를 측정·보정
여러 mask 후보 유지	높음	SAM2Long, SENTRY	tree/tracklet 방식이 이미 강함	후보 수를 위험과 계산 예산에 따라 동적으로 결정
오류 시 자동 재프롬프트·전역 재탐색	높음	Prompt Self-Correction, SAMITE, IMTrack, StreamDAM	heuristic trigger, local/global 전환과 위치 생성이 이미 있음	보정된 위험의 품질과 안전한 **abstention**을 중심 질문으로 설정
distractor/negative memory	높음	DAM4SAM	유사 객체 특화 memory가 이미 있음	다중 객체 간 배타적 mask ownership/occlusion graph
text로 대상 재확인	중상	SAMWISE, MPG-SAM2, SAM2Dual, SAM 3	open-vocabulary 자체는 최신 모델이 강함	instance-level visual anchor와 category text의 충돌 해소
고해상도 경계 head	중상	HQ-SAM2, SAM2Refiner	공간 refinement만으로는 차별화 부족	optical flow 정렬 후 시간축 **boundary consistency**
corruption/domain adapter	중상	SAM2-Adapter, MoGA, 각종 의료 모델	특정 dataset fine-tuning 논문이 많음	unseen corruption에서 uncertainty calibration과 safe memory
상태 기반 통합 memory controller	중상	OASAMT, StreamDAM	보임/불확실/부재 상태와 행동 연결이 이미 제안됨	dataset 간 calibration과 위험 상한을 갖는 정책
학습형 memory controller	낮음~중간	SAM2RL	일반화하는 policy는 아직 미완성	여러 영상에 통하는 constrained/offline RL controller

가장 명확한 연구 공백

공백 **A — SAM 2** 내부 신뢰도의 확률적 보정

대부분은 predicted IoU, object-presence score, mask stability를 그대로 쓰거나 고정 임계값으로 자른다. 그러나 “score 0.8”이 실제로 “정확할 확률 80%”인지, MOSE에서 정한 threshold가 LVOS나 악천후 영상에서도 통하는지는 별개다. ICML 2026 RUAC가 domain shift에서 SAM2의 mask-level confidence confusion을 직접 보였지만, 주 대상은 정적 mask의

pixel uncertainty이며 그 불확실성을 **video memory write**와 미래 **drift** 위험에 연결하는 문제는 남는다.

공백 **B —** 평균 **J&F**가 숨기는 실패와 복구 과정

같은 J&F라도 한 방법은 잠깐 틀렸다 바로 복구할 수 있고, 다른 방법은 한 번 drift한 뒤 끝까지 실패할 수 있다. 가림 시작, 재등장, identity switch를 사건 단위로 나눈 평가가 필요하다.

공백 **C —** 위험에 비례한 계산과 행동

모든 프레임에서 tree search나 global detector를 실행하면 비싸다. 평상시에는 SAM 2.1을 그대로 쓰고, 위험이 커질 때만 후보 검증·재탐색을 켜는 정책이 필요하다. 다만 StreamDAM과 Lean-SAM2가 이미 event/risk-based 호출을 제안했으므로, **event-trigger** 자체가 아니라 잘 보정된 위험도와 정확도–계산비용 보장이 연구의 차별점이어야 한다.

---

5. 최우선 추천 연구안

5.1 제목 후보

한국어

교차 데이터셋 신뢰도 보정에 기반한 SAM 2.1의 선택적 메모리 갱신과 오류 누적 억제

영어

Cross-Dataset Calibrated Risk Control for Selective Memory Updates in Long-Term Video Object Segmentation with SAM 2.1

5.2 한 문장 연구 주장

> SAM 2.1의 원시 내부 점수를 그대로 신뢰하지 않고 현재 또는 가까운 미래의 실제 실패 확률로 보정한 뒤, 그 위험에 따라 메모리 기록을 선택하면, 고정 임계값 방법보다 데이터셋이 바뀌어도 메모리 오염과 장기 drift를 더 안정적으로 줄일 수 있다.

5.3 연구 질문

- **RQ1.** predicted IoU, object-presence score, mask stability는 실제 mask 오류를 얼마나 잘 예측하며 dataset이 바뀌면 얼마나 miscalibrated 되는가?
- **RQ2.** 여러 신호를 결합해 보정한 실패 위험이 단일 confidence보다 잘못된 memory write를 더 잘 막는가?
- **RQ3.** 보정된 위험에 따라 memory write를 보류하면 미래 5~10프레임 안의 drift 확률과 가림·재등장 후 recovery가 개선되는가?
- **RQ4.** 한 dataset에서 정한 정책이 다른 dataset과 영상 손상 조건에서도 유지되는가?
- **RQ5.** 정확도 향상에 필요한 추가 계산량은 얼마이며, 항상 실행하는 multi-hypothesis 방식보다 효율적인가?

5.4 입력 신호

처음부터 모든 신호를 넣지 말고 다음 순서로 ablation한다.

1. **SAM 2** 내부 신호
    - predicted IoU
    - object-presence logit
    - mask stability score
    - 3개 mask 후보의 score margin과 mask 간 disagreement
2. 시간 일관성 신호
    - 이전 mask와 현재 mask의 IoU
    - mask 면적·중심점·종횡비 변화율
    - 단순 constant-velocity 예측과 현재 mask의 residual
3. 정체성 신호
    - 현재 object pointer와 첫 프레임/신뢰 anchor pointer의 cosine similarity
    - 선택 사항: DINOv2 feature prototype과 현재 대상 feature의 거리
4. 비싼 검증 신호 **—** 위험 **frame**에서만
    - forward/backward propagation consistency
    - optical-flow warp 후 mask IoU
    - 2~3개 후보의 짧은 tracklet consistency

5.5 신뢰도 보정

정답 mask가 있는 개발 split에서 두 종류의 실패 label을 만든다. 현재 frame 실패는 `IoU < 0.5`, identity switch, 또는 대상이 있는데 empty mask인 경우다. 미래 위험은 현재 **mask**를 **memory**에 쓴 뒤 **5~10**프레임 안에 지속 실패가 발생하는지로 정의한다. 후자가 단순 mask 품질 예측을 넘어 memory admission 연구로 만드는 핵심이다.

비교할 방법은 다음이면 충분하다.

- raw score 또는 수작업 weighted sum
- logistic regression
- temperature scaling
- isotonic regression
- 선택 사항: conformal risk control로 허용 memory contamination risk α에 맞춘 threshold

중요한 것은 복잡한 network가 아니라 **held-out** 영상과 다른 **dataset**에서 **calibration**이 유지되는지, 그리고 정한 risk budget을 실제로 지키는지다. frame을 무작위로 섞어 train/test하면 같은 영상의 인접 frame이 누수되므로, 반드시 video 단위로 분할한다.

5.6 3상태 제어기

이 제어기는 실험을 수행하기 위한 최소 구현이다. OASAMT와 StreamDAM 때문에 **3**상태 구조 그 자체를 논문의 주 기여로 내세우지는 않는다**.** 동일한 상태 구조에서 raw threshold와 calibrated risk를 바꿔 성능 차이를 검증하는 것이 목적이다.

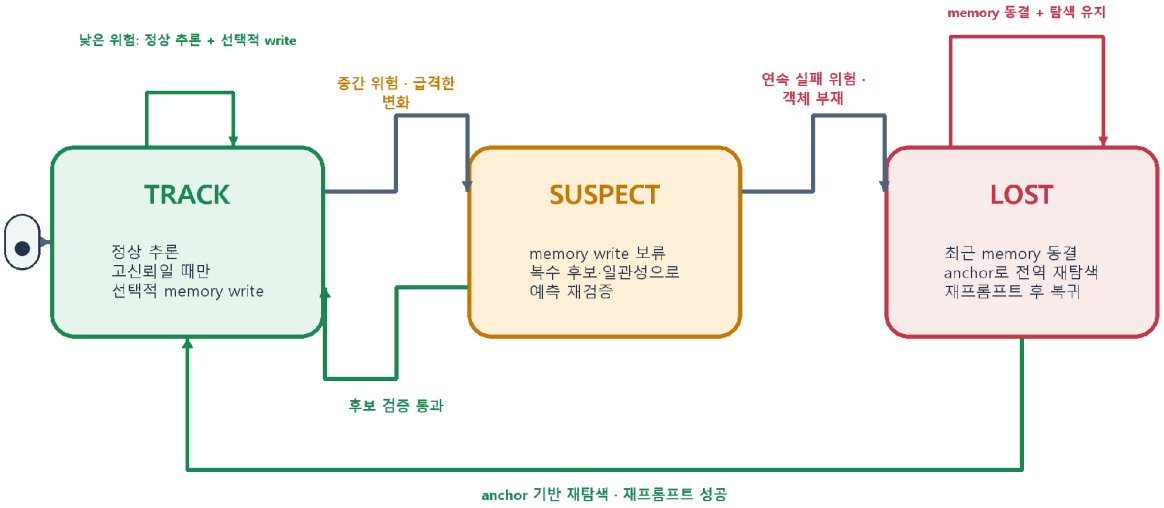


그림 *2.* 보정된 실패 위험에 따른 ***TRACK–SUSPECT–LOST*** 상태 제어

- **TRACK**: SAM 2.1 정상 추론. 보정된 성공확률이 높고 anchor와 충분히 일치할 때만 자동 예측을 memory에 쓴다.
- **SUSPECT**: 현재 예측을 memory에 쓰지 않는다. 복수 후보 또는 짧은 양방향 consistency로 후보를 재검증한다.
- **LOST**: 최근 prediction memory를 동결한다. 첫 프레임과 고신뢰 anchor를 이용해 global 후보를 찾고 box/mask prompt로 SAM 2.1을 다시 시작한다.

초기 과제에서는 short/long bank를 새로 설계하지 않아도 된다. 기본 **memory**의 **write** 허용/보류와 **anchor slot** 몇 개만으로 시작해야 calibration 효과를 분리해 볼 수 있다.

5.7 이 연구가 기존 방법과 다른 지점

기존 방법	주요 기준	제안 연구의 구분점
SAM2Long	predicted IoU·occlusion score로 경로 누적	score 자체의 calibration과 dataset 간 전이를 평가
DAM4SAM	distractor introspection과 두 memory	일반 실패 확률과 memory 오염의 정량 관계를 모델링
SAMURAI/UAMP	motion+appearance uncertainty	motion model 추가가 아니라 여러 신호의 확률적 신뢰도 정렬이 중심
Prompt Self-Correction	pointer EMA와 particle-filter 재프롬프트	고정 heuristic 대신 보정된 위험과 상태에 따른 행동 선택
SENTRY	tracklet cycle consistency로 write 검증	모든 검증 신호를 동일하게 쓰지 않고 위험도·예산에 따라 단계적으로 호출
Lean-SAM2	anchor 기반 pruning과 risk-aware dense refresh	encoder 효율이 아니라 failure probability와 recovery outcome이 중심
SAM2Dual	고정 주기 dual memory와 text reweighting	고정 주기 write가 아니라 calibrated event-triggered write; 논문이 밝힌 failure case를 직접 공략
StreamDAM	learned presence 하나로 write/read/abstain/re-detect 통합	상태 제어가 아니라 **score calibration, cross-dataset transfer, risk budget** 준수가 중심
RUAC	domain shift에서 pixel-wise uncertainty–accuracy alignment	정적 mask uncertainty를 video의 memory admission과 미래 drift 확률로 연결

중요: “calibration-aware controller가 최초”라고 미리 단정하면 안 된다. 2026년 프리프린트가 빠르게 추가되고 있으므로, 구현 범위를 확정한 날에 제목·초록·코드 수준으로 novelty search를 한 번 더 해야 한다. 현재 문헌 기준으로 가장 방어하기 쉬운 기여는 새 상태기계를 만들었다는 주장이 아니라, 미래 **memory contamination** 위험의 보정**, dataset** 간 전이**,** 정해진 위험 상한 아래의 성능 향상이다.

---

6. 실험 설계

6.1 기준 모델과 비교군

최소 비교군:

1. SAM 2.1 동일 크기 checkpoint
2. SAM 2.1 + raw confidence threshold memory gate
3. SAM 2.1 + calibrated risk memory gate
4. SAM 2.1 + calibrated gate + event recovery
5. 공개 구현이 안정적이면 SAM2Long
6. 연구축에 따라 SAMURAI 또는 DAM4SAM 중 하나

모든 방법에서 checkpoint, 영상 해상도, 첫 프레임 prompt 형식, 객체 수 처리 방식, forward/backward propagation 여부를 고정해야 한다.

6.2 데이터셋

데이터셋	쓰는 이유	권장 용도
MOSEv2	가림, 소형·밀집 객체, 재등장, 저조도, 악천후, multi-shot, camouflage가 포함된 매우 어려운 현실 조건	핵심 평가. 전체가 크면 validation의 사건 표본을 층화 추출
LVOS v2	긴 영상과 큰 시간 간격, 재등장	장기 drift와 threshold 전이 평가
DiDi	유사 방해물이 많은 단일 객체 tracking	identity switch와 false reacquisition 분석
RobustPVOS	실제/합성 악천후·저조도·blur	domain shift calibration 실험의 선택 dataset
Long-RVOS	2,193개 장기 referring 영상, 가림·재등장·shot change와 temporal/video IoU 제공	텍스트 확장 또는 장기 사건 평가의 선택 dataset. 기본 VOS와 prompt protocol은 구분
DAVIS 2017	작고 익숙하며 평가 도구가 안정적	구현 sanity check. 최종 성능 주장은 DAVIS만으로 하지 않음
SA-V	SAM 2의 공식 대규모 분포	여유가 있으면 val subset. full train은 필수 아님

추천 조합은 **MOSEv2 + LVOS v2**, 그리고 DAVIS는 디버깅 전용이다. calibration은 MOSE 일부에서 학습하고 LVOS에서 그대로 평가한 뒤, 반대 방향도 해보는 cross-dataset 실험이 핵심이다.

6.3 성능 지표

표준 **mask** 품질

- Region Jaccard `J`
- Boundary F-measure `F`
- `J&F = (J + F) / 2`

실패 검출과 **calibration**

- failure AUROC, AUPRC
- Expected Calibration Error(ECE)
- Brier score
- risk–coverage curve: 위험 frame을 보류할수록 남은 prediction 정확도가 어떻게 변하는가

시간축과 복구

- memory contamination rate: 실제 IoU가 기준 이하인 mask가 memory에 기록된 비율
- drift rate 또는 identity switch rate
- disappearance/reappearance event별 recovery success
- frames-to-recover: 재등장 후 정상 IoU로 돌아오는 데 걸린 frame 수
- false reacquisition: 다른 유사 객체를 목표로 잘못 복구한 비율
- longest failure streak

비용

- FPS와 영상당 처리 시간
- peak VRAM
- SUSPECT/LOST trigger 비율
- 추가 후보 평가 횟수, global recovery 호출 횟수

6.4 핵심 **ablation**

실험	답하려는 질문
predicted IoU만 사용	SAM 2 내부 품질 head만으로 충분한가?
+ object score/stability	내부 신호 조합이 실패 검출을 높이는가?
+ mask 후보 disagreement	모호성을 명시하면 위험 예측이 좋아지는가?
+ temporal change	갑작스러운 motion/면적 변화가 추가 정보를 주는가?
+ anchor similarity	유사 객체로의 identity jump를 잡는가?
raw threshold vs calibrated threshold	확률 보정이 실제 policy를 개선하는가?
write gate만	메모리 오염 차단만으로 얻는 이득은 얼마인가?
recovery만	재탐색 자체의 효과는 얼마인가?
gate + recovery	둘의 상호보완 효과가 있는가?
fixed vs event-triggered expensive check	같은 계산 예산에서 어느 쪽이 좋은가?

6.5 통계 검증

- frame별 표본을 독립이라고 가정하지 말고 **sequence** 단위 **paired bootstrap 95%** 신뢰구간을 사용한다.
- 가능하면 sequence별 J&F 차이에 Wilcoxon signed-rank test를 추가한다.
- recovery metric은 가림/재등장 **event** 단위 **bootstrap**을 쓴다.
- threshold와 calibration model을 평가 split에 맞춰 조정하지 않는다.

6.6 성공 기준 예시

다음은 연구 시작 전에 정하는 목표이지, 결과를 보장하는 수치가 아니다.

- raw score 대비 failure AUPRC와 ECE가 유의하게 개선
- SAM 2.1 대비 MOSE/LVOS의 J&F 개선
- memory contamination rate 20% 이상 상대 감소
- 재등장 사건의 median recovery delay 감소
- false reacquisition 증가 없이 개선
- 전체 frame 중 20% 미만에서만 비싼 검증을 호출해 계산 증가 제한

---

7. 차선 연구안 세 가지

연구안 **B —** 소형·얇은 객체의 시간 일관적 경계 보정

질문**:** SAM2Refiner식 공간 refinement 뒤에도 남는 frame 간 boundary flicker를 motion-aligned loss 또는 inference-time refinement로 줄일 수 있는가?

- 입력: SAM 2.1 mask logit, Hiera의 고해상도 중간 feature, 이전 frame boundary
- 방법: optical flow로 이전 boundary를 warp하고, uncertainty가 큰 좁은 band만 작은 refinement head로 재예측
- 평가: mBIoU, Boundary F, temporal boundary consistency, small-object J&F, FPS
- 비교: SAM 2.1, HQ-SAM2, SAM2Refiner, 단순 morphology/CRF
- 장점: 결과 시각화가 분명하고 실패 원인이 구체적
- 위험: SAM2Refiner 학습 비용이 크므로 전체 모델 복제 대신 작은 boundary head만 설계해야 함

연구안 **C —** 악조건에서 안전한 **memory**

질문**:** 밝기·blur·비/안개가 급변할 때 corruption-aware uncertainty로 memory write를 막고 clean anchor를 보존하면 RobustPVOS 성능이 오르는가?

- CVPR 2026 RobustPVOS의 corruption benchmark 사용
- brightness, contrast, blur, rain severity와 SAM 2 confidence 오류의 관계 분석
- 작은 corruption encoder 또는 통계량으로 위험을 추정해 memory gate
- 핵심은 전처리로 화면을 밝게 만드는 것이 아니라 나쁜 관측이 장기 **memory**를 오염시키지 않게 하는 것
- clean→corrupt→clean 구간의 recovery를 별도 측정

연구안 **D —** 다중 객체 가림 그래프

질문**:** 각 객체를 독립 처리하는 SAM 2.1에 mask overlap, depth/area ordering, object-pointer exclusivity를 이용한 occlusion graph를 추가하면 유사 객체의 identity switch를 줄일 수 있는가?

- node: 객체 track, edge: 겹침·가림 가능성
- 동일 pixel을 여러 객체가 차지할 때 confidence만으로 합치지 않고 가림 순서와 identity anchor로 소유권 결정
- SAM2MOT·SAMURAI++와 직접 비교가 필요
- 신규성 잠재력은 높지만 구현과 annotation 분석 난도가 가장 높음

선택표

연구안	신규성 잠재력	구현 난도	결과 해석	학기 과제 적합도
A. Calibration-aware memory/recovery	높음	중	매우 좋음	**1**순위
B. Temporal boundary refinement	중상	중	매우 좋음	**2**순위
C. Corruption-safe memory	중상	중	좋음	**3**순위
D. Multi-object occlusion graph	높음	높음	좋음	도전적 선택
RL general memory controller	매우 높음	매우 높음	불안정	학기 과제로는 비추천

---

8. 12주 실행 계획

주차	목표	산출물/중단 기준
1	SAM 2.1 inference와 DAVIS evaluator 재현	공식 수치와 큰 차이가 나면 환경부터 수정
2	MOSEv2/LVOS loader와 sequence별 평가	baseline J/F/J&F 표
3	내부 score·pointer·mask 후보 logging	frame별 feature/score/GT IoU 데이터
4	실패 사건 정의와 시각화	가림·재등장·drift 사례 20개 이상
5	raw score failure detector	AUROC/AUPRC/ECE baseline
6	현재 오류·미래 drift 위험의 logistic/temperature/isotonic calibration	video-held-out와 cross-dataset 비교
7	selective memory write/freeze 구현	contamination rate 변화
8	TRACK/SUSPECT/LOST controller	상태 전이 로그와 정성 예시
9	anchor 기반 recovery 1차	recovery success/delay
10	전체 ablation과 계산 비용	정확도–비용 Pareto
11	bootstrap·오류 유형별 분석	표, 그래프, failure cases
12	보고서·발표 자료	재현 명령, 설정, 최종 결론

최소 완성선

시간이 부족하면 global re-detection까지 억지로 넣지 않는다. 다음 세 가지만 완성해도 연구로 성립한다.

1. SAM 2 내부 신뢰도의 dataset별 calibration 분석
2. 보정된 확률에 따른 selective memory write
3. J&F뿐 아니라 memory contamination과 recovery event 평가

---

9. 논문을 읽을 권장 순서

1. SAM 2 원 논문 — 구조와 원래 한계
2. 공식 SAM 2.1 코드 — 실제 baseline과 출력 신호
3. SAM2Long — 오류 누적과 multi-hypothesis
4. DAM4SAM — distractor와 memory update
5. SAMURAI — motion-aware memory
6. SENTRY — consistency-validated write
7. Lean-SAM2 — anchor, 위험 기반 fallback, 효율
8. SAM2Dual — 가장 최신 dual-memory 결과와 명시된 실패 사례
9. StreamDAM — presence-aware write/abstain/re-detection 통합과 연구 중복 경계
10. RUAC — domain shift에서의 SAM2 uncertainty–accuracy 정렬
11. SAM2Refiner — mask 품질·경계 축
12. RobustPVOS — 실제 악조건과 domain shift

---

10. 연구계획서에 바로 쓸 수 있는 초안

연구 필요성

SAM 2.1은 첫 프레임에서 지정된 객체를 후속 프레임으로 전파하는 강력한 비디오 분할 모델이지만, 가림, 유사 객체, 빠른 운동, 재등장 상황에서 잘못된 mask가 memory에 기록되면 오류가 장기간 누적될 수 있다. 기존 연구는 장·단기 memory, motion cue, distractor-aware memory, 복수 후보 탐색, 상태 기반 재탐색 등을 통해 이 문제를 완화했으나, 많은 방법이 SAM 2의 내부 confidence를 그대로 사용하거나 데이터셋별 고정 임계값에 의존한다. 따라서 내부 confidence가 현재 mask 오류와 이후 drift 확률을 얼마나 정확히 나타내는지와, 보정된 위험을 이용한 memory 제어가 새로운 영상 분포에서도 안정적으로 작동하는지를 연구할 필요가 있다.

연구 목적

본 연구는 SAM 2.1이 출력하는 predicted IoU, object-presence score, mask stability와 시간적·정체성 일관성 신호를 결합해 현재 오류와 가까운 미래 drift 확률을 추정하고 이를 calibration한다. 이후 보정된 위험에 따라 신뢰도가 낮은 mask의 memory 기록을 막고, 선택적으로 후보 검증 또는 복구 절차를 실행한다. 이를 통해 장기 영상에서의 memory 오염과 오류 누적을 줄이는 것을 목표로 한다.

예상 기여

1. SAM 2.1 내부 confidence의 video-held-out 및 cross-dataset calibration 분석

2. 현재 mask 품질이 아니라 미래 drift까지 예측하는 memory admission risk 정의
3. 보정된 위험에 기반한 선택적 memory write와 risk-budget 제어
4. 평균 J&F를 넘어 memory contamination, failure detection, 가림 후 recovery를 측정하는 평가 프로토콜
5. 정확도 향상과 추가 계산량 사이의 정량적 trade-off 분석

---

11. 조사 범위와 주의사항

- 이 문서는 2026-08-20까지 공개된 **SAM 2** 직접 개선 연구와 설계에 영향을 주는 인접 연구를 폭넓게 정리한 것이다. 매일 새 프리프린트가 나오는 분야이므로 문자 그대로 모든 응용 논문을 포함한 것은 아니다.
- 같은 이름에 SAM이 들어가도 SAM 1 기반인 연구는 직접 선행연구에서 제외했다.
- 서로 다른 논문의 성능 수치는 checkpoint, prompt 형식, 해상도, dataset split, multi-object 처리 방식이 다르므로 수치만으로 가로 비교하지 않는다.
- 첫 프레임 정답 mask를 쓰는 VOS 결과와 detector가 만든 box, 텍스트 prompt, prompt-free 결과를 같은 표에서 순위처럼 비교하면 안 된다.
- 최종 주제 확정 직전에 Google Scholar/arXiv/CVF/IEEE에서 핵심 구문을 다시 검색해 2026년 하반기 중복을 점검해야 한다.

처음부터 배우는 SAM 2 완전 이해 가이드

처음부터 배우는 SAM 2 완전 이해 가이드

> 이 탭의 목적: 컴퓨터비전 지식이 전혀 없는 상태에서 출발해, 옆 탭의 선행연구 문서에 나오는 과제·모델 구조·메모리·실패 원인·평가지표·신뢰도 보정·실험 설계를 스스로 설명할 수 있는 수준까지 올라가는 것이다.

이 탭을 다 읽었을 때 할 수 있어야 하는 것

- classification, detection, segmentation, tracking의 차이를 출력 형태로 구분한다.
- SAM 2가 현재 프레임, prompt, 과거 memory를 어떻게 합쳐 mask를 만드는지 단계별로 설명한다.
- occlusion 뒤 drift와 identity switch가 왜 생기며, 틀린 mask의 memory 기록이 왜 위험한지 설명한다.
- SAM2Long, DAM4SAM, SAMURAI, UAMP, SENTRY, Lean-SAM2 같은 선행연구가 각각 어떤 문제를 이미 다뤘는지 구분한다.
- IoU, Dice, J, F, J&F를 직접 계산하고 평균 점수가 숨기는 실패 사건을 찾는다.
- confidence, uncertainty, calibration을 구분하고 ECE, Brier, AUROC, AUPRC가 무엇을 측정하는지 말한다.
- baseline, ablation, data leakage, cross-dataset test, bootstrap confidence interval을 포함한 공정한 실험을 설계한다.
- 최종 제안인 calibration-aware selective memory와 TRACK/SUSPECT/LOST controller가 무엇을 새롭게 검증하려는지 설명한다.

가장 효과적인 읽는 순서

1. PART A에서 이미지가 숫자가 되는 과정과 컴퓨터비전 과제를 익힌다.
2. PART B에서 신경망, Transformer, promptable foundation model의 최소 원리를 익힌다.
3. PART C에서 SAM 2의 구조와 memory loop를 한 frame씩 따라간다.
4. PART D에서 가림·방해물·저조도·작은 객체가 어떤 오류를 만드는지 이해한다.
5. PART E에서 평가·통계·재현성의 기준을 익힌다.
6. PART F에서 이 과제연구의 연구 질문, controller, 실험표를 다시 조립한다.
7. 마지막 용어사전과 자가점검 문제로 이해가 빈 곳을 확인한다.

처음 읽을 때 지켜야 할 세 가지

- 약어를 외우기 전에 입력, 출력, 정답이 무엇인지 먼저 확인한다.
- 논문의 성능 숫자를 보기 전에 prompt 조건, checkpoint, dataset, metric이 같은지 확인한다.

- 평균 점수가 올랐는지만 보지 말고 어떤 실패가 줄었고 비용이 얼마나 늘었는지 확인한다.



그림 *1.* 분류에서 영상 객체 분할까지: 모델에게 묻는 질문과 출력의 정밀도가 어떻게 달라지는지 보여 준다.

전체 내용을 한 문장으로 먼저 보기

SAM 2는 사용자가 지정한 객체를 현재 영상에서 분할하고 과거 결과를 memory로 저장해 다음 frame을 돕지만, 틀린 결과가 memory에 들어가면 오류가 자기증폭될 수 있으므로, 내부 점수를 실제 미래 실패확률에 맞게 calibration하고 위험도에 따라 memory 기록·검증·복구 행동을 다르게 하자는 것이 옆 탭 연구안의 핵심이다.

PART A — 이미지와 컴퓨터비전 과제의 기초

1. AI, 머신러닝, 딥러닝, 컴퓨터비전

1.1 인공지능 — Artificial Intelligence, AI

한 줄 정의: 사람이 지능적으로 한다고 여기는 판단, 계획, 인식, 언어 이해 등을 컴퓨터가 수행하게 만드는 넓은 분야다.

AI는 가장 큰 우산이다. 예를 들어 정해진 규칙만 따라 체스를 두는 프로그램도 AI라고 부를 수 있다. 반드시 데이터를 학습해야 AI인 것은 아니다.

비유: “이동수단”이라는 큰 범주 안에 자동차, 자전거, 기차가 들어가는 것과 같다.

예시

- 사진을 보고 고양이인지 판단한다.
- 질문을 읽고 답을 생성한다.
- 자율주행차가 보행자를 발견하고 멈춘다.
- 게임에서 다음 행동을 계획한다.

1.2 머신러닝 — Machine Learning, ML

한 줄 정의**:** 사람이 모든 규칙을 직접 작성하지 않고, 데이터의 예시에서 규칙이나 패턴을 학습하게 하는 AI 방법이다.

전통적인 규칙 기반 프로그램은 사람이 "귀가 뾰족하고 수염이 있으면 고양이" 같은 규칙을 적는다. 머신러닝은 고양이와 개 사진을 많이 보여 주고 둘을 구분하는 기준을 데이터에서 찾게 한다.

비유**:** 정답 풀이법을 통째로 알려 주는 대신 많은 문제와 답을 주고 학생이 공통 규칙을 발견하게 하는 것과 같다.

1.3 딥러닝 — Deep Learning, DL

한 줄 정의**:** 여러 층으로 이루어진 인공신경망을 사용해 데이터에서 복잡한 표현을 단계적으로 학습하는 머신러닝의 한 종류다.

초기 층은 선과 색 변화 같은 단순 특징을 찾고, 뒤쪽 층은 눈, 바퀴, 사람의 형태처럼 더 복잡한 특징을 조합한다. 현대 이미지 분류, 객체 탐지, 영상 분할 대부분이 딥러닝을 사용한다.

왜 **deep**인가**?** 입력과 출력 사이에 학습되는 층이 많이 쌓여 있기 때문이다. "깊다"는 말이 모델이 인간처럼 깊게 생각한다는 뜻은 아니다.

1.4 컴퓨터비전 — Computer Vision, CV

한 줄 정의**:** 컴퓨터가 이미지와 영상에서 의미 있는 정보를 얻도록 만드는 분야다.

컴퓨터비전은 "문제 분야"이고, 딥러닝은 그 문제를 푸는 "방법" 중 하나다. 컴퓨터비전에는 딥러닝 이전의 윤곽선 검출, 색상 분석, optical flow 같은 전통 알고리즘도 포함된다.

관계 정리

인공지능 AI ⊃ 머신러닝 ML ⊃ 딥러닝 DL

컴퓨터비전은 이들과 겹치는 응용 분야다. 컴퓨터비전 문제를 딥러닝으로 풀 수도 있고 전통 영상처리로 풀 수도 있다.

하나의 예로 연결하기**:** 재활용 로봇이 컨베이어 벨트 위 물체를 처리한다고 하자.

- AI: 로봇이 상황을 보고 행동을 결정하는 전체 시스템
- ML: 플라스틱과 캔의 차이를 데이터에서 학습
- DL: 여러 층의 신경망으로 모양과 재질 특징을 학습
- CV: 카메라 영상에서 캔의 위치와 윤곽을 찾는 부분

2. 이미지와 영상은 어떻게 숫자가 되는가

2.1 픽셀 **— Pixel**

한 줄 정의**:** 디지털 이미지를 이루는 가장 작은 격자 칸이다.

사진을 아주 크게 확대하면 작은 정사각형들이 보인다. 각 칸에는 그 위치의 색 또는 밝기를 나타내는 숫자가 들어 있다. 픽셀 하나 자체에는 “사람의 눈”이나 “자동차 바퀴”라는 의미가 없다. 주변 픽셀과 함께 해석될 때 물체의 일부가 된다.

비유**:** 모자이크 벽화의 타일 한 조각이다. 타일 하나만 보면 의미가 없지만 수천 개를 배열하면 그림이 된다.

2.2 RGB와 채널 **— RGB, Channel**

컬러 이미지는 보통 빨강 Red, 초록 Green, 파랑 Blue 밝기를 합쳐 표현한다. 이 세 장의 숫자 지도를 RGB channel이라고 한다.

8-bit 이미지에서는 각 값이 보통 0부터 255까지다.

- 검정: (0, 0, 0)
- 흰색: (255, 255, 255)
- 순수한 빨강: (255, 0, 0)
- 노랑: (255, 255, 0)

모델에 넣기 전에는 0~255를 0~1로 나누거나, 평균과 표준편차를 이용해 정규화 normalization하기도 한다.

비유**:** 투명한 빨강 필름, 초록 필름, 파랑 필름 세 장을 겹쳐 컬러 사진을 만드는 것과 같다.

입력 채널과 특징 채널은 다르다

- 입력 RGB 이미지: 3 channels
- 흑백 의료 영상: 흔히 1 channel
- 깊이까지 포함한 RGB-D: RGB 3개와 depth 1개
- 신경망 중간 feature: 64, 256, 1024 channels처럼 많아질 수 있음

중간 feature channel은 실제 색이 아니다. 어떤 채널은 세로 경계에, 다른 채널은 특정 질감이나 물체 부분에 강하게 반응하는 학습된 숫자 지도다.

자주 하는 오해**:** channel 수와 class 수는 같지 않다. 256 feature channels를 쓴다고 256개 종류를 분류하는 것이 아니다.

2.3 해상도 **— Resolution**

한 줄 정의**:** 이미지의 가로와 세로 픽셀 수다.

1920×1080 영상은 가로 1920, 세로 1080이며 한 frame에 약 207만 픽셀이 있다. 해상도가 높으면 작은 물체와 가는 경계를 더 많이 담을 수 있지만 계산량과 메모리 사용량도 커진다.

주의**:** 원본이 저해상도인데 단순히 네 배 확대한다고 새로운 세부 정보가 생기는 것은 아니다. 기존 픽셀 사이를 보간 interpolation한 것일 뿐이다.

이미지 해상도와 ***feature map*** 해상도

원본이 1024×1024여도 encoder를 지나며 feature가 256×256, 64×64처럼 줄어들 수 있다. 이 과정을 downsampling이라고 한다. 계산은 줄지만 작은 물체나 1~2픽셀짜리 경계가 사라질 수 있다. SAM 2의 경계 개선 논문들이 high-resolution feature와 decoder를 추가하는 이유다.

2.4 프레임과 FPS — Frame, Frames Per Second

frame: 영상을 이루는 한 장의 정지 이미지다.

FPS: 1초에 몇 frame을 보여 주거나 처리하는지를 뜻한다. 30 FPS 영상 10초는 약 300 frames다.

영상에는 공간 정보뿐 아니라 시간 정보가 있다.

- 공간 spatial: 한 frame 안에서 어디에 무엇이 있는가?
- 시간 temporal: 앞뒤 frame 사이에 무엇이 어떻게 움직였는가?

주의**:** 영상 자체가 30 FPS라고 모델도 30 FPS로 처리한다는 뜻은 아니다. 모델 처리 속도가 8 FPS이면 실시간 30 FPS 입력을 따라가지 못한다.

2.5 텐서 — Tensor

한 줄 정의**:** 숫자를 여러 축으로 배열한 다차원 상자다. 딥러닝에서는 이미지, 영상, feature, mask를 모두 tensor로 표현한다.

- 숫자 하나: scalar
- 숫자 한 줄: vector
- 행과 열: matrix
- 세 축 이상: tensor

예를 들어 RGB 이미지 한 장은 높이 H × 너비 W × 채널 C 형태로 생각할 수 있다.
라이브러리에 따라 축 순서가 달라진다.

- 보기 쉬운 표기: (H, W, C) = (1080, 1920, 3)
- PyTorch에서 흔한 표기: (B, C, H, W)
- 영상 batch: (B, T, C, H, W)

여기서 B는 한 번에 처리하는 데이터 개수인 batch size, T는 frame 수다.

비유**:** 스프레드시트가 2차원 표라면 tensor는 여러 장의 표를 시간과 batch 방향으로 쌓은 파일함이다.

Shape를 읽는 습관

논문과 코드를 볼 때 tensor의 shape를 추적하면 오류를 많이 줄일 수 있다. 예를 들어 (2, 30, 3, 512, 512)는 RGB 영상 2개를 한 번에 처리하며, 각 영상은 30 frames이고 frame 해상도는 512×512라는 뜻이다.

2.6 Mask, Bounding Box, Ground Truth, Annotation

- **mask:** 각 픽셀이 대상인지 아닌지를 표시한 지도. 이진 mask는 보통 대상 1, 배경 0이다.

- **bounding box:** 대상을 둘러싼 직사각형. 보통 (x_min, y_min, x_max, y_max)로 표현한다.
- **annotation:** 사람이 데이터에 붙인 정답 표시. class label, box, mask 등이 될 수 있다.
- **ground truth, GT:** 평가와 학습에 정답으로 사용하는 annotation.

Box는 만들기 쉽지만 빈 배경까지 포함하고 정확한 모양을 표현하지 못한다. Mask는 정확하지만 사람이 픽셀 단위로 그리는 비용이 크다.

3. 컴퓨터비전 문제의 종류

한 교실 사진을 예로 들어 보자.

- **Image Classification** — 모델이 답하는 질문: 이 사진은 무엇을 보여 주는가? · 대표 출력: 교실 또는 class probability
- **Object Detection** — 모델이 답하는 질문: 어떤 물체가 어디 있는가? · 대표 출력: class + box + confidence
- **Semantic Segmentation** — 모델이 답하는 질문: 각 픽셀은 어떤 종류인가? · 대표 출력: 픽셀별 class map
- **Instance Segmentation** — 모델이 답하는 질문: 각 물체 개체의 정확한 모양은? · 대표 출력: 개체별 mask와 ID
- **Panoptic Segmentation** — 모델이 답하는 질문: 화면의 모든 픽셀은 무엇이며 개체 번호는 무엇인가? · 대표 출력: class와 instance가 결합된 전체 지도
- **Tracking** — 모델이 답하는 질문: 시간에 따라 같은 물체가 어디로 이동하는가? · 대표 출력: frame별 위치 또는 mask와 지속되는 ID

3.1 이미지 분류 — Image Classification

사진 전체에 하나 이상의 label을 붙인다.

- single-label: 고양이 하나를 선택
- multi-label: 사람, 자전거, 도로를 동시에 선택

분류는 물체가 사진의 어디에 있는지는 알려 주지 않는다. 고양이가 사진 구석에 작게 있어도 출력은 단지 “고양이”일 수 있다.

3.2 객체 탐지 — Object Detection

물체의 class와 위치를 직사각형 box로 찾는다. 출력에는 보통 class, confidence, box 좌표가 들어간다.

예: person, confidence 0.93, box (120, 40, 310, 470)

분류와 비교**:** 분류는 사진 전체에 답 하나, detection은 여러 물체마다 답과 위치를 준다.

segmentation과 비교**:** box는 물체를 넉넉히 둘러싸기 때문에 팔 사이의 배경이나 자동차 아래 도로도 포함한다. 정확한 윤곽이 필요하면 segmentation이 필요하다.

3.3 의미론적 분할 — Semantic Segmentation

모든 픽셀에 class를 붙인다. 모든 학생 픽셀은 person, 모든 의자 픽셀은 chair가 된다.

같은 class의 서로 다른 개체를 구분하지 않는다. 학생 세 명이 붙어 서 있으면 모두 같은 person 영역으로 표시될 수 있다.

적합한 예**:** 도로, 하늘, 건물, 물, 농경지처럼 “어떤 종류의 영역인가”가 중요한 문제.

3.4 개체 분할 — Instance Segmentation

각 물체의 픽셀을 정확히 찾으면서 같은 class의 개체도 따로 구분한다.

- 학생 A: person, instance ID 1
- 학생 B: person, instance ID 2
- 학생 C: person, instance ID 3

semantic과 비교**:** 둘 다 픽셀 mask를 만들지만 semantic은 종류만, instance는 “어느 개체인지”까지 구분한다.

3.5 파놉틱 분할 — Panoptic Segmentation

Semantic segmentation과 instance segmentation을 하나로 합친다. 화면의 모든 픽셀에 class를 붙이고, 셀 수 있는 물체는 instance도 나눈다.

- **things:** 사람, 자동차, 컵처럼 개별 개체를 셀 수 있는 대상
- **stuff:** 하늘, 도로, 잔디처럼 덩어리로 보는 배경 영역

교실에서는 학생과 의자는 개체별로 나누고, 벽과 바닥도 빠짐없이 class를 붙인다.

3.6 추적 — Tracking

여러 frame에서 같은 물체의 정체성, 즉 identity를 유지하며 위치를 따라간다. 1번 학생이 다른 학생 뒤에 잠깐 가려져도 다시 나타났을 때 계속 ID 1이어야 한다.

Tracking의 출력은 문제에 따라 box, point, keypoint, mask가 될 수 있다. 따라서 “tracking은 항상 box”라고 생각하면 안 되지만, 전통적인 tracking benchmark는 box를 많이 사용한다.

어려운 상황

- occlusion: 다른 물체에 가려짐
- disappearance: 화면 밖으로 나가거나 완전히 사라짐
- reappearance: 다시 등장함
- distractor: 목표와 비슷하게 생긴 방해 물체
- fast motion: frame 사이 이동이 너무 큼
- deformation: 사람의 자세처럼 모양이 크게 변함
- camera cut: 장면이 갑자기 바뀜

3.7 VOS — Video Object Segmentation

Video Object Segmentation: 영상에서 특정 객체의 mask를 frame마다 예측하면서 동일 객체를 유지하는 문제다. “분할과 시간적 추적”이라고 생각하면 된다.

전형적인 설정에서는 첫 frame의 정답 mask 또는 box를 주고 이후 frame을 자동으로 분할한다. 논문에 따라 다음 하위 설정이 있다.

- **semi-supervised VOS:** 첫 frame target annotation을 제공
- **unsupervised** 또는 **automatic VOS:** 첫 frame 정답 없이 눈에 띄는 객체를 자동 발견
- **interactive VOS:** 사용자가 중간에 click이나 수정 prompt를 제공
- **long-term VOS:** 장시간 가림, 화면 이탈, 재등장을 포함

VOS의 대표 지표는 영역 IoU인 J, 경계 정확도인 F, 두 평균인 J&F다.

3.8 PVOS — Promptable Video Object Segmentation

점, box, mask 같은 visual prompt로 사용자가 "이 객체"를 지정하면 모델이 그 mask를 영상 전체에 전파하는 문제다. SAM 2가 대표적인 promptable video segmentation 모델이다.

예를 들어 첫 frame에서 강아지 몸 안을 한 번 클릭하면 이후 frame에서도 강아지를 분할한다. 필요하면 중간 frame에 positive point, negative point, box, correction mask를 추가할 수 있다.

VOS와 관계**:** PVOS는 VOS 중에서도 사용자의 prompt로 어떤 객체를 분할할지 유연하게 지정하고 수정할 수 있는 설정이다.

약어 주의**:** 일부 후속 논문은 PVOS를 Promptable Video Object Segmentation이라는 뜻으로 쓰지만 약어가 완전히 통일된 것은 아니다. SAM 2 원 논문이 image와 video를 함께 포괄하며 공식적으로 강조한 표현은 **PVS, Promptable Visual Segmentation**이다. 논문마다 full name과 task definition을 먼저 확인해야 한다.

3.9 VOT — Visual Object Tracking

첫 frame에서 한 target의 box나 mask를 주고 이후 위치를 계속 추정한다. 전통 VOT는 주로 box와 tracking 성공률, precision, AUC, EAO를 평가한다. SAM 2 기반 tracker는 내부적으로 mask를 예측해도 최종 평가는 box로 변환하는 경우가 많다.

VOS와 비교

- VOS: 픽셀 단위 mask 품질이 핵심
- VOT: target 위치와 추적 지속성이 핵심이며 box 평가가 흔함

따라서 VOT AUC가 올랐다는 결과만으로 mask 경계가 좋아졌다고 말할 수 없다.

3.10 MOT — Multiple Object Tracking

영상 속 여러 객체를 동시에 검출하고 ID를 유지한다. 예를 들어 도로의 차량 20대에 ID를 붙이고 이동 경로를 만든다.

보통 detection과 association 두 단계로 이해할 수 있다.

1. 각 frame에서 사람이나 차량을 검출한다.
2. 이전 frame의 track과 현재 detection이 같은 개체인지 연결한다.
3. 가림 뒤에도 ID가 바뀌지 않게 한다.

MOT와 **multi-object VOS**의 차이**:** MOT는 box 중심인 경우가 많고 새로 등장하는 객체도 찾는다. Multi-object VOS는 지정된 여러 객체의 mask를 전파하는 경우가 많다. Box 대신 mask까지 추적하는 분야는 MOTS, Multi-Object Tracking and Segmentation이라고 부르기도 한다.

3.11 RVOS — Referring Video Object Segmentation

자연어 표현이 가리키는 객체를 영상 전체에서 분할한다.

예: “빨간 모자를 쓰고 왼쪽에서 달리는 사람”

모델은 문장의 색, 위치, 행동 정보를 영상과 연결해 target을 선택한 뒤 mask를 추적해야 한다. RVOS는 “어떤 객체인지”를 점이나 box가 아니라 문장으로 알려 준다.

3.12 VRS — Video Reasoning Segmentation

문장에 target이 직접 드러나지 않고, 상식이나 시간적 사건을 추론해야 하는 객체를 영상에서 분할한다.

- RVOS: “빨간 셔츠를 입은 선수” — 눈에 보이는 속성으로 바로 찾을 수 있음
- VRS: “경기에서 마지막 골을 넣은 선수” — 영상을 시간 순서로 이해하고 사건을 추론해야 함

VRS는 language understanding, temporal reasoning, object localization, mask propagation을 모두 요구한다. 최근 reasoning segmentation 문헌에서 VRS는 대체로 Video Reasoning Segmentation을 뜻하지만, 일부 문맥에서는 Video Referring Segmentation처럼 다른 full name으로도 쓰일 수 있다. 처음 등장한 full name과 입력 문장이 명시적인 지시인지 추론이 필요한 지시인지 반드시 확인해야 한다.

3.13 한 번에 구분하기

- 사진 전체에 이름 하나 → classification
- 물체마다 직사각형 → detection
- 픽셀마다 종류 → semantic segmentation
- 픽셀마다 종류와 개체 → instance segmentation
- 배경까지 모두 종류와 개체 → panoptic segmentation
- 시간에 따라 같은 ID 유지 → tracking
- 영상에서 지정된 객체의 mask 유지 → VOS
- 점, box, mask prompt로 지정 → PVOS
- SAM 2 원문의 image·video 통합 promptable task → PVS
- 주로 box로 한 target 추적 → VOT
- 여러 target을 detection하고 ID 연결 → MOT
- 명시적 문장으로 target 지정 → RVOS
- 암시적 문장을 추론해 target 지정 → VRS

PART B — 신경망·학습·프롬프트와 모델 적응

4. 신경망 안에서는 무슨 일이 일어나는가

4.1 인공신경망 — Neural Network

한 줄 정의: 입력 숫자를 여러 층에서 변환해 원하는 출력으로 바꾸는, 학습 가능한 함수다.

신경망의 기본 단위는 입력에 weight를 곱하고 bias를 더한 뒤 activation function을 통과시킨다. 수많은 단위가 층으로 연결되어 복잡한 변환을 만든다.

비유**:** 사진이 여러 검사대를 통과하는 공장과 같다.

1. 앞쪽 검사대는 밝기 변화와 선을 찾는다.
2. 중간 검사대는 모서리, 질감, 물체 부분을 조합한다.
3. 뒤쪽 검사대는 “이 영역이 강아지일 가능성”이나 mask를 만든다.

신경망이 사람의 뇌를 그대로 복제한 것은 아니다. 이름과 초기 아이디어가 신경세포에서 왔지만, 실제로는 큰 행렬 계산을 반복하는 수학 모델이다.

4.2 층 — Layer

한 줄 정의**:** 입력 tensor를 다른 tensor로 바꾸는 하나의 계산 단계다.

Convolution layer, linear layer, attention layer, normalization layer처럼 역할이 다른 층이 있다. “깊은 모델”은 이런 층이 많이 연결된 모델이다.

층을 많이 쌓는다고 무조건 좋아지지는 않는다. 계산량, 학습 난도, 과적합 가능성도 커진다. 좋은 구조는 필요한 정보를 잃지 않으면서 효율적으로 변환한다.

4.3 파라미터 — Parameter

한 줄 정의**:** 학습 과정에서 데이터에 맞게 값이 바뀌는 숫자다. Weight와 bias가 대표적이다.

처음에는 대체로 임의 값에서 시작하거나 사전학습 checkpoint에서 가져온다. 모델은 loss를 줄이는 방향으로 parameter를 조금씩 수정한다.

비유**:** 사진 보정 앱의 수백만 개 조절 손잡이와 같다. 학습은 정답에 가까워지도록 손잡이 위치를 자동으로 조정하는 과정이다.

*Parameter*와 *hyperparameter*

- **parameter:** 모델이 학습하는 값. 예: convolution weight
- **hyperparameter:** 사람이 실험 전에 정하는 설정. 예: learning rate, batch size, epoch 수, LoRA rank

두 단어를 혼동하지 말아야 한다. “모델이 100M parameters를 가진다”는 것은 학습되는 숫자가 약 1억 개라는 뜻이지, 연구자가 설정 1억 개를 직접 고른다는 뜻이 아니다.

Parameter 수와 *GPU memory*

FP32 parameter 하나는 보통 4 bytes, FP16은 2 bytes다. 그러나 학습할 때는 parameter만 저장하는 것이 아니다. Gradient, optimizer state, activation도 필요하다. 따라서 “100M parameters이니 FP16으로 200 MB면 된다”라고 계산하면 실제 학습 VRAM을 크게 과소평가한다.

4.4 특징 — Feature

한 줄 정의**:** 모델이 입력에서 뽑아낸 유용한 정보 표현이다.

Feature는 사람이 “이 채널은 귀를 의미한다”라고 직접 정하지 않는다. 학습 과정에서 문제 해결에 유용하도록 만들어진다.

- 낮은 층 feature: 색 변화, 가로선, 세로선, 단순 질감
- 중간 층 feature: 원, 바퀴, 눈, 털무늬 같은 부분
- 높은 층 feature: 물체 전체, 의미, 대상의 정체성

Feature는 보통 feature map 또는 feature vector 형태다.

- **feature map:** 공간 위치가 남아 있는 H×W×C tensor. 분할처럼 “어디인가”가 중요한 작업에 유리하다.
- **feature vector:** 공간 정보를 요약한 긴 숫자 목록. 이미지 검색이나 분류에 자주 쓴다.

원본 픽셀과 비교**:** 픽셀은 센서가 기록한 색이고, feature는 모델이 목적에 맞게 변환한 표현이다.

4.5 Encoder, Backbone, Decoder

Encoder

입력을 받아 중요한 feature로 바꾸는 부분이다. 이미지 encoder는 RGB 픽셀을 여러 해상도의 feature map으로 변환한다.

비유**:** 긴 교과서를 읽고 핵심 내용을 구조화된 필기로 바꾸는 학생.

Backbone

모델의 주된 feature extractor를 가리킨다. 많은 논문에서 image encoder와 비슷한 뜻으로 쓰지만 완전히 같은 말은 아니다.

- backbone: 전체 시스템에서 핵심 시각 feature를 만드는 주 구조라는 역할 중심 표현
- encoder: 입력을 표현으로 변환한다는 기능 중심 표현

SAM 2에서 Hiera는 image encoder이자 visual backbone이다.

Decoder

Encoder feature를 사용해 task에 맞는 출력을 만든다. Segmentation decoder는 낮아진 feature 해상도를 결합하고 확대해 픽셀 mask를 만든다.

비유**:** 요약 필기를 보고 질문의 형식에 맞는 답안지를 작성하는 학생.

같은 encoder feature라도 decoder를 바꾸면 분류, detection, segmentation처럼 다른 출력을 만들 수 있다.

*Skip connection*과 *multi-scale feature*

Encoder 뒤쪽 feature는 의미는 강하지만 공간 해상도가 낮을 수 있다. 앞쪽 feature는 경계와 위치가 자세하지만 의미는 약하다. U-Net 같은 구조는 앞쪽과 뒤쪽 feature를 skip connection으로 연결한다. 여러 크기의 feature를 함께 쓰는 것을 multi-scale feature fusion이라고 한다.

작은 물체와 가는 경계를 개선하는 논문에서 자주 등장하는 이유다.

4.6 CNN — Convolutional Neural Network

한 줄 정의**:** 작은 필터 kernel를 이미지 위로 이동시키며 주변 패턴을 찾는 신경망이다.

예를 들어 3×3 kernel은 한 번에 주변 9개 픽셀을 보고 계산한다. 같은 kernel를 이미지 여러 위치에 반복 적용하므로 위치가 바뀌어도 비슷한 패턴을 찾기 쉽다.

비유**:** 작은 돋보기를 사진 위에서 한 칸씩 움직이며 "여기에 세로 경계가 있는가?"를 검사하는 것과 같다.

CNN의 주요 용어

- **kernel** 또는 **filter:** 지역 패턴을 찾는 작은 weight 묶음
- **stride:** kernel가 한 번에 이동하는 칸 수
- **padding:** 가장자리도 처리하기 위해 주변에 값을 덧붙이는 것
- **pooling:** 가까운 값을 요약해 해상도를 줄이는 것
- **receptive field:** 한 출력 값이 참고할 수 있는 원본 영역
- **dilated convolution:** kernel 원소 사이 간격을 벌려 parameter를 크게 늘리지 않고 넓은 영역을 보는 convolution
- **depthwise convolution:** channel별로 convolution을 수행해 계산량을 줄이는 방식

장점과 한계

CNN은 가까운 픽셀의 관계와 질감, 경계를 잘 다루고 구조적 편향 inductive bias가 강하다. 반면 멀리 떨어진 두 영역의 관계를 직접 연결하려면 층을 많이 거쳐야 한다.

4.7 Transformer

한 줄 정의**:** 입력을 token들로 나타내고 attention을 이용해 token 사이 관계를 학습하는 구조다.

언어에서는 단어가 token이 될 수 있다. 이미지에서는 이미지를 작은 patch로 나누어 각 patch를 token으로 만들 수 있다. 영상에서는 patch와 시간까지 포함한 spatiotemporal token을 사용할 수 있다.

비유**:** CNN이 가까운 동네부터 차례로 조사한다면 Transformer는 회의실에서 모든 대표가 서로를 보며 "누구의 정보가 지금 중요한가?"를 판단하는 것과 같다.

Vision Transformer, ViT

이미지를 patch로 나누고 각 patch를 vector로 바꾼 뒤 Transformer에 넣는 구조다. 예를 들어 224×224 이미지를 16×16 patch로 나누면 14×14, 즉 196개의 patch token이 생긴다.

Hierarchical Transformer

처음에는 고해상도 작은 영역을 보고, 뒤로 갈수록 token 수를 줄이며 더 큰 의미를 보는 계층형 구조다. CNN의 multi-scale 장점과 Transformer의 관계 모델링을 결합하려는 설계다. SAM 2의 Hiera encoder가 여기에 해당한다.

4.8 Attention

한 줄 정의**:** 현재 계산에 어떤 정보가 얼마나 중요한지 가중치를 정해 모으는 방법이다.

Attention은 보통 Query, Key, Value 세 요소로 설명한다.

- **Query, Q:** 내가 지금 찾고 싶은 것
- **Key, K:** 각 정보가 무엇을 나타내는지 알려 주는 표지
- **Value, V:** 실제로 가져올 내용

도서관 비유**:** "강아지의 현재 모습"이라는 query로 책의 색인 key들을 비교하고, 관련성이 높은 책의 내용 value를 더 많이 읽는다.

*Self-attention*과 *cross-attention*

- **self-attention:** 같은 집합 안의 token끼리 관계를 계산한다. 한 이미지의 patch들이 서로 참고한다.
- **cross-attention:** 서로 다른 집합 사이 관계를 계산한다. 현재 frame feature가 과거 memory feature를 참고하거나, image token이 text token을 참고한다.

SAM 2의 memory attention은 현재 frame이 과거 frame에 저장된 객체 정보를 선택적으로 읽는 cross-attention으로 이해할 수 있다.

*Attention*의 한계

Token 수가 많으면 모든 쌍의 관계를 계산하는 비용이 커질 수 있다. 고해상도 이미지와 긴 영상에서 window attention, sparse attention, token pruning, memory compression 같은 효율화가 등장하는 이유다.

4.9 Embedding과 Token

Embedding

한 줄 정의**:** 이미지, patch, 단어, prompt, 객체 같은 대상을 의미를 담은 숫자 vector로 바꾼 표현이다.

비슷한 의미나 모습은 embedding 공간에서 가까워지도록 학습할 수 있다. 첫 frame의 target crop embedding과 현재 후보 crop embedding을 비교하면 재등장한 객체를 찾는 데 사용할 수 있다.

Token

한 줄 정의**:** Transformer가 처리하는 하나의 정보 단위다.

Token은 단순한 번호가 아니라 embedding vector로 변환되어 계산된다.

- language token: 단어 또는 부분 단어
- image token: image patch를 나타내는 vector
- prompt token: point, box, text prompt 정보
- object token 또는 object pointer: 특정 객체의 정체성을 요약한 vector
- memory token: 과거 frame에서 저장된 정보 단위

*Feature, embedding, token*의 차이

세 용어는 겹쳐 쓰일 수 있지만 초점이 다르다.

- **feature:** 모델이 추출한 유용한 표현이라는 가장 넓은 말
- **embedding:** 대상을 고정 길이 vector 공간에 나타냈다는 점을 강조
- **token:** Transformer가 attention으로 처리하는 한 단위라는 점을 강조

한 image patch의 feature vector가 동시에 patch embedding이자 image token일 수 있다.

4.10 활성값 — Activation

신경망이 현재 입력을 처리하며 중간에 만들어 내는 값이다. Parameter는 여러 입력에 공통으로 쓰이는 장기적인 모델 지식이고, activation은 현재 사진이나 batch를 처리할 때만 생기는 중간 계산 결과다.

학습에서는 backward 계산을 위해 activation을 많이 보관하므로 frozen backbone이라도 GPU memory가 필요하다. "학습 parameter가 적다"와 "VRAM이 거의 들지 않는다"는 같은 말이 아니다.

4.11 모델의 세 가지 memory를 구분하기

논문에서 memory라는 단어는 서로 다른 뜻으로 쓰인다.

1. **Model parameter:** 학습되어 checkpoint에 저장되는 장기 지식
2. **GPU memory, VRAM:** 계산 중 tensor를 올려 두는 하드웨어 공간
3. **Video memory bank:** 이전 frame의 feature와 mask를 저장해 다음 frame이 참고하는 데이터 구조

SAM 2 연구에서 "memory contamination"은 대개 세 번째 의미다. 잘못 분할한 frame을 memory bank에 저장해 이후 예측까지 연쇄적으로 틀리는 현상을 뜻한다.

5. 모델은 어떻게 배우고 평가되는가

5.1 Dataset, Sample, Label

- **dataset:** 학습과 평가에 사용하는 데이터 전체
- **sample:** 이미지 한 장, 영상 clip 하나처럼 데이터 한 개
- **label:** 정답 class, box, mask, 문장 등
- **annotation:** 사람이 label을 만드는 작업 또는 만들어진 표시

Segmentation은 픽셀마다 정답을 표시해야 하므로 classification보다 annotation 비용이 훨씬 크다. Foundation model과 promptable model이 주목받는 이유 중 하나는 새로운 작업마다 모든 mask를 다시 그리는 비용을 줄일 가능성이 있기 때문이다.

5.2 Training, Validation, Test

Training set

모델 parameter를 실제로 업데이트하는 데이터다. 학생이 문제집을 풀며 실력을 바꾸는 단계에 해당한다.

Validation set

Learning rate, epoch 수, threshold, adapter 구조 같은 선택을 비교하고 조기 종료 early stopping 시점을 정하는 데이터다. 모의고사에 해당한다. Validation 결과를 보면서 설정을 계속 바꾸므로 최종 성능이라고 보기 어렵다.

Test set

모든 설계를 확정한 뒤 마지막에 일반화 성능을 확인하는 데이터다. 기말고사에 해당한다. Test 결과를 반복해서 보고 구조를 바꾸면 사실상 test set에 맞춰진 것이므로 공정한 평가가 아니다.

Data leakage

Test 정보가 training이나 model selection에 흘러 들어가는 문제다. 같은 영상의 연속 frames를 무작위로 train과 test에 나누면 거의 같은 장면이 양쪽에 들어갈 수 있다. 영상 연구에서는 video 단위, 환자 연구에서는 patient 단위로 분리해야 한다.

5.3 학습 한 단계

1. **Forward pass:** 입력을 모델에 넣어 prediction을 계산한다.
2. **Loss computation:** prediction과 ground truth의 차이를 loss로 계산한다.
3. **Backward pass, backpropagation:** 각 parameter가 loss에 얼마나 영향을 주었는지 gradient를 계산한다.
4. **Optimizer step:** gradient를 이용해 parameter를 조금 수정한다.
5. 이 과정을 여러 batch와 epoch에 걸쳐 반복한다.

비유**:** 문제를 풀고, 틀린 정도를 확인하고, 어떤 생각 때문에 틀렸는지 되짚고, 풀이 습관을 조금 바꾸는 과정이다.

5.4 Loss

한 줄 정의**:** 학습 중 prediction이 정답과 얼마나 다른지 나타내는 미분 가능한 벌점이다.

Loss가 작아지는 방향으로 parameter를 업데이트한다. Loss와 평가 metric은 목적이 다르다.

- loss: 학습을 움직이는 신호
- metric: 사람이 결과를 해석하고 비교하는 점수

Segmentation에서 자주 쓰는 ***loss***

- **Binary Cross-Entropy, BCE:** 각 픽셀이 target인지 아닌지의 확률을 평가
- **Cross-Entropy, CE:** 여러 class 중 정답 class의 확률을 평가
- **Dice loss:** prediction과 GT 영역의 겹침을 직접 높이는 데 유리
- **IoU loss:** intersection-over-union을 학습 목표로 사용
- **Focal loss:** 쉬운 배경보다 어려운 픽셀과 작은 대상에 더 집중
- **Boundary loss:** 객체 경계 근처의 오차를 더 중요하게 처리

여러 loss를 가중합해 쓰기도 한다. 예를 들어 total loss = BCE + Dice + 0.2 × Boundary loss처럼 구성할 수 있다. 가중치는 hyperparameter다.

주의**:** Training loss가 낮아져도 test 성능이 반드시 좋아지는 것은 아니다. 모델이 training data를 외운 과적합일 수 있다.

5.5 Gradient, Optimizer, Learning Rate

- **gradient:** parameter를 어느 방향으로 바꾸면 loss가 커지거나 작아지는지 나타내는 기울기
- **optimizer:** gradient를 이용해 parameter를 업데이트하는 규칙. SGD, AdamW 등이 대표적
- **learning rate:** 한 번에 parameter를 얼마나 크게 바꿀지 정하는 값

Learning rate가 너무 크면 정답 주변을 계속 뛰어넘고, 너무 작으면 학습이 매우 느리거나 나쁜 지점에 머물 수 있다.

5.6 Batch, Iteration, Epoch

- **batch:** 한 번의 forward와 backward에 함께 넣는 sample 묶음
- **batch size:** 그 묶음의 sample 수
- **iteration** 또는 **step:** optimizer가 한 번 업데이트된 횟수
- **epoch:** training set 전체를 한 번 본 것

영상은 frame 수와 해상도 때문에 memory를 많이 사용하므로 batch size 1이나 짧은 clip을 쓰는 경우도 흔하다.

5.7 Checkpoint

한 줄 정의**:** 특정 시점의 모델 parameter와 필요하면 optimizer 상태 등을 파일로 저장한 것이다.

Checkpoint는 게임의 저장 지점과 비슷하다. 학습 중 가장 좋은 validation 성능의 checkpoint를 저장하고 나중에 inference하거나 fine-tuning할 수 있다.

Model architecture와 **checkpoint**의 차이

- architecture: 층이 어떻게 연결되는지에 대한 설계도
- checkpoint: 그 설계도의 parameter에 들어 있는 실제 학습 값

같은 Hiera-B+ architecture라도 어떤 데이터로 학습한 checkpoint인지에 따라 성능이 달라질 수 있다.

5.8 Pretraining

한 줄 정의**:** 특정 작은 과제를 풀기 전에 크고 다양한 데이터로 일반적인 표현을 먼저 학습하는 것.

비유**:** 의대 전공 공부 전에 생물과 화학의 기초를 폭넓게 배우는 것과 같다.

ImageNet class로 pretraining하거나, mask data로 segmentation pretraining하거나, label 없이 self-supervised pretraining할 수 있다. SAM 2는 대규모 이미지와 영상 segmentation data에서 일반적인 promptable segmentation 능력을 배운 foundation model이다.

5.9 Fine-tuning

한 줄 정의**:** pretrained model을 새로운 dataset이나 task에 맞게 추가 학습하는 것.

- **full fine-tuning:** 대부분 또는 모든 parameter를 업데이트
- **partial fine-tuning:** decoder나 일부 stage만 업데이트
- **parameter-efficient fine-tuning:** adapter나 LoRA처럼 적은 parameter만 업데이트

Fine-tuning은 domain 성능을 높일 수 있지만 데이터가 적으면 과적합하고 원래의 범용 능력을 잃을 수 있다. 이를 catastrophic forgetting이라고 부르기도 한다.

5.10 Frozen

한 줄 정의**:** 해당 모듈의 parameter를 학습 중 업데이트하지 않도록 고정한 상태.

"Frozen image encoder"는 encoder를 forward 계산에는 사용하지만 gradient로 weight를 바꾸지 않는다는 뜻이다. Encoder 계산 자체가 사라지는 것은 아니다.

Frozen과 ***no gradient***의 실무적 의미

Parameter update를 막으면 optimizer state와 parameter gradient가 줄어 memory를 아낄 수 있다. 그러나 trainable adapter에 encoder feature가 필요하고 계산 그래프를 유지해야 하면 activation memory는 여전히 들 수 있다. 구현 위치에 따라 절감 폭이 다르다.

5.11 Inference

한 줄 정의**:** 학습된 모델을 실제 입력에 사용해 prediction을 만드는 과정이다.

Inference에서는 일반적으로 backward와 optimizer update를 하지 않는다. 그래서 training보다 memory 사용량이 적고 빠르다. 논문의 FPS가 training 속도인지 inference 속도인지 확인해야 한다.

Offline과 ***online*** 또는 ***streaming***

- **offline:** 전체 영상을 미리 보고 앞뒤 정보를 모두 사용할 수 있음
- **online** 또는 **streaming:** 미래 frame을 보지 않고 도착하는 순서대로 처리

Long video에서 전체 결과를 본 뒤 최적 경로를 고르는 방법은 정확하더라도 실시간 카메라에는 바로 사용할 수 없다.

5.12 Overfitting과 Generalization

- **overfitting:** training data에서는 매우 잘하지만 새로운 data에서는 성능이 떨어지는 현상
- **generalization:** 보지 못한 sample, 장면, domain에서도 배운 규칙이 작동하는 능력

Data augmentation, regularization, 더 다양한 data, parameter 수 제한, early stopping 등이 과적합 완화에 쓰인다.

5.13 지도학습, 자기지도학습, 약지도학습

- **supervised learning:** 사람이 만든 label을 정답으로 학습

- **self-supervised learning:** data 자체에서 학습 문제를 만들어 label 없이 표현을 학습
- **weakly supervised learning:** 정확한 mask 대신 image label, point, scribble, box처럼 더 약한 annotation으로 학습
- **unsupervised learning:** 명시적 정답 없이 구조를 찾음

"Unsupervised VOS"라는 task 이름과 "모델 pretraining이 완전히 label-free"라는 주장은 같은 뜻이 아니다. Task의 입력 조건과 모델이 과거에 어떻게 학습되었는지를 따로 확인해야 한다.

6. Prompt, Zero-shot, Foundation Model

6.1 Prompt

한 줄 정의**:** 모델에게 무엇을 원하는지 지정해 주는 입력 신호다.

언어 모델에서는 문장이 prompt이고, segmentation model에서는 point, box, mask, text, reference image 등이 prompt가 될 수 있다.

SAM 계열의 대표 ***visual prompt***

- **positive point:** "이 픽셀은 target 안쪽이다"라고 알려 주는 클릭
- **negative point:** "이 픽셀은 target이 아니다"라고 알려 주는 클릭
- **bounding box:** "이 사각형 안의 대상에 집중하라"는 위치 힌트
- **mask prompt:** 이전 mask 또는 대략 그린 영역을 더 정밀하게 수정하기 위한 힌트

구체 예시**:** 두 마리 강아지가 붙어 있는 사진에서 왼쪽 강아지 몸 안에 positive point를 찍고 오른쪽 강아지에 negative point를 찍으면 어느 개체를 원하는지 더 분명해진다.

Text prompt

"빨간 우산을 든 사람"처럼 자연어로 target을 지정한다. SAM 2 기본 모델은 본래 class name이나 문장을 직접 이해하는 open-vocabulary detector가 아니다. Text를 사용하려면 Grounding DINO, CLIP, vision-language model 같은 외부 모듈이 box나 feature prompt를 만들어 주거나, 별도의 text adapter를 학습하는 경우가 많다.

Prompt encoder

Point와 box 같은 prompt를 embedding으로 바꾸는 모듈이다. Image feature와 결합할 수 있는 숫자 표현으로 변환한다.

Prompt quality가 중요한 이유

Model이 좋아도 target 밖의 point나 너무 큰 box를 주면 잘못된 mask가 나올 수 있다. 논문 비교에서는 다음을 반드시 같게 맞춰야 한다.

- point 개수와 위치를 사람이 정했는지, GT에서 자동 생성했는지
- box가 GT box인지 detector가 예측한 box인지
- 첫 frame만 prompt했는지 중간 frame에도 correction했는지
- 여러 mask 후보 중 어떤 기준으로 하나를 골랐는지

GT mask에서 만든 완벽한 box와 실제 detector box를 비교하면 공정하지 않다.

6.2 Prompt-free와 Automatic Prompting

- **prompt-free:** 사용자가 직접 prompt를 주지 않아도 모델이 target mask를 생성하는 설정
- **automatic prompting:** detector, support image, 다른 model, 이전 prediction 등이 point나 box를 자동 생성

두 용어는 같은 듯 보이지만 다르다. 사용자 입력이 없더라도 내부 detector가 target-domain label로 학습되었다면 전체 시스템이 완전히 zero-shot인 것은 아니다.

SAM 2의 Automatic Mask Generator는 격자 형태로 많은 point prompt를 넣고 겹치는 mask를 필터링해 여러 객체 mask를 만든다. 이는 text를 이해해 원하는 class를 찾는 object detector와는 다르다.

6.3 Zero-shot

한 줄 정의**:** 평가하려는 새로운 class, dataset 또는 task의 labeled example로 추가 학습하지 않고 수행하는 설정.

Zero-shot은 문맥에 따라 범위가 달라 반드시 “무엇에 대해 zero-shot인가?”를 확인해야 한다.

- **zero-shot class generalization:** 학습 중 보지 않은 class를 처리
- **zero-shot transfer:** target dataset에서 fine-tuning하지 않고 바로 평가
- **zero-shot mask generation:** target-domain mask label 없이 detector box를 SAM에 입력
- **zero-shot tracking:** 첫 frame prompt는 받지만 해당 tracking dataset으로 재학습하지 않음

중요한 오해

- Zero-shot이라고 pretraining이 없다는 뜻은 아니다. 대규모 data로 이미 학습된 model을 쓴다.
- Zero-shot이라고 prompt가 없다는 뜻도 아니다. 첫 frame mask나 text prompt를 받을 수 있다.
- System 일부가 target data로 학습되었다면 전체 pipeline의 zero-shot 범위를 정확히 밝혀야 한다.

예를 들어 SAR 영상으로 학습한 detector가 ship box를 찾고 pretrained SAM 2가 mask를 만든다면 “mask annotation에 대해서는 zero-shot”일 수 있지만 “SAR target domain에 대해 완전히 zero-shot”이라고 말하기는 어렵다.

6.4 One-shot, Few-shot, Support Set

- **one-shot:** class 또는 task마다 labeled example 하나를 사용
- **few-shot:** 매우 적은 labeled examples를 사용
- **support set:** 새 target을 설명하기 위해 제공되는 소수의 example 묶음
- **query:** support를 참고해 prediction해야 하는 새 입력

의료 영상에서 비슷한 장기의 image-mask 한 쌍을 SAM 2 memory에 넣고 새 환자를 분할한다면 support-guided few-shot 방식으로 볼 수 있다.

“첫 frame mask 한 장을 받는 semi-supervised VOS”를 무조건 one-shot learning이라고 부르지는 않는다. 그 mask는 추적할 instance를 지정하는 task input이고, model parameter를 학습시키는 example이 아닐 수 있기 때문이다.

6.5 Foundation Model

한 줄 정의**:** 크고 다양한 data에서 사전학습되어 여러 task와 domain에 재사용할 수 있는 범용 기반 model.

비유**:** 한 메뉴만 만드는 전용 기계가 아니라 다양한 요리의 기초 재료와 조리법을 갖춘 큰 주방이다. 새 요리에 맞는 작은 조정이나 prompt가 여전히 필요할 수 있다.

Foundation model의 일반적 특징은 다음과 같다.

- 매우 크고 다양한 pretraining data
- 여러 downstream task에 transfer 가능
- prompt 또는 작은 fine-tuning으로 활용
- 같은 backbone을 다양한 domain에서 재사용

Segmentation Foundation Model

점이나 box prompt에 따라 다양한 객체를 분할하도록 대규모 mask data에서 학습된 model이다. SAM과 SAM 2가 대표적이다.

Foundation이라는 말이 ***“***완벽함***”***을 뜻하지는 않는다

대규모 pretraining을 해도 다음 상황에서 실패할 수 있다.

- 의료, 수중, SAR처럼 일반 사진과 다른 domain
- 매우 작거나 가는 객체
- 낮은 contrast와 어두운 장면
- 긴 가림 뒤 재등장
- 비슷한 distractor가 많은 영상
- 부정확한 prompt

이 틈을 domain adaptation, memory control, boundary refinement, robustness 연구가 다룬다.

6.6 Generalist와 Specialist

- **generalist model:** 다양한 class와 scene을 폭넓게 처리
- **specialist model:** 특정 domain이나 task에서 높은 성능을 목표로 함

SAM 2는 generalist에 가깝고, 의료 병변 전용으로 fine-tune한 모델은 specialist에 가깝다. Specialist가 해당 dataset에서 더 정확하더라도 다른 domain의 범용성이 떨어질 수 있다. 연구에서는 target-domain 성능과 원래 generalization을 함께 보는 것이 좋다.

6.7 Open-vocabulary와 Open-world

- **closed-set:** 학습 때 정한 class 목록 안에서만 예측
- **open-vocabulary:** text embedding 등을 이용해 학습 class 목록 밖의 새로운 이름도 query로 사용

- **open-world:** 모르는 class의 존재를 인식하고 새 class를 점차 배울 수 있는 더 넓은 설정

SAM 2는 “아무 물체나 point로 지정해 mask를 만들 수 있다”는 의미에서 category-agnostic이지만, 기본 모델 자체가 class name을 출력하는 open-vocabulary classifier는 아니다.

8. 새 도메인에 모델을 적응시키는 방법

8.1 Domain

한 줄 정의**:** 데이터가 만들어지는 환경과 분포의 묶음이다.

사진 속 물체 class가 같아도 촬영 장비, 조명, 배경, 해상도, 색, noise가 달라지면 domain이 달라질 수 있다.

예:

- 일반 스마트폰 사진과 내시경 영상
- 맑은 낮 도로와 야간 안개 도로
- 자연광 RGB 사진과 SAR 위성 영상
- 수면 위 사진과 수중 카메라 영상
- 병원 A의 CT와 병원 B의 CT

8.2 Domain Shift

학습 data와 실제 사용 data의 분포가 달라지는 현상이다.

사람은 “둘 다 물고기”라고 쉽게 알아도, 일반 사진에서 학습한 모델은 수중의 푸른 색조, 부유물, 흐린 경계 때문에 feature가 달라져 실패할 수 있다.

Domain shift는 입력 색만 달라지는 단순 문제가 아니다.

- **appearance shift:** 색, 밝기, 질감, noise 변화
- **sensor shift:** RGB, depth, 열화상, SAR 등 장비 차이
- **class** 또는 **label shift:** 등장 class 비율이나 정의가 다름
- **geometric shift:** 항공 영상처럼 시점과 scale이 다름
- **temporal shift:** frame rate, motion blur, 장면 변화가 다름

8.3 Source Domain과 Target Domain

- **source domain:** 원래 학습에 사용한 환경
- **target domain:** 새로 적용하려는 환경

예를 들어 일반 자연 이미지로 사전학습한 SAM 2를 복부 CT에 적용한다면, 일반 이미지가 source에 가깝고 CT가 target domain이다.

8.4 Domain Adaptation

한 줄 정의**:** Source에서 학습한 모델이 target domain에서도 잘 작동하도록 차이를 줄이는 방법.

Target label 사용 정도에 따른 구분

- **supervised domain adaptation:** target의 정답 label을 충분히 사용
- **semi-supervised domain adaptation:** target label 일부와 unlabeled data를 함께 사용
- **unsupervised domain adaptation, UDA:** source label과 unlabeled target data를 사용
- **source-free domain adaptation:** source 원본 data 없이 pretrained source model과 target data만 사용
- **test-time adaptation, TTA:** 실제 test 입력이 들어올 때 일부 parameter나 통계를 즉석에서 조정

*Domain generalization*과 비교

- **domain adaptation:** target domain data를 어떤 형태로든 미리 보며 적응
- **domain generalization:** 특정 target을 미리 보지 않고 여러 source에서 일반화 능력을 높임

논문에서 target test image 자체로 online update한다면 일반적인 offline fine-tuning보다 test-time adaptation에 가깝다.

8.5 Transfer Learning

한 줄 정의**:** 한 data나 task에서 배운 지식을 다른 data나 task에 재사용하는 큰 개념.

Pretrained encoder를 의료 segmentation에 가져오는 것은 transfer learning이다. Domain adaptation은 transfer learning 중에서도 source와 target의 분포 차이를 명시적으로 다루는 경우가 많다.

8.6 Full Fine-tuning과 Partial Fine-tuning

- **full fine-tuning:** backbone, decoder 등 대부분의 parameter를 업데이트
- **partial fine-tuning:** decoder, 마지막 stage 등 일부 기존 parameter만 업데이트

Full fine-tuning은 target 성능을 크게 높일 가능성이 있지만 training VRAM과 data가 많이 필요하고 범용 능력을 잃을 수 있다. Partial fine-tuning은 더 싸고 안정적이지만 domain gap이 매우 크면 부족할 수 있다.

8.7 PEFT — Parameter-Efficient Fine-Tuning

한 줄 정의**:** Pretrained model 대부분을 frozen한 채, 적은 수의 parameter만 학습해 새 task나 domain에 적응시키는 방법의 총칭이다.

PEFT는 특정 구조 하나의 이름이 아니다. Adapter, LoRA, prompt tuning, bias-only tuning 등이 PEFT에 포함된다.

*PEFT*가 유리한 이유

- Trainable parameter와 optimizer state가 적다.
- 여러 domain마다 작은 추가 weight만 저장할 수 있다.
- 원래 pretrained weight를 덜 훼손한다.
- 작은 dataset에서 full fine-tuning보다 과적합 위험을 줄일 수 있다.

PEFT가 해결하지 못하는 것

- Frozen backbone의 forward 계산은 그대로 필요하다.
- 고해상도 activation과 video memory 때문에 VRAM이 여전히 클 수 있다.
- Parameter가 적다고 항상 빠른 것은 아니다.
- Domain gap이 매우 크면 작은 모듈만으로 부족할 수 있다.

따라서 논문에서 “efficient”라고 하면 trainable parameter, 저장 용량, training time, inference latency, VRAM 중 무엇이 효율적인지 따로 확인해야 한다.

8.8 Adapter

한 줄 정의**:** Pretrained network 사이에 끼워 넣어 새 정보만 학습하는 작은 추가 모듈.

대표적인 bottleneck adapter는 다음처럼 작동한다.

1. 큰 feature dimension을 작은 dimension으로 줄인다.
2. 작은 공간에서 비선형 변환을 한다.
3. 원래 dimension으로 다시 늘린다.
4. 결과를 원래 feature에 residual로 더한다.

비유**:** 기존 대형 공장을 뜯지 않고 생산 라인 사이에 특정 제품용 소형 가공 장치를 붙이는 것과 같다.

Adapter의 종류

- **MLP adapter:** fully connected layer로 channel 정보를 변환
- **convolutional adapter:** 지역 경계와 질감을 다루기 위해 convolution 사용
- **depthwise-dilated adapter:** 적은 parameter로 여러 scale의 지역 정보를 수집
- **stage-specific adapter:** backbone 해상도 stage마다 별도 adapter 사용
- **shared adapter:** 여러 stage가 같은 weight를 공유
- **temporal adapter:** 여러 frame 관계를 처리
- **deformable convolution adapter:** 물체 모양에 맞게 sampling 위치를 학습

어디에 넣는가가 중요하다

앞 stage에 넣으면 색과 texture 같은 낮은 수준 domain shift에 영향을 주기 쉽고, 뒤 stage에 넣으면 의미 feature를 바꾸기 쉽다. 모든 stage에 넣으면 표현력은 커지지만 parameter와 계산도 늘어난다.

8.9 LoRA — Low-Rank Adaptation

한 줄 정의**:** 큰 weight 행렬 자체는 frozen하고, 그 변화량을 작은 두 행렬의 곱으로 학습하는 PEFT 방법.

원래 weight를 W라고 할 때 실제 계산에는 W와 작은 변화량 ΔW를 함께 사용한다. LoRA는 ΔW를 B×A처럼 rank가 낮은 두 행렬로 표현한다. Rank r이 원래 차원보다 훨씬 작으므로 trainable parameter가 줄어든다.

비유**:** 거대한 사전을 통째로 다시 쓰지 않고, 필요한 수정사항만 얇은 정정표 두 장의 조합으로 보관하는 것과 같다.

Rank

LoRA의 rank는 변화량이 표현할 수 있는 방향의 수를 결정한다. Rank가 너무 작으면 적응 능력이 부족하고, 너무 크면 parameter 절감 장점이 줄어든다.

*LoRA*를 넣는 위치

Transformer의 query, key, value projection이나 output projection, MLP weight 등에 넣을 수 있다. 어느 층과 projection에 넣었는지에 따라 결과가 다르므로 “LoRA 사용”만으로 실험을 충분히 설명할 수 없다.

Merge

학습 후 LoRA 변화량을 원래 weight에 합칠 수 있는 구조에서는 inference 때 별도 branch 계산을 줄일 수 있다. 다만 모든 구현과 quantized model에서 항상 완전히 무료인 것은 아니다.

8.10 Adapter와 LoRA 비교

- 바꾸는 방식 — Adapter: 새 작은 layer를 feature 경로에 추가 · LoRA: 기존 weight의 저랭크 변화량을 추가
- 비선형성 — Adapter: activation과 convolution 등을 자유롭게 사용 가능 · LoRA: 기본 형태는 linear weight update
- 지역 경계 처리 — Adapter: convolutional adapter로 직접 설계 가능 · LoRA: 주로 projection weight 적응
- **inference overhead** — Adapter: 새 layer 때문에 생길 수 있음 · LoRA: merge 가능하면 매우 작을 수 있음
- 설계 선택 — Adapter: 위치, kernel, bottleneck 등 다양 · LoRA: rank, 대상 layer, scaling 등이 중요
- 공통점 — Adapter: 원래 backbone 대부분을 frozen하는 PEFT

어느 쪽이 항상 우월한 것은 아니다. 수중 색과 texture에는 convolutional adapter가, language 또는 attention projection 적응에는 LoRA가 유리할 수 있다. 공정한 비교를 하려면 trainable parameter 수, training data, prompt, learning rate를 최대한 맞춰야 한다.

8.11 Prompt Tuning과 Visual Prompt Tuning

Model 본체를 frozen하고 학습 가능한 prompt token을 입력에 추가하는 PEFT 방법이다. 사람이 주는 실제 point prompt와 구분해야 한다.

- **user prompt:** 사용자가 target을 지정하는 point, box, text
- **learnable prompt:** training으로 값이 업데이트되는 가상 token

둘 다 prompt라고 부르지만 하나는 task input이고 다른 하나는 trainable parameter다.

8.12 Online Learning과 Test-Time Update

Video를 처리하는 중에 새 frame이나 correction을 이용해 작은 모듈을 업데이트하는 방법이다.

장점은 현재 영상에 빠르게 적응할 수 있다는 것이다. 단점은 잘못된 pseudo-label로 학습하면 drift가 커지고, frame 순서에 따라 결과가 달라지며, inference 중 backward 때문에 지연이 생긴다는 것이다.

공정한 평가를 위해 다음을 밝혀야 한다.

- Test label을 직접 쓰는가?
- Model prediction을 pseudo-label로 쓰는가?
- 영상마다 parameter를 reset하는가?
- Update 시간이 FPS에 포함되는가?

9. 모델을 가볍게 하거나 세부 정보를 보강하는 방법

9.1 Knowledge Distillation

한 줄 정의**:** 크고 강한 teacher model의 지식을 작고 빠른 student model이 따라 배우게 하는 방법.

비유**:** 정답만 보는 것이 아니라 우수한 선배가 문제를 바라보는 방식과 중간 풀이까지 배우는 것.

무엇을 따라 배우는가

- **logit distillation:** teacher의 class score 또는 확률
- **feature distillation:** 중간 feature map
- **attention distillation:** 어떤 token이나 위치를 중요하게 본 패턴
- **mask distillation:** teacher의 soft mask 또는 boundary
- **memory distillation:** video memory representation과 attention output

Student는 GT loss와 distillation loss를 함께 사용할 수 있다.

한계

- Teacher inference까지 필요한 training 비용이 든다.
- Teacher의 오류와 편향도 전달될 수 있다.
- 서로 feature shape가 다르면 projection module이 필요하다.
- 작은 student가 teacher의 모든 능력을 담을 수는 없다.

Distillation은 training을 싸게 만든다는 뜻이 아니라, 학습이 끝난 student의 inference를 가볍게 만드는 경우가 많다.

9.2 Quantization

한 줄 정의**:** Weight와 activation을 더 낮은 정밀도의 숫자로 표현해 memory와 연산을 줄이는 방법.

대표 형식:

- FP32: 32-bit floating point
- FP16 또는 BF16: 16-bit floating point
- INT8: 8-bit integer
- INT4: 4-bit integer

비유**:** 1 mm 단위 자 대신 1 cm 단위 자로 재는 것과 같다. 저장과 계산은 간단해지지만 매우 미세한 차이를 놓칠 수 있다.

방법

- **post-training quantization, PTQ:** 학습 완료 후 calibration data로 범위를 정하고 변환
- **quantization-aware training, QAT:** 학습 중 낮은 정밀도 오차를 흉내 내며 적응
- **weight-only quantization:** weight만 낮은 bit로 저장
- **weight-activation quantization:** weight와 activation을 모두 낮은 bit로 계산

Segmentation에서 주의할 점

전체 IoU는 비슷해도 작은 객체, 얇은 구조, boundary confidence가 먼저 손상될 수 있다. Quantization 연구는 mIoU만이 아니라 boundary metric과 size별 metric을 확인해야 한다.

9.3 Pruning

한 줄 정의**:** 중요하지 않다고 판단한 weight, channel, attention head, token 또는 memory entry를 제거하거나 건너뛰는 방법.

종류

- **unstructured weight pruning:** weight 개별 원소를 0으로 만듦
- **structured pruning:** channel, layer, attention head처럼 묶음 단위로 제거
- **token pruning:** 중요하지 않은 image 또는 memory token을 계산에서 제외
- **dynamic pruning:** 입력마다 다른 token이나 경로를 선택
- **memory pruning:** 오래되거나 품질 낮은 frame과 token을 memory에서 제거

Parameter 감소와 실제 속도

Weight를 많이 0으로 만들어도 GPU가 일반 dense 연산을 그대로 하면 실제 FPS가 거의 늘지 않을 수 있다. Structured pruning이나 hardware가 지원하는 sparsity가 실제 가속으로 이어지기 쉽다.

9.4 Token Compression과 Sparse Retrieval

Video model은 많은 frame의 고해상도 token을 memory에 저장하면 비용이 커진다. 이를 줄이는 방법은 다음과 같다.

- 평균 또는 attention pooling으로 token 수 축소
- target 주변 token만 유지
- 현재 frame과 관련도 높은 top-k memory만 읽기
- 배경 window는 작은 shortcut으로 처리
- uncertainty가 높을 때만 dense path 사용

속도와 정확도의 trade-off를 보여 주려면 top-k 값에 따른 J&F, latency, VRAM을 함께 그리는 것이 좋다.

9.5 Wavelet

한 줄 정의**:** 이미지를 위치 정보가 남아 있는 여러 주파수 대역과 scale로 분해하는 신호처리 방법.

2D wavelet transform은 흔히 다음 네 성분을 만든다.

- **LL:** 가로와 세로 모두 낮은 주파수. 큰 형태와 부드러운 영역
- **LH, HL:** 한 방향의 빠른 변화. 가로 또는 세로 경계
- **HH:** 두 방향의 높은 주파수. 가는 세부, 모서리, noise

비유**:** 사진을 “전체적인 색과 큰 형태” 악보와 “급격한 변화와 세부 장식” 악보로 나누어 보는 것.

왜 ***segmentation***에 쓰는가

Encoder downsampling에서 사라지기 쉬운 경계와 texture를 high-frequency branch로 보존하고, semantic feature와 다시 결합할 수 있다.

한계

High-frequency에는 유용한 경계뿐 아니라 sensor noise와 압축 artifact도 들어 있다. 단순히 고주파를 모두 강화하면 오히려 거친 false boundary가 생길 수 있다. Attention이나 gate로 필요한 주파수만 선택하는 이유다.

Fourier transform과 비교

Fourier transform은 전체 신호를 전역적인 주파수 성분으로 보는 데 강하고, wavelet은 위치와 scale 정보를 함께 유지하는 데 유리하다. 경계가 이미지 어디에 있는지 알아야 하는 segmentation에는 wavelet이 편리할 수 있다.

9.6 Optical Flow

한 줄 정의**:** 두 frame 사이에서 각 픽셀 또는 위치가 어느 방향으로 얼마나 이동했는지 나타내는 2D motion vector field.

각 픽셀에 (dx, dy)가 있다고 생각하면 된다. 이전 mask를 flow에 따라 이동시키는 것을 warping이라고 한다.

비유**:** 첫 사진의 작은 종이 조각마다 화살표를 붙여 다음 사진에서 어디로 움직였는지 표시한 지도.

활용

- 이전 mask를 현재 frame 위치로 warp해 prompt 생성
- motion consistency로 잘못된 mask 검사
- 움직이는 target과 정적 배경 구분
- camera motion 보정

한계

- 완전히 가려진 부분의 실제 motion은 관측할 수 없음
- 빠른 motion과 blur에서 matching 실패
- texture가 없는 영역과 반복 무늬에서 모호함
- 물체 경계에서 foreground와 background flow가 섞임
- 조명 변화가 실제 motion처럼 보일 수 있음

*Optical flow*와 *tracking* 비교

Optical flow는 frame 사이의 지역적 motion field를 계산한다. Tracking은 장시간 같은 객체 identity를 유지하는 더 큰 문제다. Flow만으로 긴 가림 뒤 같은 객체를 다시 찾기는 어렵다.

9.7 Kalman Filter

한 줄 정의**:** 이전 위치와 속도를 바탕으로 다음 상태를 예측하고, 새 관측이 오면 예측을 수정하는 확률적 추정 방법.

잠깐 target mask가 사라졌을 때 box 중심과 크기를 예측하는 데 사용할 수 있다. 계산이 매우 가볍다는 장점이 있다.

한계**:** 일정한 속도 같은 단순 motion 가정이 흔하므로 급격한 방향 전환, camera cut, 긴 가림에서는 예측이 빗나갈 수 있다. Kalman box는 identity를 이해하지 못하므로 비슷한 distractor를 구분하려면 appearance feature가 필요하다.

9.8 Tiling

초고해상도 항공 영상을 작은 tile로 나누어 처리하는 방법이다. GPU memory를 줄이고 작은 객체가 resize 과정에서 사라지는 것을 막을 수 있다.

하지만 tile 경계에 걸친 객체가 두 조각으로 분리되거나, 넓은 문맥을 잃어 비슷한 구조를 혼동할 수 있다. Overlap, contextual padding, seam merging, adaptive tile size가 이를 보완한다.

9.9 Mixed Precision

FP32와 FP16 또는 BF16 계산을 섞어 속도와 memory를 줄이는 방법이다. Modern GPU의 tensor core를 활용할 수 있다. 아주 작은 gradient가 사라지는 underflow를 막기 위해 loss scaling을 쓰기도 한다.

Mixed precision은 model 구조를 바꾸는 compression과 다르다. 같은 model을 더 낮은 계산 정밀도로 실행하는 최적화다.

9.10 Gradient Accumulation과 Gradient Checkpointing

- **gradient accumulation:** 작은 batch를 여러 번 처리한 gradient를 모아 큰 effective batch처럼 update
- **gradient checkpointing** 또는 **activation checkpointing:** 일부 activation을 저장하지 않고 backward 때 다시 계산해 VRAM 절약

둘 다 8 GB GPU에서 학습 가능성을 높이지만 training time은 늘 수 있다. 이름에 checkpoint가 들어가도 “모델 저장 파일”과는 다른 개념이다.

9.11 경량화 기법 한눈에 비교

- **PEFT —** 줄이는 대상: 학습되는 parameter · 보통 필요한 것: target data 학습 · 주의점: inference가 반드시 빨라지지는 않음
- **Distillation —** 줄이는 대상: 최종 student 크기 · 보통 필요한 것: teacher와 재학습 · 주의점: teacher 오류 전달

- **Quantization** — 줄이는 대상: 숫자 bit 수 · 보통 필요한 것: calibration 또는 QAT · 주의점: 작은 객체와 경계 손상 가능
- **Structured pruning** — 줄이는 대상: channel, layer, head · 보통 필요한 것: pruning과 보통 재학습 · 주의점: 무엇을 지울지 어려움
- **Token pruning** — 줄이는 대상: 입력별 token 계산 · 보통 필요한 것: saliency 또는 gate · 주의점: target token을 잘못 버릴 위험
- **Memory pruning** — 줄이는 대상: 저장하고 읽는 과거 정보 · 보통 필요한 것: quality와 relevance score · 주의점: 재등장 단서까지 지울 위험
- **Mixed precision** — 줄이는 대상: 연산 정밀도 · 보통 필요한 것: 지원 GPU와 안정화 · 주의점: 일부 연산은 FP32 유지 필요
- **Tiling** — 줄이는 대상: 한 번에 처리할 공간 크기 · 보통 필요한 것: merge 규칙 · 주의점: seam과 global context 문제

13. 특히 헷갈리는 개념 비교

13.1 Training-free, Zero-shot, Frozen, Prompt-free

- **Training-free method** — 실제 뜻: 제안 방법을 위해 추가 parameter 학습을 하지 않음 · 있어도 되는 것: pretrained model, search, threshold, optical flow
- **Zero-shot transfer** — 실제 뜻: target dataset label로 추가 학습하지 않음 · 있어도 되는 것: 대규모 pretraining, test-time prompt
- **Frozen module** — 실제 뜻: 해당 module weight를 업데이트하지 않음 · 있어도 되는 것: forward 계산, trainable adapter
- **Prompt-free inference** — 실제 뜻: 사람이 prompt를 직접 입력하지 않음 · 있어도 되는 것: automatic detector나 learned prompt generator

한 model이 동시에 frozen이고 zero-shot일 수도 있지만 네 용어는 서로 바꿔 쓸 수 없다.

13.2 Accuracy, Robustness, Efficiency

- **accuracy:** 정상 조건에서 prediction이 GT와 얼마나 맞는가
- **robustness:** corruption, domain shift, prompt noise, occlusion에서도 얼마나 유지되는가
- **efficiency:** time, memory, energy, parameter, 사용자 개입을 얼마나 아끼는가

“더 나은 성능”을 주장하려면 무엇이 더 나아졌고 무엇을 희생했는지 밝혀야 한다. J&F +1점과 FPS 2배를 얻었지만 저조도 성능이 크게 떨어졌다면 단일 숫자로 우열을 정하기 어렵다.

13.3 Confidence, Uncertainty, Correctness

- confidence가 높음: model이 스스로 확신
- uncertainty가 높음: model이 선택에 확신하지 못함
- correctness: 실제 GT와 맞음

Model은 틀렸는데 confidence가 높을 수 있다. 따라서 confidence threshold는 validation에서 calibration하고, 가능하면 여러 독립 신호로 검증한다.

13.4 Parameter 수, 파일 크기, VRAM, 속도

- Parameter 수가 줄면 checkpoint 저장 크기는 대체로 줄어든다.
- Training VRAM은 activation과 optimizer 때문에 별개다.
- FPS는 hardware와 kernel 최적화 영향을 크게 받는다.
- Token 수를 줄이면 parameter는 그대로여도 attention 속도가 빨라질 수 있다.

따라서 “parameter 90% 감소”만으로 “10배 빨라졌다”고 결론 내리면 안 된다.

13.5 Class-agnostic과 Open-vocabulary

- **class-agnostic segmentation:** class 이름과 무관하게 prompt된 foreground mask를 만듦
- **open-vocabulary segmentation:** text로 임의 class 이름을 지정하고 class와 mask를 연결

SAM 2 기본 능력은 class-agnostic promptable segmentation에 가깝다.

13.6 Fine-tuning과 Domain Adaptation

Fine-tuning은 pretrained weight를 추가 학습하는 행위다. Domain adaptation은 source-target 분포 차이를 줄이려는 문제 설정과 방법론이다. Target labeled data로 단순 fine-tuning해 domain 성능을 올리는 것도 넓게는 adaptation이지만, UDA처럼 별도 목표를 가진 연구와 구분해야 한다.

13.7 Detector box와 GT box

GT box는 정답 mask에서 만든 거의 완벽한 prompt다. Detector box는 class 누락, 위치 오차, 중복이 생긴다. SAM 2가 GT box에서 높은 성능을 냈다고 실제 자동 pipeline도 같은 성능을 낸다고 볼 수 없다.

13.8 Frame-wise Segmentation과 Video Propagation

- frame-wise: 각 frame을 독립 이미지로 처리
- video propagation: 과거 memory와 temporal relation을 사용

Frame-wise 방식은 memory contamination이 없지만 매 frame prompt가 필요할 수 있고 시간 일관성이 약하다. Propagation은 효율적이지만 한 번의 오류가 이어질 수 있다.

13.9 Semantic Segmentation과 SAM 2

Semantic segmentation은 모든 pixel에 class를 붙인다. SAM 2는 prompt가 가리키는 object mask를 만드는 category-agnostic model이다.

SAM 2 encoder에 U-Net decoder를 붙여 organ class를 자동 출력하면 그것은 SAM 2 backbone을 이용한 semantic model이지, 원래 promptable SAM 2 전체 기능을 개선한 것과는 다르다.

13.10 Image-only Improvement와 Video Improvement

Image encoder adapter가 static image mask를 개선해도 memory propagation, occlusion recovery, temporal consistency가 좋아졌다고 자동으로 말할 수 없다. Video 개선을 주장하려면 실제 영상 dataset과 temporal metric이 필요하다.

14. 논문 문장을 해석하는 연습

다음과 같은 문장을 보자.

> We freeze the Hiera image encoder and insert stage-specific depthwise-dilated adapters, while fine-tuning a lightweight boundary-aware decoder with Dice and BCE losses.

이를 부품별로 번역하면 다음과 같다.

1. **freeze the Hiera image encoder:** SAM 2 backbone weight는 업데이트하지 않는다.
2. **insert adapters:** 기존 층 사이에 작은 trainable module을 추가한다.
3. **stage-specific:** 서로 다른 해상도 stage가 같은 adapter를 공유하지 않는다.
4. **depthwise-dilated:** channel별 가벼운 convolution과 간격 있는 kernel로 여러 크기의 지역 정보를 본다.
5. **lightweight decoder:** 최종 mask를 만드는 작은 모듈을 학습한다.
6. **boundary-aware:** 객체 내부뿐 아니라 경계를 더 잘 맞추려는 구조 또는 loss가 있다.
7. **Dice and BCE losses:** 영역 겹침과 픽셀별 foreground 확률을 함께 학습한다.

이 문장만으로는 아직 다음 정보를 알 수 없다.

- Trainable parameter 수가 얼마인가?
- Prompt는 point, box, none 중 무엇인가?
- Static image인지 video인지?
- Memory encoder와 attention을 실제로 사용했는가?
- Input resolution과 GPU는 무엇인가?
- Baseline도 같은 training data와 prompt를 사용했는가?
- Boundary F가 실제로 개선되었는가?

논문 제목과 한 문장만 읽고 “SAM 2 전체를 개선했다”고 결론 내리지 말고, 구조 그림, training 설정, 평가 protocol을 함께 확인해야 한다.

15. 과제연구 논문을 읽는 순서

15.1 첫 번째 읽기 — 문제만 파악

1. 무엇이 실패한다고 주장하는가?
2. 입력과 출력은 무엇인가?
3. Image task인가 video task인가?
4. 실제 SAM 2의 어느 module을 사용하는가?

15.2 두 번째 읽기 — 방법을 부품으로 나누기

1. Backbone은 frozen인가?

2. 새 trainable module은 어디에 들어가는가?
3. Prompt를 누가 어떻게 만드는가?
4. Memory에 무엇을 쓰고 무엇을 읽는가?
5. Decoder와 output resolution이 바뀌는가?
6. Training-free인가, fine-tuning인가, test-time update인가?

15.3 세 번째 읽기 — 실험의 공정성 확인

1. Baseline과 같은 checkpoint인가?
2. 같은 dataset split과 resolution인가?
3. 같은 prompt 개수와 품질인가?
4. 같은 training data와 epoch인가?
5. 수치가 mean인지 best run인지?
6. 여러 random seed를 사용했는가?
7. 속도 측정 hardware가 같은가?

15.4 네 번째 읽기 — 한계와 확장 찾기

1. 어느 조건에서 실패하는가?
2. 새 module이 정말 필요한가, 단순 threshold로도 되는가?
3. Accuracy를 얻으며 speed나 generalization을 잃지 않았는가?
4. Component별 ablation이 충분한가?
5. 다른 domain과 긴 영상에서도 작동하는가?

16. SAM 2 과제연구를 설계할 때의 기본 원칙

16.1 "정확도를 측정한다"에서 "원인을 검증한다"로

약한 연구 질문:

> SAM 2는 이 dataset에서 정확도가 몇 점인가?

더 좋은 연구 질문:

> 저조도에서 첫 mask confidence가 떨어질 때 잘못된 frame이 memory에 기록되는 것이 장기 성능 하락의 주원인인가?

더 좋은 질문은 원인, 개입 방법, 비교군, 측정 지표가 함께 들어 있다.

16.2 한 번에 하나의 핵심 주장

예를 들어 memory 개선 연구라면 먼저 다음 최소 비교를 만든다.

1. Vanilla FIFO
2. Confidence-only gate
3. Diversity-only selection
4. Confidence와 diversity 결합

동시에 detector, LoRA, optical flow, boundary decoder를 모두 추가하면 성능은 오를 수 있지만 무엇이 원인인지 설명하기 어렵다.

16.3 Accuracy 외에 반드시 기록할 것

- J, F, J&F 또는 task에 맞는 정확도
- 객체 size와 corruption severity별 성능
- Prompt 수와 correction 수
- FPS 또는 latency
- Peak VRAM
- Trainable Params
- 실패 사례 영상
- Random seed와 평균·표준편차

16.4 8 GB급 GPU에서 현실적인 방향

- 작은 SAM 2 checkpoint부터 시작
- Backbone frozen
- Decoder-only 또는 adapter, LoRA
- 짧은 clip과 작은 batch
- Mixed precision
- Gradient accumulation과 필요 시 activation checkpointing
- 공개 checkpoint의 training-free memory policy 먼저 실험
- Full SA-1B 또는 SA-V 재학습은 피하기

Training-free memory selection, automatic re-prompting, lightweight boundary head, adapter 비교는 대규모 foundation model을 처음부터 학습하는 것보다 과제연구에 현실적이다.

PART C — SAM, SAM 2, SAM 2.1과 비디오 메모리

2. SAM: 한 장의 이미지에서 prompt를 mask로 바꾸는 모델

Segment Anything — ICCV 2023의 구조는 크게 세 부분이다.

```
이미지 ──> Image Encoder ──> image embedding ──────────────┐
                                                           ├─> Mask Decoder ─> masks + predicted IoU
점/box/mask ──> Prompt Encoder ──> prompt embedding ──────┘
```

2.1 Image encoder: 사진을 특징 지도(feature map)로 바꾸기

원본 SAM은 큰 Vision Transformer를 사용한다. 픽셀 RGB 값을 그대로 두고 mask를 그리는 것이 아니라, 이미지 전체를 여러 토큰과 feature map으로 바꾼다.

- 얕은 시각 특징: 선, 모서리, 색 변화, 작은 무늬
- 깊은 시각 특징: 물체의 부분, 형태, 주변 문맥

Image encoder는 무거운 부분이지만 한 이미지에 대해 한 번만 실행할 수 있다. 같은 사진에 사용자가 점을 여러 번 추가하더라도 image embedding을 재사용하고, 비교적 가벼운 prompt encoder와 mask decoder만 다시 돌릴 수 있다.

2.2 Prompt encoder: 사용자의 지시를 벡터로 번역하기

Prompt는 모델에게 무엇을 분할해야 하는지 알려 주는 입력이다.

- Positive point: 이 픽셀은 목표 안쪽이다.
- Negative point: 이 픽셀은 목표 바깥쪽이다.
- Box: 이 사각형 안의 물체를 찾으라는 힌트다.
- Mask: 이전 예측이나 사람이 그린 영역을 더 정밀한 dense prompt로 준다.

점과 box 같은 sparse prompt는 위치 encoding과 prompt 종류 embedding을 결합한다. Mask 같은 dense prompt는 convolution으로 낮은 해상도의 feature로 바꿔 image embedding과 결합한다.

2.3 Mask decoder: image와 prompt를 서로 대화시키기

SAM의 mask decoder는 two-way transformer를 사용한다.

1. Prompt token이 image token을 보며 지시와 관련된 픽셀이 어디인가?를 찾는다.
2. Image token도 prompt 정보를 받아 현재 사용자가 원하는 영역에 맞게 갱신된다.
3. Mask token이 동적 분류기처럼 작동해 각 위치가 foreground일 가능성을 계산한다.
4. 낮은 해상도의 mask logits를 원본 크기에 가깝게 올려 최종 mask를 만든다.

two-way는 prompt가 image만 일방적으로 읽는 것이 아니라 prompt와 image 표현이 서로 갱신된다는 뜻이다.

2.4 왜 candidate mask가 여러 개인가?

자전거 바퀴에 점 하나를 찍으면 정답은 하나로 고정되지 않는다.

- 타이어만 선택했을 수 있다.
- 바퀴 전체를 선택했을 수 있다.
- 자전거 전체를 선택했을 수 있다.

SAM은 이런 구조적 모호성을 평균 내어 흐릿한 한 mask로 만들지 않고, 보통 여러 candidate mask를 낸다. 원 논문에서는 흔한 part–subpart–whole 모호성을 다루기 위해 세 후보가 충분하다고 보았다.

2.5 Predicted IoU는 실제 IoU가 아니다

실제 IoU는 정답 mask (G)가 있을 때만 계산할 수 있다.

$$IoU(M,G)=(|M\cap G|)/(|M\cup G|)$$

실제 서비스 추론에서는 (G)가 없으므로 IoU를 알 수 없다. SAM이 내는 predicted IoU는 자기 **mask**가 실제 정답과 얼마나 겹칠 것인지 모델이 추정한 품질 점수다. 따라서 다음을 구분해야 한다.

- Actual IoU: 정답으로 사후 계산한 평가값
- Predicted IoU: 정답 없이 모델이 예측한 자기평가값

높은 predicted IoU가 항상 올바른 identity를 뜻하지는 않는다. 다른 사람을 아주 깔끔하게 분할하면 mask 모양 품질은 좋아 보여도 목표 identity는 틀릴 수 있다.

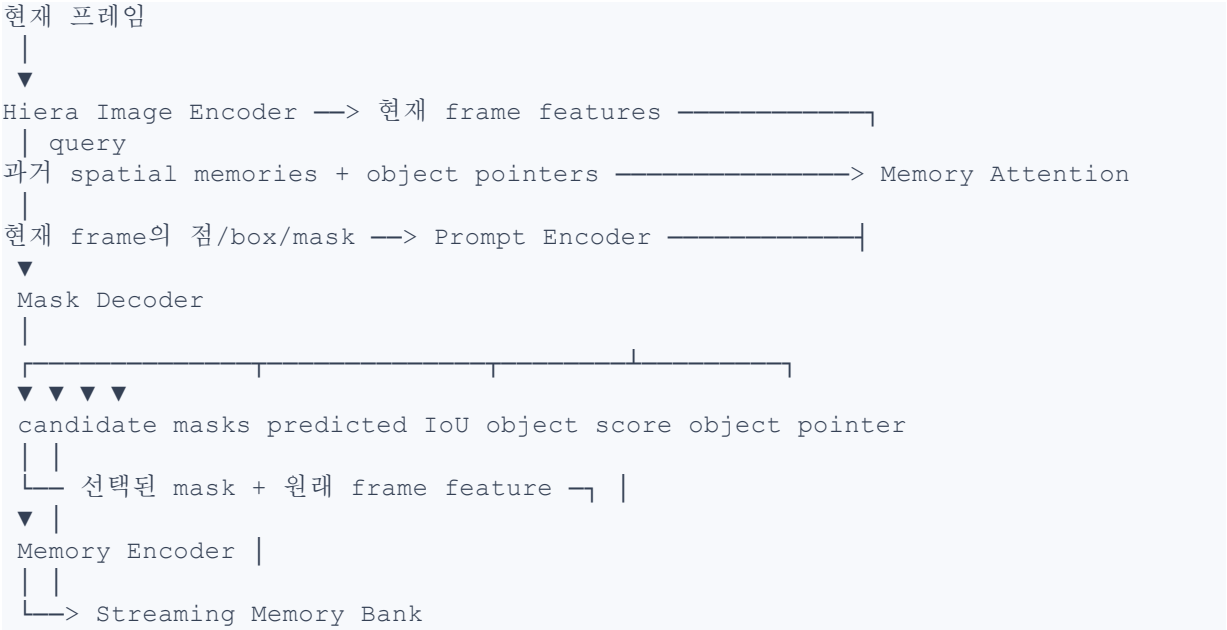
2.6 한 장 이미지의 실제 추론 흐름

1. 이미지를 resize·normalize한다.
2. Image encoder가 image embedding을 만든다.
3. 사용자의 점·box·mask를 prompt encoder가 embedding으로 만든다.
4. Mask decoder가 candidate masks와 predicted IoU를 출력한다.
5. 보통 predicted IoU가 가장 높은 후보를 기본 mask로 표시한다.
6. 사용자가 positive/negative point를 더 주면 같은 image embedding으로 다시 decoding한다.

SAM에는 시간축 기억이 없다. 다음 이미지가 들어오면 이전 mask는 자동으로 이어지지 않는다.

3. SAM2: SAM에 시간축과 기억을 추가하다

SAM 2 — ICLR 2025는 이미지를 프레임이 하나뿐인 비디오로 보고 image와 video를 하나의 구조로 처리한다.



3.1 Hiera image encoder: 여러 크기의 시각 정보를 동시에 만들기

SAM2는 MAE로 사전학습된 **Hiera** 계층형 encoder를 사용한다. `hierarchical`은 네트워크가 진행될수록 해상도는 낮추고 표현은 더 추상적으로 만든다는 뜻이다.

- Stage 1, stride 4: 해상도가 높아 얇은 경계와 작은 구조에 유리하다.
- Stage 2, stride 8: 중간 크기 세부 정보를 담는다.
- Stage 3, stride 16: 더 넓은 문맥과 물체 수준 정보를 담는다.
- Stage 4, stride 32: 가장 넓고 추상적인 정보를 담는다.

SAM2는 FPN으로 stride 16·32 feature를 융합해 주 image embedding을 만든다. Stride 4·8의 고해상도 feature는 memory attention을 통과하지 않고 mask decoder의 upsampling 단계로 skip connection되어 경계 복원에 도움을 준다.

쉬운 비유로는 다음과 같다.

- 고해상도 feature: 확대경으로 보는 털·끈·가장자리
- 저해상도 deep feature: 멀리서 보는 전체 형태와 문맥

3.2 Prompt encoder: SAM과 거의 같은 입력 언어

SAM2의 prompt encoder는 SAM 설계를 따른다.

- positive/negative click
- bounding box
- mask prompt

비디오에서는 첫 프레임뿐 아니라 어느 프레임에서든 새 prompt를 줄 수 있다. 중간 프레임에서 오류를 수정하면 그 정보가 새로운 conditioning memory가 되어 다른 프레임 예측을 고칠 수 있다.

3.3 Memory attention: 현재 장면을 과거 기억으로 조건화하기

현재 프레임 encoder feature만 보면 지금 보이는 물체가 처음 지정한 바로 그 물체인가?를 알기 어렵다. Memory attention은 현재 feature를 query로 하고 과거의 spatial memory와 object pointer를 key/value처럼 참고한다.

기본적으로 각 memory-attention block은 다음 순서를 따른다.

1. 현재 frame token끼리 self-attention한다.
2. 과거 spatial memory와 object pointer에 cross-attention한다.
3. MLP로 feature를 갱신한다.

공개 기본 구조는 이 block을 4층 사용한다. 결과는 단순한 현재 image feature가 아니라 과거에 지정된 목표를 고려한 현재 feature다.

주의: attention은 정답을 논리적으로 검색하는 데이터베이스 조회가 아니다. 현재 feature와 닮은 memory를 강하게 참고하는 학습된 유사도 연산이므로, memory가 잘못되면 오히려 오답을 증폭할 수 있다.

3.4 Mask decoder가 내는 네 종류의 핵심 출력

3.4.1 Candidate masks

현재 prompt와 memory에 맞는 여러 mask 후보다. 비디오에서도 part/whole 모호성과 distractor 모호성이 있으므로 여러 후보가 생성될 수 있다.

3.4.2 Predicted IoU

각 candidate mask의 예상 분할 품질이다. 모델이 추정하는 점수이지 실제 정답 IoU가 아니다. 기본 SAM2는 추가 prompt가 모호성을 해결하지 않으면 현재 프레임에서 predicted IoU가 가장 높은 mask 하나를 다음 전파에 사용한다.

3.4.3 Object score 또는 *occlusion/presence score*

현재 프레임에 목표가 보이는지 추정하는 별도 head의 출력이다. SAM에서는 positive prompt가 주어진 정지 이미지에 유효한 영역이 항상 있다고 가정하기 쉽지만, 비디오에서는 목표가 완전히 가려지거나 화면 밖으로 나갈 수 있다.

이 점수도 완벽한 확률이 아니다. 가려진 목표를 보임으로 잘못 판단하거나 비슷한 distractor를 목표로 판단할 수 있다.

3.4.4 Object pointer

선택된 mask에 해당하는 mask token을 변환한 가벼운 벡터다. Spatial memory가 어느 픽셀에 무엇이 있었는가를 담는 지도라면, object pointer는 이 프레임에서 모델이 목표라고 요약한 것을 담는 압축된 명함에 가깝다.

원문 구현은 256차원 pointer를 64차원 네 token으로 나누어 memory attention에서 사용한다. Pointer는 identity에 도움이 되지만, 잘못된 mask token에서 만들어지면 잘못된 대상의 명함이 된다.

3.5 Memory encoder: 현재 예측을 미래가 읽을 기억으로 바꾸기

Memory encoder는 별도의 무거운 image encoder를 다시 돌리지 않는다.

1. 선택된 mask logits를 convolution으로 downsample한다.
2. 이를 현재 프레임의 **unconditioned Hiera feature**와 위치별로 결합한다.
3. 가벼운 convolution block으로 융합한다.
4. 압축된 spatial memory feature를 만든다.

즉, memory에는 이미지 자체만 들어가는 것도 아니고 mask만 들어가는 것도 아니다. 그 프레임의 시각 특징과 모델이 목표라고 칠한 위치가 결합된 표현이 들어간다. 그래서 wrong mask를 encode하면 잘못된 위치의 시각 특징이 목표 정보로 묶인다.

3.6 Streaming memory bank: 모든 과거를 저장하지 않는 이유

수천 프레임을 모두 attention에 넣으면 시간과 메모리가 계속 증가한다. SAM2는 bounded streaming memory를 사용한다.

- Prompted/conditioning memory: 사용자가 직접 prompt를 준 프레임의 기억
- Non-conditioning memory: 모델이 자동으로 예측한 최근 프레임의 기억
- Object pointers: 각 프레임 mask token에서 나온 고수준 벡터

원 논문은 최근 memory를 FIFO로 관리한다. 공개 기본 코드의 `num_maskmem=7`은 전형적인 first-frame-only VOS에서 입력/조건 프레임 **1**개 **+** 최근 프레임 **6**개를 뜻한다. 추가 prompt를 여러 프레임에 주는 interactive setting은 conditioning memory 선택이 더 복잡해질 수 있다.

3.7 SAM2.1은 SAM2와 완전히 다른 구조인가?

아니다. SAM2.1은 같은 큰 구조를 유지하면서 학습과 일부 positional/memory 설정을 강화한 업데이트다. Meta의 공식 설명에 따르면 다음이 개선되었다.

- 비슷하게 생긴 물체와 작은 물체를 더 잘 다루도록 data augmentation 강화

- 더 긴 frame sequence로 학습해 occlusion 처리 개선
- spatial memory와 object pointer memory의 positional encoding 조정
- 더 강한 updated checkpoint 제공

ICLR 2025 최종 SAM2 논문의 결과는 사실상 개선된 SAM2.1 버전을 사용하지만, 논문에서는 간단히 SAM2라고 부른다. 따라서 논문·코드 비교 시 `SAM2`, `SAM2.1`, `SAM2.1Long`이 정확히 어떤 checkpoint와 config를 뜻하는지 확인해야 한다.

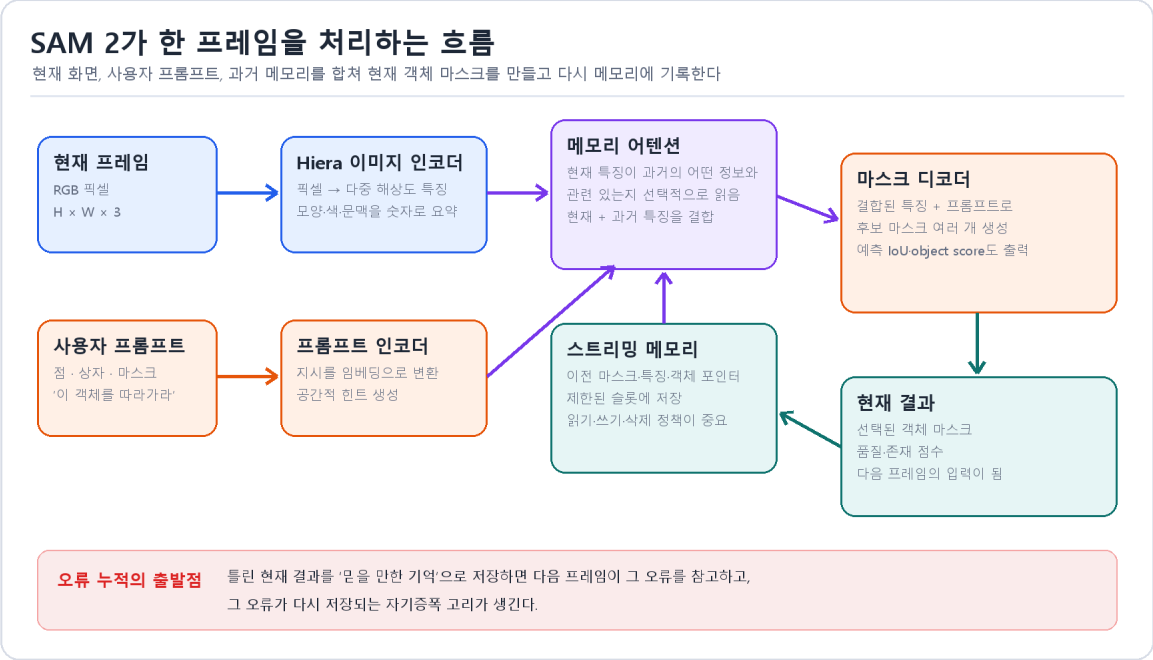


그림 *2. SAM 2*의 한 *frame* 처리 흐름*:* 현재 *feature*와 *prompt*가 과거 *memory*를 읽고, 결과 *mask*가 다시 미래 *memory*가 되는 순환 구조다.

4. 정지 이미지와 비디오에서 실제 추론 흐름

4.1 단일 이미지 추론

1. Hiera encoder가 한 장의 multiscale feature를 만든다.
2. 과거 memory가 없으므로 memory attention에는 실질적인 과거 정보가 없다. 모델은 no-memory 상태로 동작한다.
3. Prompt encoder가 점·box·mask를 변환한다.
4. Mask decoder가 후보 mask와 predicted IoU를 낸다.
5. 기본 후보를 표시하고 추가 prompt가 들어오면 같은 image feature로 다시 decoding한다.

이 경우 memory encoder와 streaming bank는 다음 프레임 전파를 위해 꼭 사용할 필요가 없다. 그래서 SAM2는 image에 적용될 때 SAM처럼 행동한다.

4.2 첫 비디오 프레임

사용자가 프레임 1의 강아지에 positive click 또는 mask를 준다고 하자.

1. Frame 1을 Hiera encoder로 변환한다.
2. 과거 memory 없이 prompt를 이용해 mask 후보를 만든다.
3. 후보 중 하나를 선택한다.
4. 선택 mask에서 object pointer를 만든다.
5. Frame 1 feature와 mask를 memory encoder에 넣는다.
6. 결과를 **conditioning memory**, 즉 강한 기준 기억으로 저장한다.

4.3 두 번째 이후 프레임

프레임 `t`의 전형적인 흐름은 다음과 같다.

1. 현재 frame `I_t`를 Hiera encoder로 변환해 `F_t`를 얻는다.
2. Memory attention이 `F_t`와 과거 memories·pointers를 융합해 memory-conditioned feature `F_t_tilde`를 만든다.
3. 현재 프레임에 새 prompt가 없다면, 과거 memory가 사실상 target prompt 역할을 한다.
4. Mask decoder가 후보 `M_t^1`, `M_t^2`, `M_t^3`, predicted IoU, object score, mask token을 출력한다.
5. 한 후보 `M_hat_t`를 선택한다.
6. `F_t`와 `M_hat_t`를 memory encoder에 넣어 새 memory `E_t`를 만든다.
7. `E_t`와 pointer를 bank에 넣고, recent queue가 가득 찼다면 가장 오래된 recent memory를 제거한다.

4.4 중간 프레임에서 사용자가 수정하는 경우

프레임 20에서 모델이 강아지를 놓쳤다고 하자. 사용자가 강아지 위에 positive click을 추가하면 다음이 가능하다.

1. Frame 20 mask를 다시 계산한다.
2. Frame 20이 새 conditioning frame이 된다.
3. 이 수정 정보를 사용해 이후 방향뿐 아니라 도구 설정에 따라 이전 방향으로도 다시 전파할 수 있다.

SAM2의 장점은 correction을 받을 때 매번 처음부터 새 tracker를 시작하지 않고 기존 object memory 문맥을 이용한다는 점이다.

4.5 여러 객체를 동시에 분할하는 경우

기본 SAM2는 모든 객체가 Hiera image feature는 공유하지만, 그 이후의 memory bank와 mask decoding은 객체별로 따로 수행한다. 객체 A와 B가 서로 겹친다는 사실을 명시적으로 토론하는 공동 memory가 기본 구조에는 없다.

따라서 다음 문제가 생길 수 있다.

- 두 object mask가 같은 distractor를 동시에 선택한다.
- A와 B가 교차할 때 ID를 맞바꾼다.
- 각 object score는 높지만 전체적으로 mask가 겹친다.

이는 multi-object association이나 inter-object communication이 별도 연구 주제가 되는 이유다.

5. 메모리 관리 용어를 정확히 이해하기

5.1 Autoregression: 오늘의 답이 내일의 입력이 되는 구조

SAM2의 비디오 추론은 넓은 의미의 autoregressive process다.

M_t=f(I_t, P_t, B_t-1)

B_t=Update(B_t-1, Encode(I_t,M_t))

- `I_t`: 현재 영상 프레임
- `P_t`: 현재 프레임 prompt, 없을 수도 있음
- `B_(t-1)`: 이전까지의 memory bank
- `M_hat_t`: 모델이 고른 현재 mask

현재 예측 `M_hat_t`가 memory를 바꾸고, 바뀐 memory가 다음 예측 `M_hat_(t+1)`에 영향을 준다. 언어 모델이 앞에서 만든 token을 뒤 token 생성에 쓰는 것과 비슷하지만, 여기서는 mask-conditioned memory가 전달된다.

5.2 FIFO: 먼저 들어온 **recent memory**가 먼저 나간다

FIFO는 First-In, First-Out이다. Recent queue가 `[F2, F3, F4]`로 가득 찬 상태에서 `F5`가 들어오면 가장 오래된 `F2`가 빠져 `[F3, F4, F5]`가 된다.

장점:

- 구현이 단순하다.
- 계산량과 VRAM이 영상 길이와 함께 끝없이 증가하지 않는다.
- 최근 외형·최근 위치 변화에 잘 적응한다.

단점:

- 최근 프레임이 가림·blur·오답으로 가득하면 좋은 기억이 빠르게 밀려난다.
- 오래전의 선명한 target 모습이 첫 prompt frame 하나를 제외하고 사라질 수 있다.
- 최근과 좋은 품질은 같은 개념이 아니다.

5.3 Anchor memory

Anchor는 target identity를 잃지 않게 붙잡아 두는 기준 기억이다. 논문마다 이름과 구현이 다르며, 기본 SAM2의 공식 모듈명은 아니다.

- 첫 prompt frame을 항상 남기는 가장 단순한 anchor
- 가림 전 고품질 frame을 추가 anchor로 보존
- 여러 시점·각도·크기의 대표 frame을 anchor set으로 보존
- 이미지 feature 대신 object prototype이나 text concept를 anchor로 사용

Anchor가 너무 오래되면 현재 외형과 달라지는 **anchor staleness**가 생긴다. 반대로 자동으로 anchor를 자주 갱신하면 오답이 anchor가 되는 위험이 있다.

5.4 Short-term memory와 long-term memory

- **Short-term** — 주목적: 최근 변화와 운동에 빠르게 적응 · 보통 담는 것: 직전 몇 프레임 · 장점: 자세·크기·조명 변화 반영 · 위험: 가림이 이어지면 오답으로 모두 오염될 수 있음
- **Long-term** — 주목적: 오래전의 안정적인 identity 보존 · 보통 담는 것: 첫 frame, 고품질·다양한 keyframe · 장점: 재등장과 장기 외형 비교에 유리 · 위험: 오래되어 현재 모습과 달라질 수 있고 selection이 어려움

기본 SAM2에는 prompted conditioning memory와 recent FIFO memory가 있다. 많은 후속 연구가 이를 더 명시적인 short/long, recent/anchor, appearance/distractor memory로 재구성한다.

5.5 Memory admission, write, read, retention, eviction, pruning

이 용어들을 섞지 말아야 연구 질문이 선명해진다.

- **Write** — 질문: 현재 정보를 memory 형태로 만들고 기록한다 · 예시: Frame 8 mask와 image feature를 encode해 bank에 넣음
- **Admission** — 질문: 이 후보를 아예 쓸 자격이 있는가? · 예시: predicted IoU가 높고 motion과 일치할 때만 write
- **Read/Retrieval** — 질문: 현재 frame이 어느 memory를 얼마나 참고하는가? · 예시: 최근 3개와 anchor 2개만 attention에 사용
- **Retention** — 질문: 이미 쓴 memory 중 무엇을 계속 보존하는가? · 예시: 첫 frame과 다양한 고품질 frame 유지
- **Eviction** — 질문: 공간이 부족할 때 무엇을 완전히 제거하는가? · 예시: 가장 오래되거나 중복된 frame 삭제
- **Pruning** — 질문: memory 내부의 frame/token 일부를 줄인다 · 예시: target과 관련 없는 background token 제거
- **Fusion** — 질문: 여러 memory의 정보를 어떻게 합치는가? · 예시: short/long 값을 uncertainty gate로 가중합
- **Refresh/Re-anchor** — 질문: 낡은 기준을 새 기준으로 갱신하는가? · 예시: 재등장 후 검증된 모습으로 새 anchor 추가

예를 들어 `memory 개선`이라는 말만으로는 부족하다. 다음처럼 구체화해야 한다.

> 본 연구는 mask 품질과 가림 상태를 이용해 **admission**을 제어하고, short/long memory의 **retention**을 분리하며, 재등장 때 prototype 기반 **retrieval**을 수행한다.

5.6 Memory 정책을 설계할 때의 네 가지 상충 관계

1. 적응성 대 안정성: 자주 쓰면 최신 외형에는 적응하지만 오답도 빨리 배운다.
2. 최근성 대 대표성: 최근 frame은 현재와 가깝지만 선명하고 정체성을 잘 보여 준다는 보장은 없다.
3. 정확도 대 계산량: 많은 memory와 후보를 읽으면 강건할 수 있지만 느려진다.
4. 빠른 복구 대 오탐 억제: 재탐색 threshold를 낮추면 빨리 찾지만 distractor를 target으로 채택할 수 있다.

6. 실패 용어와 인과 사슬

6.1 용어 정의

- **Occlusion,** 가림 — 정확한 뜻: 목표가 장면에는 있지만 다른 물체 뒤에 일부 또는 전부 가려짐 · 예시: 사람이 기둥 뒤를 지나감
- **Partial occlusion** — 정확한 뜻: 일부 픽셀만 보임 · 예시: 사람 상체만 보임
- **Full occlusion** — 정확한 뜻: 장면에 있지만 현재 RGB에서 목표 픽셀이 전혀 보이지 않음 · 예시: 공이 손 뒤에 완전히 숨음
- **Disappearance / out-of-view** — 정확한 뜻: 목표가 화면 밖으로 나가 현재 프레임에 존재하지 않음 · 예시: 자동차가 화면 오른쪽 밖으로 나감
- **Reappearance** — 정확한 뜻: 가림 또는 화면 이탈 후 다시 보임 · 예시: 다른 쪽에서 자동차가 다시 등장
- **Distractor** — 정확한 뜻: 목표와 헷갈릴 수 있는 비슷한 물체·부분·배경 · 예시: 같은 유니폼의 다른 선수, 셔츠의 로고
- **Drift** — 정확한 뜻: 예측이 시간에 따라 목표에서 서서히 벗어남 · 예시: 사람 전체→셔츠→옆 사람으로 이동
- **Identity switch** — 정확한 뜻: 추적 대상 ID가 다른 개체로 바뀜 · 예시: 선수 7번 대신 선수 9번을 계속 추적
- **Memory contamination** — 정확한 뜻: 틀린/부분/부재 frame을 목표 기억으로 저장해 bank가 오염됨 · 예시: 다른 선수의 feature를 target memory로 기록
- **Error accumulation** — 정확한 뜻: 앞 프레임 오류가 다음 프레임 조건이 되어 연쇄 확대됨 · 예시: 한 번의 잘못된 mask가 여러 bad memory를 생성
- **Target loss** — 정확한 뜻: 모델이 더 이상 원래 target을 안정적으로 지역화하지 못함 · 예시: 재등장해도 찾지 못함

6.2 가장 중요한 인과 사슬

```
가림·빠른 운동·blur·유사 객체 출현
 ↓
현재 프레임의 target 시각 증거가 약해짐
 ↓
여러 candidate mask가 모호해지거나 모두 같은 오답으로 치우침
 ↓
predicted IoU / object score가 오답을 충분히 경고하지 못함
 ↓
틀린 mask가 선택됨
 ↓
Memory encoder가 '틀린 위치의 image feature + 틀린 mask'를 target memory로 결합
 ↓
Memory contamination
 ↓
다음 프레임 memory attention이 틀린 영역을 더 강하게 참고
 ↓
다음 mask도 틀릴 확률 증가
 ↓
새로운 bad memory가 다시 기록됨
 ↓
FIFO에서 오래된 good recent memory가 밀려남
```

```
 ↓
Drift가 커지고 distractor로 identity switch
 ↓
원래 target이 reappear해도 오염된 memory 때문에 무시하거나 다른 개체를 계속 추적
```

6.3 왜 **predicted IoU**가 높은데 **identity switch**가 가능한가**?**

Predicted IoU head가 주로 답하려는 질문은 `이 mask의 모양과 경계가 그럴듯한가?`에 가깝다. 하지만 tracking에서 더 중요한 질문은 `이 mask가 처음 지정한 바로 그 개체인가?`이다.

같은 유니폼을 입은 다른 선수를 아주 정확한 경계로 분할하면 다음이 동시에 가능하다.

- mask boundary quality: 높음
- predicted IoU: 높을 수 있음
- target identity correctness: 틀림

따라서 identity를 지키려면 predicted IoU 외에도 appearance prototype, object pointer consistency, motion, neighbor/distractor 관계, forward-backward consistency 같은 신호가 필요하다.

6.4 **Object score**가 있는데도 왜 부재 처리가 어려운가**?**

Object score도 학습된 추정값이다. 다음 두 상황의 영상 픽셀이 비슷할 수 있다.

- target이 거의 보이지 않지만 작은 일부는 남아 있음
- target은 사라졌고 비슷한 distractor만 있음

너무 민감하게 absent를 선언하면 어려운 target을 놓치고, 너무 둔감하면 distractor를 target으로 memory에 쓴다. 그래서 최신 연구는 `보임/부재` 두 상태보다 `visible/uncertain/absent` 세 상태 또는 uncertainty-aware soft gate를 사용하려 한다.

6.5 **Failure**와 **memory contamination** 중 무엇이 원인인가**?**

항상 한 방향은 아니다.

- 첫 오답 mask가 contamination을 만든다.
- contamination이 이후 오답 확률을 높인다.
- 이후 오답이 더 많은 contamination을 만든다.

즉, feedback loop다. 연구에서는 이를 끊는 위치를 명시해야 한다.

- Decoder 단계: 더 좋은 후보 생성·선택
- Admission 단계: 의심 frame을 쓰지 않기
- Retention 단계: 오래된 좋은 기억 지키기
- Read 단계: 오염 가능 memory의 가중치 줄이기
- Recovery 단계: target을 전역 재탐색하고 재프롬프트하기

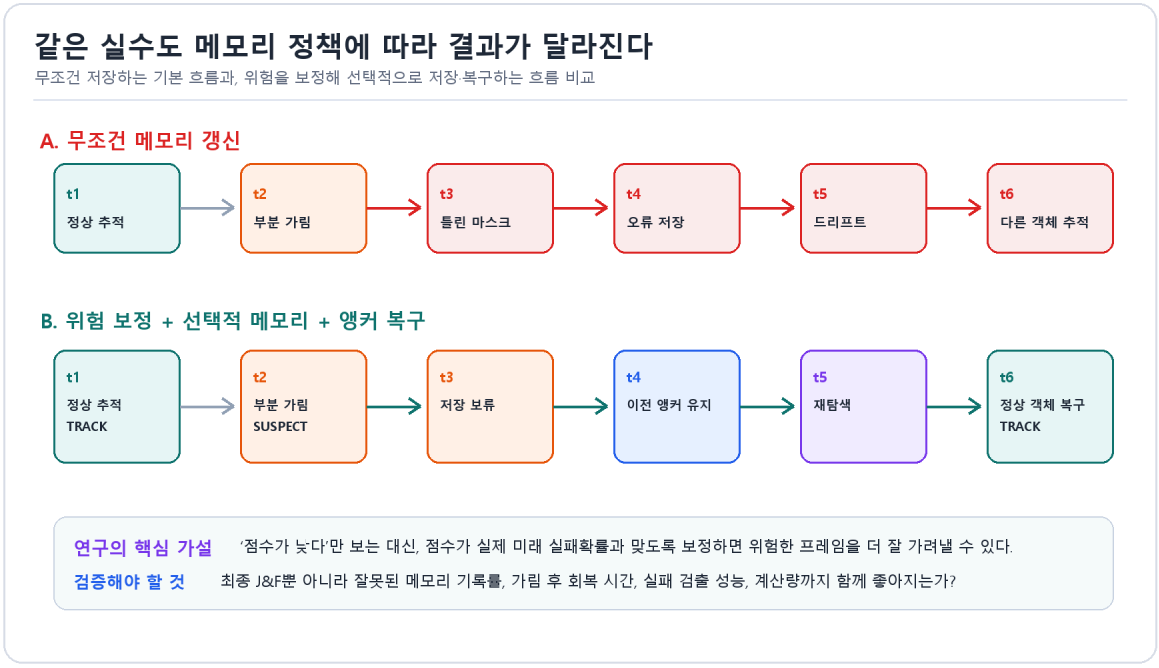


그림 *3.* 같은 부분 가림에서도 무조건 *memory*를 갱신하면 *drift*가 커지지만, 위험 기반 선택적 기록과 *anchor recovery*를 쓰면 잘못된 *identity*의 자기증폭을 막을 수 있다.

7. 12프레임 장난감 예시: 한 번의 가림이 **identity switch**가 되는 과정

7.1 상황과 표기

영상에는 처음 지정한 빨간 공 **A**와 매우 비슷한 빨간 공 **B**가 있다. 설명을 단순화하기 위해 recent memory를 3칸만 둔다. 실제 공개 SAM2 기본 설정은 전형적으로 first input frame과 최근 6개 frame을 사용한다.

- `A1`, `A2`: 해당 frame의 올바른 공 A memory
- `P4`: 부분 가림된 불완전한 A memory
- `B5`: distractor B를 target이라고 잘못 저장한 memory
- `Anchor A1`: 사용자가 지정한 첫 frame memory, recent FIFO와 별도 보존
- `[x,y,z]`: 오래된 것에서 최신 순서의 toy recent FIFO

7.2 프레임별 진행

- **1** — 실제 장면: 사용자가 공 A를 클릭. A가 선명함 · 기본 SAM2식 탐욕·FIFO에서 가능한 진행: A mask 생성, 첫 conditioning memory와 pointer 저장 · Toy recent memory: `[]`, Anchor=`A1` · 상태 기반 개선 정책: 동일. 장기 identity prototype도 A1에서 생성
- **2** — 실제 장면: A가 오른쪽으로 조금 이동 · 기본 SAM2식 탐욕·FIFO에서 가능한 진행: A2를 정확히 선택하고 write · Toy recent memory: `[A2]` · 상태 기반 개선 정책: reliable visible이므로 short memory에 write

- **3** — 실제 장면: A가 더 이동, 여전히 선명 · 기본 SAM2식 탐욕·FIFO에서 가능한 진행: A3를 write · Toy recent memory: `[A2,A3]` · 상태 기반 개선 정책: A2와 다른 각도라면 장기 keyframe 후보로 보존 가능
- **4** — 실제 장면: 손이 A의 70%를 가림 · 기본 SAM2식 탐욕·FIFO에서 가능한 진행: 작은 부분 mask P4의 predicted IoU가 가장 높아 선택·write될 수 있음 · Toy recent memory: `[A2,A3,P4]` · 상태 기반 개선 정책: area 급감·후보 disagreement·motion residual로 `uncertain`; 출력할 수는 있지만 write는 보류
- **5** — 실제 장면: A가 손 뒤에 완전히 가려짐. B는 화면에 보임 · 기본 SAM2식 탐욕·FIFO에서 가능한 진행: object score가 B를 target으로 오판하면 B5 선택·write, 가장 오래된 A2가 빠짐 · Toy recent memory: `[A3,P4,B5]` · 상태 기반 개선 정책: `absent`로 판단해 empty output 또는 출력 보류. Memory와 motion update를 freeze
- **6** — 실제 장면: A는 여전히 안 보이고 B만 움직임 · 기본 SAM2식 탐욕·FIFO에서 가능한 진행: 오염 memory B5가 B6 선택을 강화 · Toy recent memory: `[P4,B5,B6]` · 상태 기반 개선 정책: absent 유지, memory write 없음. 전역 재탐색은 하되 채택하지 않음
- **7** — 실제 장면: A는 아직 가려지고 B가 anchor A1과 비슷한 자세 · 기본 SAM2식 탐욕·FIFO에서 가능한 진행: B7 선택. 마지막 좋은 recent A3가 FIFO에서 사라짐 · Toy recent memory: `[B5,B6,B7]` · 상태 기반 개선 정책: 장기 A prototype은 보존. B 후보는 appearance는 비슷해도 가림 전 trajectory와 불일치하여 보류
- **8** — 실제 장면: A가 멀리 떨어진 오른쪽에 재등장, B도 왼쪽에 존재 · 기본 SAM2식 탐욕·FIFO에서 가능한 진행: 세 recent memory가 모두 B라 memory attention이 B8을 고름. Identity switch 성립 · Toy recent memory: `[B6,B7,B8]` · 상태 기반 개선 정책: presence 변화가 reappearance search를 trigger. Full-frame 후보 중 A1 prototype·motion·cycle consistency가 가장 맞는 A8 선택. 아직 즉시 memory write하지 않음
- **9** — 실제 장면: A와 B 모두 선명 · 기본 SAM2식 탐욕·FIFO에서 가능한 진행: 기본 모델은 B9를 계속 추적 · Toy recent memory: `[B7,B8,B9]` · 상태 기반 개선 정책: A8→A9가 2프레임 일관되면 rescue를 검증하고 A9를 short memory에 admission
- **10** — 실제 장면: 사용자가 기본 모델 오류를 보고 A에 새 click · 기본 SAM2식 탐욕·FIFO에서 가능한 진행: 새 conditioning prompt 덕분에 기본 모델도 A10으로 회복 가능 · Toy recent memory: B recent는 남아 있지만 새 Anchor=`A10` · 상태 기반 개선 정책: 자동 복구 모델은 이미 A를 추적. 새 user prompt가 있다면 가장 강한 anchor로 반영
- **11** — 실제 장면: A가 빠르게 이동 · 기본 SAM2식 탐욕·FIFO에서 가능한 진행: 새 conditioning memory로 A11 가능 · Toy recent memory: `[B8,B9,A11]` · 상태 기반 개선 정책: motion residual은 크지만 appearance·cycle이 맞으면 reliable write
- **12** — 실제 장면: A가 안정적으로 보임 · 기본 SAM2식 탐욕·FIFO에서 가능한 진행: B memory가 FIFO에서 점차 빠지며 정상화 · Toy recent memory: `[B9,A11,A12]` · 상태 기반 개선 정책: A short/long memory가 모두 정상. 자동 복구 성공

7.3 이 예시에서 측정해야 할 것은 평균 **IoU**만이 아니다

- Frame 4: bad-write를 사전에 탐지했는가?
- Frame 5–7: target 부재 중 false-positive mask를 얼마나 출력했는가?

- Frame 8: reappearance를 몇 frame 만에 알아냈는가?
- Frame 8–9: 공 B로 identity switch 했는가?
- Frame 10: 사람의 correction이 없으면 회복 가능한가?
- Recent memory 중 실제 target이 아닌 entry 비율은 얼마인가?

이런 사건 기반 지표가 있어야 `평균 J&F가 0.5 올랐다`보다 왜 개선되었는지 설명할 수 있다.

8. 주요 후속 연구: 문제–아이디어–남은 한계

8.1 한눈에 보는 지도

- **SAM2Long** — 주로 건드리는 지점: 후보 선택 + 각 경로별 memory · 초보자용 한 줄 비유: 하나의 추적 일기 대신 가능성 높은 여러 일기를 병렬로 써 둔다.
- **DAM4SAM** — 주로 건드리는 지점: Admission + recent/anchor memory · 초보자용 한 줄 비유: 최근 사진첩과 범인 식별용 기준 사진첩을 따로 둔다.
- **SAMURAI** — 주로 건드리는 지점: Motion + memory selection · 초보자용 한 줄 비유: 시각 정보에 속도·방향을 알려 주는 GPS를 붙인다.
- **UAMP** — 주로 건드리는 지점: Appearance/motion uncertainty + short/long memory · 초보자용 한 줄 비유: 자신 없는 부분은 과거와 현재를 다르게 섞고 장·단기 기억을 나눈다.
- **OASAMT** — 주로 건드리는 지점: Explicit occlusion state + dual memory · 초보자용 한 줄 비유: 보임·가림 상태를 먼저 판정한 뒤 상태별 서랍에 기억을 넣는다.
- **SENTRY** — 주로 건드리는 지점: Candidate verification before write · 초보자용 한 줄 비유: 일기에 쓰기 전에 앞뒤 장면과 맞는지 역검산한다.
- **Lean-SAM2** — 주로 건드리는 지점: Target-anchored pruning + insurance + sparse routing · 초보자용 한 줄 비유: 목표 기준 사진으로 쓸모없는 기억·계산을 줄이되 비상용 기억을 남긴다.
- **SAM2Dual** — 주로 건드리는 지점: Short/long + text-aware memory · 초보자용 한 줄 비유: 작업 기억과 장기 기억을 나누고 단어 힌트로 잘못된 기억을 약화한다.
- **StreamDAM** — 주로 건드리는 지점: Presence-controlled streaming policy · 초보자용 한 줄 비유: 목표가 있는지에 따라 쓰기·읽기·침묵·재탐색을 한 신호로 제어한다.
- **RUAC** — 주로 건드리는 지점: Pixel-wise uncertainty calibration · 초보자용 한 줄 비유: 모델의 자신감 온도계를 실제 오류와 맞게 교정한다. 영상 memory 연구는 아니다.

8.2 SAM2Long

출처: SAM2Long: Enhancing SAM 2 for Long Video Segmentation with a Training-Free Memory Tree — ICCV 2025

문제

기본 SAM2는 현재 predicted IoU가 가장 높은 mask 하나를 즉시 선택한다. 그 선택이 틀리면 다른 후보는 버려져 되돌아가기 어렵다.

아이디어

고정된 수의 segmentation path를 유지한다. 각 path가 현재 프레임의 여러 mask 후보로 분기하고, 누적 log predicted-IoU가 좋은 path만 beam search처럼 남긴다. Occlusion score가

불확실할 때 비슷한 path만 남지 않도록 mask diversity도 고려하고, 품질 좋은 object memory를 선택한다.

SAM2보다 나아지는 이유

프레임 하나의 판단을 즉시 확정하지 않고 몇 frame 뒤 증거를 보고 올바른 경로가 살아남을 수 있다. 가림 뒤 target이 재등장했을 때 과거에 남겨 둔 다른 가설로 복구할 가능성이 생긴다.

남은 한계

모든 path가 같은 distractor로 치우치면 도움이 되지 않는다. 점수 자체가 SAM2 predicted IoU에 많이 의존하며, 저자도 동적 배경·유사 옷에서 semantic identity 부족을 failure로 제시한다. 여러 path 계산과 영상 종료 후 최종 경로 선택은 엄격한 online streaming에 부담이다.

8.3 DAM4SAM

출처: A Distractor-Aware Memory for Visual Object Tracking with SAM2 — CVPR 2025

문제

기본 recent FIFO는 target이 없는 frame도 밀어 넣을 수 있고, 유사 distractor가 나타났을 때 최근 오답이 bank를 차지한다.

아이디어

RAM, Recent Appearance Memory와 DRM, Distractor Resolving Memory를 분리한다. RAM은 최근 정상 외형 적응용이고, DRM은 첫 frame과 신뢰도 높은 anchor를 보존한다. 선택 mask와 alternative mask의 차이를 tracker introspection 신호로 사용해 distractor 상황을 판단하고, object presence·predicted IoU·면적 안정성을 통과한 frame만 갱신한다.

SAM2보다 나아지는 이유

가장 최근이 아니라 최근이면서 믿을 수 있음과 distractor 해결에 유용함을 구분한다. Bad frame이 들어오는 것을 줄이고 좋은 기준 frame을 FIFO에서 보호한다.

남은 한계

여러 threshold가 수작업 값이다. SAM2의 모든 alternative mask가 같은 잘못된 개체를 가리키면 disagreement 신호가 약해진다. 같은 물체의 일부와 전체를 혼동하는 internal distractor도 완전히 해결되지 않는다.

8.4 SAMURAI

출처: SAMURAI: Adapting Segment Anything Model for Zero-Shot Visual Tracking with Motion-Aware Memory — arXiv 2024, 공식 코드

문제

SAM2는 위치·속도라는 명시적인 물리 운동 모델이 없고, fixed-window memory가 품질보다 최근성에 치우친다.

아이디어

Kalman filter가 target box의 위치와 크기를 예측한다. SAM2 후보 mask가 예측 운동과 얼마나 맞는지 motion score를 만들고, mask affinity·object score와 함께 후보와 memory를 선택한다. 추가 학습이 필요 없다.

SAM2보다 나아지는 이유

비슷하게 생긴 객체가 있어도 갑자기 먼 곳에 나타난 distractor보다 이전 속도·방향에서 자연스럽게 이어지는 후보를 선호한다.

남은 한계

Kalman filter의 단순 운동 가정은 급회전, 카메라 컷, 심한 변형, 긴 완전 가림에 약하다. 주요 결과가 box 기반 VOT benchmark이고 peer-reviewed venue가 확인되지 않은 preprint다.

8.5 UAMP

출처: UAMP: Consistent Video Object Segmentation with Uncertainty-Aware Memory Propagation — PLOS One 2026

문제

Occlusion·blur·빠른 운동에서는 frame의 일부 영역만 크게 변한다. 기본 SAM2는 어디가 불확실한지 세밀하게 표현하지 못하고 short recent memory만으로 장기 일관성을 잃을 수 있다.

아이디어

작은 convolution module이 appearance uncertainty map을 예측한다. 불확실한 영역과 안정된 영역에서 현재·memory feature를 다른 비율로 섞는다. Motion uncertainty, score-based short-term selection, recurrent linear-attention long-term memory도 결합한다.

SAM2보다 나아지는 이유

프레임 전체를 동일하게 믿지 않고 위치별 불확실성을 사용하며, 최근 적응과 장기 identity 보존을 분리한다.

남은 한계

Uncertainty module과 long-term memory를 학습해야 한다. 여러 모듈이 동시에 들어가므로 어느 요소가 향상을 만들었는지 엄격한 ablation이 필요하다. 어려운 조건의 정밀 경계 문제도 남는다.

8.6 OASAMT

출처: Occlusion-aware Visual Object Tracking with Explicit Temporal State Modeling and Dual-memory Mechanism — Pattern Recognition 2026

문제

SAM2는 object score 한 번으로 상태를 판단하지만, 가림은 시간에 걸쳐 시작·유지·종료된다. Occluded frame을 정상 frame과 같은 bank에 넣으면 identity가 오염된다.

아이디어

Temporal Occlusion Classifier가 confidence, mask IoU, area ratio의 시계열을 보고 occlusion state를 판정한다. Temporal Occlusion Predictor는 가림 중에도 box trajectory를 예측해 재등장 위치 prior를 만든다. Non-Occlusion Memory Bank와 Occlusion Memory Bank를 분리한다.

SAM2보다 나아지는 이유

현재 frame의 한 점수 대신 여러 frame의 변화 패턴으로 상태를 판단하고, 정상 identity memory를 가림 memory와 분리한다.

남은 한계

TCN 학습과 occlusion label이 필요하다. 주요 목적과 평가는 box VOT에 가깝고, 새로운 domain에서는 학습된 상태 분류기가 틀릴 수 있다.

8.7 SENTRY

출처: SENTRY: SAM2-Enhanced Neighbor-Aware and Temporally Reasoned Memory for Visual Tracking — arXiv 2026, ECCV 2026 채택·공식 코드

문제

Confidence가 높은 현재 mask라고 해서 시간적으로 올바른 target이라는 보장은 없다. 오답을 memory에 쓴 뒤에는 되돌리기 어렵다.

아이디어

`Refine before write`를 적용한다. SAM2 decoder 후보, automatic mask 후보, Kalman fallback으로 candidate pool을 만든다. 각 후보를 짧은 시간 뒤에서 앞으로가 아니라 현재에서 과거로 backtrack해 최근 target trajectory와 cycle-consistent한지 검사한다. 주변 distractor trajectory와 더 잘 맞는 후보는 벌점을 주고, 엄격히 검증된 severe rescue만 memory에 쓴다.

SAM2보다 나아지는 이유

현재 frame confidence만 보지 않고 여러 frame에 걸쳐 `이 후보를 과거로 되돌렸을 때 원래 target과 연결되는가?`를 검산한다.

남은 한계

후보 생성·역추적으로 계산량이 늘고 공식 결과에서 대형 모델 FPS가 약 23–26% 감소한다. 매우 긴 disappearance로 연결할 short tracklet 자체가 없거나 candidate generator가 target을 전혀 제안하지 못하면 구조상 복구가 어렵다.

8.8 Lean-SAM2

출처: Lean-SAM2: Target-Anchored Memory and Encoder Acceleration for SAM2 — arXiv v1, 2026-07-22, 공식 코드

문제

SAM2는 full-frame Hiera encoding과 memory cross-attention이 무겁다. 단순 attention-based pruning이나 window skipping은 occlusion·distractor 상황에서 중요한 target token을 버려 정확도가 크게 떨어질 수 있다.

아이디어

- TAMP, Target-Anchored Memory Pruning: 첫 prompt foreground에서 만든 anchor와 semantic consistency가 높은 memory token을 보호한다.
- TCIM, Temporal Condensation with Insurance Memory: 과거를 visibility gate로 압축하면서 고품질 entry는 별도의 insurance bank에 보관한다.
- TARR, Target-Anchored Risk-Aware Routing: target과 관련된 image window에 heavy encoder 계산을 집중하고, 위험한 변화가 감지되면 full-frame refresh로 되돌아간다.

SAM2보다 나아지는 이유

단순히 계산을 줄이는 것이 아니라 target anchor와 비상 memory를 사용해 잘못 줄여서 target을 잃는 위험을 같이 관리한다.

남은 한계

조사일 기준 매우 최근의 arXiv preprint다. 첫 prompt anchor가 시간이 지나 외형 변화로 낡으면 관련 token을 잘못 제거하거나 full-frame fallback이 자주 발생해 속도 이득이 줄 수 있다. 보고된 속도·정확도는 독립 재현이 필요하다.

8.9 SAM2Dual

출처: SAM2Dual: Training-Free, Dual Memory for Long-Term Video Object Segmentation — arXiv v1, 2026-08-19

문제

최근 unreliable prediction이 streaming memory를 지배하면 긴 occlusion·scene change 뒤 identity가 무너진다.

아이디어

빠른 local adaptation용 short-term memory와 간격 sampling으로 identity를 남기는 long-term memory를 분리해 gated fusion한다. 초반 frame에서 compact word cue를 뽑고 text embedding으로 memory 기여도를 재가중하는 Text-Aware Memory도 제안한다. Training-free plug-in 방식이다.

SAM2보다 나아지는 이유

최근 오답이 모든 기억을 덮는 것을 줄이고, 시각 증거가 약할 때 semantic cue로 관련 memory를 강조한다.

남은 한계

조사 시점 기준 공개 이틀 된 초기 v1 preprint다. 공, 사람 같은 category-level word는 같은 class의 서로 다른 instance를 구분하지 못한다. 논문 failure에서도 신발 target이 사람 전체로 퍼지는 part–whole drift가 나타난다.

8.10 StreamDAM

출처: StreamDAM: Presence-Aware Memory for Real-Time Streaming Video Object Segmentation — arXiv v1, 2026-08-04

문제

Offline benchmark는 frame 하나를 처리하는 데 시간이 얼마나 걸리든 기다린 뒤 결과를 평가한다. 실제 30 FPS stream에서는 deadline을 놓치면 과거 mask를 다시 내보내야 하고, target이 없는데도 mask를 출력하는 문제가 중요해진다.

아이디어

학습된 presence signal 하나가 다음을 frame별로 제어한다.

- 어떤 frame을 memory에 쓸지
- memory를 얼마나 멀리 읽을지
- 현재 출력을 보류할지
- 언제 global re-detection할지

SAM2보다 나아지는 이유

Fixed policy 대신 `지금 target이 실제로 사라진 것인지, 단지 보기 어려운 것인지`에 따라 계산과 출력을 바꾼다. Accuracy와 real-time deadline을 함께 본다.

남은 한계

최근 단독 저자의 v1 preprint로 peer review와 외부 재현이 없다. Presence model이 틀리면 write·read·output·re-detection 네 동작이 동시에 잘못될 수 있다.

8.11 RUAC

출처: Segment Anything with Robust Uncertainty-Accuracy Correlation — ICML 2026 프로젝트, arXiv

먼저 바로잡을 점

RUAC은 SAM2 장기 비디오 memory tracker가 아니다. 주 대상은 domain shift가 있는 이미지 **segmentation**의 **pixel-wise uncertainty calibration**이다.

문제

SAM2의 mask-level predicted IoU 한 숫자는 경계 일부의 오류와 pixel별 불확실성을 충분히 나타내지 못한다. 새로운 촬영 환경·texture·변형에서는 confidence와 실제 accuracy의 관계가 무너질 수 있다.

아이디어

Lightweight uncertainty head와 Bayesian mask decoder를 추가한다. Style attack과 deformation attack으로 appearance·geometry shift를 만들고, uncertainty가 실제 error pixel에 높아지도록 Uncertainty–Accuracy Alignment로 학습한다.

SAM2 연구에 쓸 수 있는 이유

RUAC uncertainty map을 그대로 쓰거나 더 단순한 대체 신호를 만들어 다음처럼 사용할 수 있다.

- 불확실 pixel 비율이 높으면 memory admission 금지
- 경계 uncertainty만 높으면 mask refinement 실행
- uncertainty가 급증하면 SENTRY식 검증이나 global re-detection 호출

남은 한계

추가 학습이 필요하고, 시간적 identity·occlusion state·reappearance를 직접 모델링하지 않는다. Pixel uncertainty가 높다는 사실만으로 target이 사라졌는지 distractor로 바뀌었는지는 알 수 없다.

8.12 추가로 연결되는 연구

- **HiM2SAM, PRCV 2025** — 핵심 연결점: Kalman coarse motion과 불확실 시 CoTracker3 fine motion, short/long memory · 남은 한계: Offline point tracker와 여러 threshold, box tracking 중심
- **SAMITE, arXiv 2025** — 핵심 연결점: Feature·position prototype으로 best memory 선택, positional pseudo-prompt와 cycle check · 남은 한계: Preprint, 초기 prototype이 틀리면 오류 강화 가능
- **SAM2RL, RLC 2025 workshop** — 핵심 연결점: Admission/eviction을 RL 순차 의사결정으로 최적화 · 남은 한계: Video마다 agent를 따로 overfit한 upper-bound라 일반화 tracker가 아님
- **IMTrack, EAAI 2026** — 핵심 연결점: Keyframe으로 re-detection memory를 미리 warm-up하고 loss 후 global search · 남은 한계: Warm-up·전역 탐색 비용, box recovery 중심
- **SeC, ICLR 2026** — 핵심 연결점: 새 scene에서 LVLM이 object-centric concept를 만들고 semantic identity를 보강 · 남은 한계: 4B급 모델과 학습이 무거우며 same-class instance 구분은 여전히 어려움

8.13 선행연구를 관통하는 결론

한 방법이 모든 문제를 해결하지는 않는다.

- SAM2Long: 잘못된 결정을 늦춘다.
- DAM4SAM·SENTRY: 잘못된 기억이 들어가는 것을 막는다.
- SAMURAI·HiM2SAM: 위치·운동 정보를 보강한다.
- UAMP·RUAC: 불확실성을 더 잘 표현한다.
- OASAMT·StreamDAM: 대상 상태를 명시적으로 제어한다.
- IMTrack: 잃은 뒤 다시 찾는 과정을 따로 둔다.
- SAM2Dual·SeC: 의미와 장기 identity를 보강한다.
- Lean-SAM2: 정확도를 지키며 memory·encoder 계산을 줄이려 한다.

가장 큰 공백은 **bad write** 예방**,** 명시적 **absent** 상태**,** 재등장 후 **identity** 검증을 하나의 가벼운 구조에서 함께 다루는 것이다.

9. 기본 **SAM2** 흐름의 교육용 의사코드

아래 코드는 실제 Meta 코드의 모든 예외와 tensor 연산을 재현한 것이 아니라, 정보 흐름을 이해하기 위한 의사코드다.

```
def run_sam2_video(frames, prompts):
 conditioning_memory = # 사용자가 prompt를 준 frame
 recent_memory = FIFO(maxlen=6) # 자동 예측한 최근 frame
 object_pointers = []
 for t, frame in enumerate(frames):
 # 1. 현재 영상 자체의 시각 특징. 같은 frame에서는 재사용 가능
```

```
image_features = hiera_image_encoder(frame)
# 2. 과거 spatial memory와 object pointer로 현재 특징을 조건화
conditioned_features = memory_attention(
current=image_features,
spatial_memories=list(conditioning_memory.values()) +
 list(recent_memory.values()),
object_pointers=object_pointers,
)
# 3. 현재 frame에 user prompt가 있으면 encoding
prompt_features = prompt_encoder(prompts.get(t))
# 4. 여러 mask, 예상 품질, 존재 점수, mask token 출력
masks, predicted_ious, object_score, mask_tokens = mask_decoder(
conditioned_features,
prompt_features,
high_resolution_skip=image_features.high_res,
)
# 5. 기본적으로 predicted IoU가 가장 높은 후보 선택
k = argmax(predicted_ious)
selected_mask = masks[k]
pointer = project_to_object_pointer(mask_tokens[k])
# 6. 현재 frame + 선택 mask를 미래용 spatial memory로 encoding
spatial_memory = memory_encoder(
unconditioned_image_features=image_features,
selected_mask=selected_mask,
object_score=object_score,
)
# 7. Prompt frame과 자동 추적 frame을 구분해 저장
if t in prompts:
conditioning_memory[t] = spatial_memory
else:
recent_memory.push(t, spatial_memory) # 가득 차면 가장 오래된 recent 제거
object_pointers.append(pointer)
# 8. Object가 없다고 판단하면 빈 mask를 표시할 수 있음
if object_score < PRESENCE_THRESHOLD:
yield t, empty_mask_like(selected_mask)
else:
yield t, selected_mask
```

핵심 위험은 5–7단계다. 5단계에서 오답이 선택되면 6단계가 오답을 target memory로 바꾸고, 7단계가 이를 다음 frame의 조건으로 만든다.

10. 불확실성·상태 기반 개선안의 의사코드

다음은 여러 선행연구를 베끼는 완성 알고리즘이 아니라, 연구 질문을 분명하게 보여 주는 최소 통합 예시다.

```
def robust_update(frame, sam2_output, tracker_state):
 masks = sam2_output.masks
 iou_score = sam2_output.predicted_iou
 presence = sam2_output.object_score
 pointer = sam2_output.object_pointer
 best = masks[argmax(iou_score)]
 # SAM2 기본 출력 외의 검증 신호
 area_jump = relative_area_change(best, tracker_state.last_reliable_mask)
```

```
motion_error = distance_to_motion_prediction(best,
 tracker_state.motion_model)
pointer_shift = cosine_distance(pointer, tracker_state.identity_pointer)
disagreement = diversity_between_candidate_masks(masks)
cycle_error = optional_forward_backward_error(best, tracker_state.history)
# 예시용 reliability. 계수와 threshold는 실험으로 정해야 함
reliability = (
+ 1.0  iou_score.max()
+ 0.8  sigmoid(presence)
- 0.7  area_jump
- 0.8  motion_error
- 0.9  pointer_shift
- 0.6  cycle_error
)
state = classify_state(
presence=presence,
reliability=reliability,
disagreement=disagreement,
recent_states=tracker_state.recent_states,
) # VISIBLE, UNCERTAIN, ABSENT
if state == VISIBLE and reliability > WRITE_THRESHOLD:
output = best
write_short_term_memory(best)
update_motion_model(best)
if is_high_quality(best) and is_diverse_from_long_term(best):
retain_as_long_term_anchor(best)
elif state == UNCERTAIN:
# 화면에는 출력할 수 있지만, 미래를 오염시키지 않도록 쓰지 않음
output = best
freeze_memory_write()
keep_motion_covariance_growing()
else: # ABSENT
output = EMPTY_MASK
freeze_memory_write()
do_not_learn_from_distractors()
proposals = global_candidate_search(frame)
rescue = select_by(
proposals,
long_term_appearance_similarity=True,
motion_gate=True,
cycle_consistency=True,
)
if rescue.is_consistent_for_multiple_frames():
output = reprompt_sam2_with(rescue)
admit_verified_rescue_to_memory()
tracker_state.recent_states.append(state)
return output, tracker_state
```

이 의사코드의 연구 가설

신뢰도가 낮은 frame은 출력과 memory write를 분리하고, 부재 상태에서는 memory를 동결하며, 재등장 후보를 장기 identity와 시간 일관성으로 검증한 뒤 쓰면 SAM2의 memory contamination과 recovery delay를 동시에 줄일 수 있다.

11. 과제연구에서 모듈별로 검증하는 법

한 번에 모든 기능을 넣으면 무엇이 효과가 있었는지 알 수 없다. 다음 순서로 ablation한다.

- **A0** — 추가되는 요소: SAM2.1 baseline · 확인할 질문: 기본 오류와 처리 비용은?
- **A1** — 추가되는 요소: Confidence admission · 확인할 질문: 낮은 predicted IoU frame을 쓰지 않는 것만으로 충분한가?
- **A2** — 추가되는 요소: + area/motion gate · 확인할 질문: 기하학적 이상을 더하면 bad-write 검출이 좋아지는가?
- **A3** — 추가되는 요소: + pointer consistency · 확인할 질문: Identity 변화 감지에 도움이 되는가?
- **A4** — 추가되는 요소: + visible/uncertain/absent state · 확인할 질문: 어려움과 실제 부재를 구분할 수 있는가?
- **A5** — 추가되는 요소: + short/long retention · 확인할 질문: 재등장 뒤 recovery가 빨라지는가?
- **A6** — 추가되는 요소: + global re-detection · 확인할 질문: 사람의 correction 없이 회복하는가?
- **A7** — 추가되는 요소: + cycle verification · 확인할 질문: 빠른 복구의 오탐·identity switch를 줄이는가?

표준 **mask** 품질 외에 필요한 지표

- J&F와 시간에 따른 J&F decay
- Drift onset frame
- Memory contamination rate
- Bad-write detection AUROC, precision, recall
- Target absence 중 false-positive mask area
- Identity switch count
- Reappearance 후 recovery delay
- Post-reappearance J&F
- User correction 횟수
- FPS, VRAM, frame당 추가 decoder/encoder 호출 수
- 30 FPS deadline-aware J&F

VOS의 J&F와 box VOT의 AUC/AO는 서로 다른 task protocol이므로 한 숫자 순위로 직접 섞지 않는다. 같은 SAM2.1 checkpoint, 같은 prompt, 같은 memory budget 아래에서 비교해야 memory policy 효과를 공정하게 볼 수 있다.

12. 자주 생기는 오해 바로잡기

오해 **1. Predicted IoU**가 **0.95**면 **95%** 확률로 정답이다

아니다. 실제 정답과의 IoU를 추정하도록 학습된 score이지, 모든 domain에서 완벽히 calibrated된 정답 확률이 아니다. Identity correctness와도 다르다.

오해 **2. Object pointer**에는 항상 올바른 **target identity**가 들어 있다

아니다. 선택된 mask token에서 나오므로 mask가 틀리면 pointer도 오염될 수 있다.

오해 **3. Memory**가 많을수록 항상 좋다

좋은 memory라면 도움이 되지만, 오염되거나 중복된 memory를 많이 읽으면 느려지고 오답 attention도 커질 수 있다.

오해 **4. Object score**가 있으므로 **full occlusion** 문제는 해결되었다

아니다. Object score 자체가 어려운 장면에서 오판할 수 있고, absent를 알아도 재등장 위치·identity를 자동으로 찾는 별도 과정이 필요하다.

오해 **5. SAM2.1**은 **SAM2**와 완전히 다른 세대의 구조다

아니다. 큰 구조는 같고 checkpoint, augmentation, longer-sequence training, positional/memory 설정이 강화된 업데이트다.

오해 **6. SAMEO**는 **SAM2**의 **occlusion tracker**다

아니다. SAMEO는 정지 이미지에서 보이지 않는 부분까지 추론하는 amodal segmentation 연구다. Temporal drift·reappearance tracker가 아니다.

오해 **7. RUAC**만 붙이면 장기 추적 문제가 해결된다

아니다. RUAC은 불확실성 측정을 개선하지만 temporal state와 re-detection은 별도로 설계해야 한다.

오해 **8. Training-free**는 아무 데이터도 사용하지 않았다는 뜻이다

아니다. 이미 사전학습된 SAM2 weight를 그대로 두고 추가 fine-tuning을 하지 않는다는 뜻이다. Threshold tuning, external pretrained model, benchmark 선택은 여전히 결과에 영향을 준다.

오해 **9. Zero-shot VOS**는 첫 **frame** 정보도 전혀 안 준다

문맥에 따라 다르다. Semi-supervised VOS는 첫 frame mask를 inference-time supervision으로 준다. `Zero-shot`은 보통 target dataset에 추가 학습하지 않았다는 뜻이지 prompt가 없다는 뜻이 아니다.

PART D — 분할 평가와 신뢰도·불확실성 보정

1. 한 장의 마스크에서 시작하기

1.1 Ground truth와 prediction

Ground truth, GT는 정답 마스크다. **Prediction**은 모델이 예측한 마스크다. 이진 객체 분할에서는 각 픽셀을 다음 두 값으로 나타낼 수 있다.

- `1`: 대상 객체, foreground
- `0`: 배경, background

다음은 5×5 이미지의 정답이다. 가운데 3×3 영역이 객체다.

```
GT
. . . . .
. 1 1 1 .
. 1 1 1 .
. 1 1 1 .
. . . . .
```

모델은 같은 크기의 객체를 오른쪽으로 한 칸 밀려 예측했다고 하자.

```
Prediction
. . . . .
. . 1 1 1
. . 1 1 1
. . 1 1 1
. . . . .
```

전체 픽셀은 25개다. 두 마스크를 겹쳐 보면 다음 네 종류가 생긴다.

1.2 TP, FP, FN, TN

- **TP, True Positive** — 뜻: 실제 객체이고 모델도 객체라고 한 픽셀 · 이 예시의 수: 6
- **FP, False Positive** — 뜻: 실제 배경인데 모델이 객체라고 잘못 칠한 픽셀 · 이 예시의 수: 3
- **FN, False Negative** — 뜻: 실제 객체인데 모델이 놓친 픽셀 · 이 예시의 수: 3
- **TN, True Negative** — 뜻: 실제 배경이고 모델도 배경이라고 한 픽셀 · 이 예시의 수: 13

검산하면 6+3+3+13=25다.

자주 하는 혼동

- FP는 “예측한 것 중 틀린 부분”이다.
- FN은 “정답 객체 중 놓친 부분”이다.
- positive가 무엇인지는 연구자가 먼저 정한다. 여기서는 foreground를 positive로 정했다.
- 다중 클래스에서는 각 클래스를 한 번씩 positive로 놓는 one-vs-rest 방식으로 계산할 수 있다.

1.3 Accuracy가 분할에서 위험한 이유

정확도는 다음과 같다.

Accuracy=(TP+TN)/(TP+FP+FN+TN)

=(6+13)/(25)=0.76

76%라고 하면 꽤 좋아 보인다. 하지만 객체 영역의 3분의 1을 놓쳤고, 객체 크기의 3분의 1만큼 배경도 잘못 포함했다. 배경이 아주 큰 이미지에서는 아무것도 분할하지 않아도 TN이 많아 accuracy가 높아질 수 있다. 그래서 객체 분할은 IoU, Dice, boundary metric을 주로 사용한다.

2. Precision과 Recall

2.1 Precision — 예측한 픽셀 중 얼마나 맞았는가

Precision=(TP)/(TP+FP)

=(6)/(6+3)=(2)/(3)≈0.667

모델이 객체라고 칠한 9픽셀 중 6픽셀이 진짜 객체다.

Precision이 낮다는 것은 mask가 배경으로 많이 번졌다는 뜻이다. 예를 들어 그림자, 비슷한 옷을 입은 다른 사람, 주변 물체까지 포함한 경우 FP가 증가한다.

2.2 Recall — 실제 객체 중 얼마나 찾았는가

Recall=(TP)/(TP+FN)

=(6)/(6+3)=(2)/(3)≈0.667

실제 객체 9픽셀 중 6픽셀을 찾았다.

Recall이 낮다는 것은 객체의 일부를 놓쳤다는 뜻이다. 가느다란 다리, 자전거 바퀴살, 가려진 부분처럼 어려운 영역에서 FN이 증가한다.

2.3 Precision–recall trade-off

마스크 threshold를 낮추면 더 많은 픽셀을 객체로 선택한다. 보통 recall은 높아지지만 FP도 늘어 precision이 낮아진다. threshold를 높이면 정반대가 된다. 따라서 threshold 하나의 결과만 보고 모델 전체가 우수하다고 말하면 안 된다.

3. IoU/Jaccard와 Dice

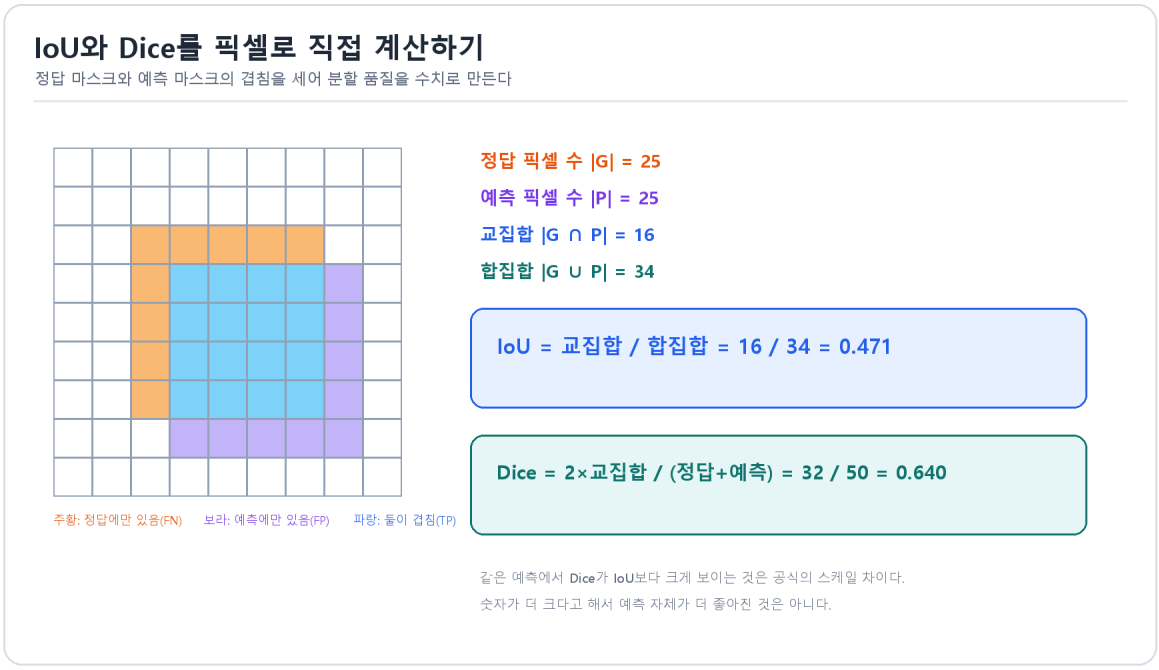


그림 *5.* 정답과 예측의 교집합·합집합을 직접 세어 *IoU*와 *Dice*를 계산하는 예시다.

3.1 IoU — Intersection over Union

IoU는 정답과 예측의 교집합을 합집합으로 나눈 값이다. Jaccard index와 같은 뜻이다.

IoU

=(|G∩ P|)/(|G∪ P|)

=(TP)/(TP+FP+FN)

예시에서는 다음과 같다.

IoU=(6)/(6+3+3)=(6)/(12)=0.5

TN이 공식에 들어가지 않는 점이 중요하다. 배경이 아무리 넓어도 객체의 겹침 정도만 평가한다.

3.2 Dice coefficient

Dice는 겹치는 영역을 두 배 한 뒤 두 마스크의 면적 합으로 나눈다.

Dice

=(2|G∩ P|)/(|G|+|P|)

=(2TP)/(2TP+FP+FN)

예시에서는 다음과 같다.

Dice=(2×6)/(12+3+3)

=(12)/(18)≈0.667

3.3 IoU와 Dice의 관계

같은 마스크 쌍에 대해 두 값은 다음처럼 변환된다.

Dice=(2IoU)/(1+IoU),

IoU=(Dice)/(2-Dice)

IoU가 0.5이면 Dice는 2×0.5/1.5=0.667이다. Dice가 숫자상 더 크게 보이지만 같은 정보를 다른 척도로 표시했을 뿐이다. Dice 0.8과 IoU 0.8을 직접 같은 수준이라고 해석하면 안 된다.

3.4 무엇을 평균하는가

한 영상에 여러 객체와 여러 프레임이 있으면 평균 방법을 명시해야 한다.

- 모든 픽셀을 한꺼번에 합쳐 계산하는 micro average
- 객체별 점수를 계산한 뒤 평균하는 macro average
- 프레임별로 계산한 뒤 평균하는 방식
- 시퀀스별 평균을 다시 평균하는 방식

작은 객체를 큰 객체와 동등하게 중요하게 보려면 객체별 또는 시퀀스별 macro average가 유용하다. 반대로 전체 픽셀 정확도가 목적이면 micro average가 적합할 수 있다. 논문 비교에서는 benchmark의 공식 평가 코드를 그대로 사용해야 한다.

4. Boundary F와 Boundary IoU

영역 IoU가 같아도 윤곽선 품질은 다를 수 있다. 머리카락, 자전거 바퀴, 얇은 도구, 글자 획에서는 경계 평가가 특히 중요하다.

4.1 경계 픽셀 구하기

입문 계산에서는 foreground 픽셀 중 상하좌우 4-neighbor에 배경이 하나라도 있는 픽셀을 boundary라고 하자. 3×3 정사각형에서는 가운데 한 픽셀만 내부이고 나머지 8픽셀이 경계다.

앞의 GT와 prediction은 각각 경계 픽셀이 8개다. tolerance를 0으로 놓은 단순 예시에서는 정확히 같은 위치의 경계 픽셀이 4개다.

P_boundary=(4)/(8)=0.5,

R_boundary=(4)/(8)=0.5

Boundary F-measure는 두 값의 조화평균이다.

F

=/2P_boundaryR_boundary

P_boundary+R_boundary

=0.5

실제 DAVIS 평가는 경계 위치가 몇 픽셀 정도 어긋난 것을 허용하는 tolerance를 사용한다. 따라서 위의 정확 일치 계산과 실제 공식 F 값은 달라질 수 있다. 해상도에 따라 같은 2픽셀 오차의 의미가 다르기 때문에 tolerance 정의와 resize 방식도 고정해야 한다.

4.2 Boundary IoU

Boundary IoU는 정답과 예측의 경계 주변 띠 영역을 만든 뒤 그 영역의 IoU를 계산한다. 개념을 보여 주기 위해 경계 폭이 0인 집합으로 단순화하면 다음과 같다.

BIoU=(|B_G∩ B_P|)/(|B_G∪ B_P|)

=(4)/(8+8-4)=(4)/(12)≈0.333

실제 구현에서는 contour 주변에 폭 d인 영역을 만들며, d를 픽셀 수로 둘지 영상 대각선 비율로 둘지 명시해야 한다.

4.3 Boundary F와 Boundary IoU 차이

- Boundary F는 예측 경계와 GT 경계가 tolerance 안에서 대응되는지를 precision과 recall로 평가한다.
- Boundary IoU는 경계 띠 영역의 교집합과 합집합을 평가한다.
- 둘 다 경계를 보지만 계산법과 tolerance 처리 방식이 다르므로 값을 섞어 쓰면 안 된다.

5. DAVIS의 J, F, J&F와 mIoU, mBIoU

5.1 J: region similarity

DAVIS에서 J는 foreground mask의 Jaccard, 즉 IoU다.

앞의 예시에서는 다음과 같다.

J=0.5

5.2 F: contour accuracy

F는 경계 precision과 boundary recall의 조화평균이다. 단순 tolerance 0 예시에서는 0.5다.

5.3 J&F

보통 두 값의 산술평균으로 보고한다.

J&F

=(J+F)/(2)

예시에서는 (0.5+0.5)/2=0.5다. 논문 표의 `J&F`는 논리 연산 AND가 아니라 두 지표의 평균이라는 점에 주의한다.

DAVIS 공식 평가는 객체와 프레임에 대한 평균뿐 아니라 recall과 temporal decay를 함께 보고하는 설정도 있다. 본 연구에서는 공식 코드가 출력하는 필드를 그대로 보존하고, 대표값만 임의로 재계산하지 않는다.

5.4 mIoU — mean IoU

다중 클래스 semantic segmentation에서는 클래스마다 IoU를 계산한 뒤 평균한다.

앞의 이진 예시에서 foreground IoU는 0.5다. 이번에는 background를 positive로 바꾸면 다음과 같다.

- background TP = 기존 TN = 13
- background FP = 기존 FN = 3
- background FN = 기존 FP = 3

IoU_bg=(13)/(13+3+3)=(13)/(19)≈0.684

foreground와 background를 모두 클래스 평균에 넣는다면 다음과 같다.

mIoU=(0.5+0.684)/(2)≈0.592

그러나 VOS의 J는 보통 지정 객체의 foreground IoU를 뜻한다. 따라서 `mIoU 59.2%`와 `J 50.0%`가 같은 예시에서도 나올 수 있다. 배경 포함 여부와 평균 단위를 반드시 적어야 한다.

5.5 mBIoU — mean Boundary IoU

각 클래스의 Boundary IoU를 계산한 뒤 평균한 값이다.

mBIoU=(1)/(C)Σ_c=1^CBIoU_c

예를 들어 두 대상 클래스의 BIoU가 0.333과 0.750이라면 다음과 같다.

mBIoU=(0.333+0.750)/(2)=0.5415

클래스가 하나뿐인 binary foreground 평가라면 mBIoU와 그 foreground BIoU가 같다. 배경도 독립 클래스로 포함하는 구현이라면 결과가 달라질 수 있다.

5.6 지표 선택 요약

- 객체 면적이 얼마나 정확히 겹치는가 — 우선 지표: IoU, J, Dice
- 얇은 구조와 윤곽이 정확한가 — 우선 지표: F, Boundary IoU
- 다중 **semantic class** 성능은 어떤가 — 우선 지표: mIoU, mBIoU
- 영상 전체에서 객체 분할 품질은 어떤가 — 우선 지표: J&F와 시퀀스별 분포
- 가림 후 다시 찾는가 — 우선 지표: recovery success, time-to-recovery, false reacquisition

6. Confidence, probability, uncertainty는 다르다

6.1 Confidence — 모델이 내놓은 자신감 점수

Confidence는 모델이 “이 결과가 좋다”고 보는 정도를 나타내는 숫자다. softmax 최대값, sigmoid 값, object score, predicted IoU 등이 confidence로 쓰일 수 있다.

하지만 0.9라는 점수가 곧 “90% 확률로 맞는다”는 뜻은 아니다. 0.9를 출력한 사례 100개 중 실제로 60개만 맞을 수도 있다. 이 모델은 과신**, overconfidence** 상태다.

6.2 Probability — 반복했을 때의 빈도 의미를 갖는 값

잘 보정된 확률 0.8은 비슷한 조건의 사례를 많이 모았을 때 약 80%가 정의한 사건을 만족해야 한다. 여기서 사건을 정확히 정해야 한다.

- 픽셀 사건: 이 픽셀이 foreground인가?
- mask 사건: 이 mask의 J가 0.7 이상인가?
- 미래 실패 사건: 앞으로 10프레임 안에 J<0.3이 되는가?
- 메모리 사건: 이 frame을 memory에 기록해도 이후 추적을 오염시키지 않는가?

서로 다른 사건의 확률을 같은 `confidence` 열에 섞으면 해석할 수 없다.

6.3 Uncertainty — 무엇을 모르는가

Uncertainty는 예측이 불확실한 정도다. 단순히 1-confidence로 만들 수 있지만, 항상 충분하지는 않다.

- **Aleatoric uncertainty**: blur, occlusion, 작은 객체처럼 입력 자체가 애매해서 생긴다.
- **Epistemic uncertainty**: 학습 데이터에 없던 장면이나 도메인이라 모델 지식이 부족해서 생긴다.
- **Temporal uncertainty**: 현재 frame 하나는 그럴듯하지만 과거 identity와 연결이 불안정한 경우다.

SAM 2 연구에서는 한 숫자보다 여러 신호가 유용하다.

- mask predicted IoU 또는 quality score
- object presence score
- object pointer의 프레임 간 변화
- mask 면적·중심·모양 변화
- 초기 대상 anchor와 현재 crop의 feature similarity
- motion model residual
- forward–backward cycle inconsistency

6.4 Predicted IoU는 실제 IoU가 아니다

SAM 계열의 predicted IoU는 mask 품질을 추정하도록 학습된 출력이다. 현재 frame의 GT를 보지 않고 "실제 IoU가 어느 정도일 것"이라고 추정한다.

- 실제 IoU: GT가 있어야 계산할 수 있는 평가값
- predicted IoU: 추론 중 모델이 내놓는 품질 추정값

predicted IoU 0.85가 실제 IoU 0.85를 보장하지 않는다. 특히 긴 가림, 비슷한 distractor, domain shift에서는 높은 점수를 유지한 채 틀린 객체를 따라갈 수 있다. 따라서 predicted IoU 자체를 baseline failure score로 삼되, 보정 전후를 평가해야 한다.

7. Calibration curve와 reliability diagram

7.1 무엇을 그리는가

Reliability diagram은 confidence를 여러 구간 bin으로 나누고 다음 두 값을 비교한다.

- x축: bin 안의 평균 예측 확률
- y축: bin 안의 실제 성공 비율

완벽히 보정되었다면 점들이 y=x 대각선에 가깝다.

- 점이 대각선 아래: 예측 확률보다 실제 성공률이 낮다 → 과신
- 점이 대각선 위: 실제 성공률이 더 높다 → 과소신

7.2 숫자 예시

mask가 좋다는 사건을 `실제 J ≥ 0.7`로 정의하자. 8개 사례에 대한 예측 확률과 실제 정답은 다음과 같다.

- **1** — 예측 성공확률 p: 0.95 · 실제 성공 y: 1
- **2** — 예측 성공확률 p: 0.90 · 실제 성공 y: 0
- **3** — 예측 성공확률 p: 0.85 · 실제 성공 y: 1
- **4** — 예측 성공확률 p: 0.80 · 실제 성공 y: 1
- **5** — 예측 성공확률 p: 0.75 · 실제 성공 y: 0
- **6** — 예측 성공확률 p: 0.70 · 실제 성공 y: 1
- **7** — 예측 성공확률 p: 0.65 · 실제 성공 y: 0
- **8** — 예측 성공확률 p: 0.60 · 실제 성공 y: 0

2개씩 bin을 묶으면 다음과 같다.

- **0.60–0.69** — 평균 confidence: 0.625 · 실제 accuracy: 0.000 · 절대 차이: 0.625
- **0.70–0.79** — 평균 confidence: 0.725 · 실제 accuracy: 0.500 · 절대 차이: 0.225
- **0.80–0.89** — 평균 confidence: 0.825 · 실제 accuracy: 1.000 · 절대 차이: 0.175
- **0.90–1.00** — 평균 confidence: 0.925 · 실제 accuracy: 0.500 · 절대 차이: 0.425

표본이 8개뿐이라 diagram이 거칠다. 실제 연구에서는 bin 수를 고정해 보고하고, equal-width와 equal-frequency binning 중 무엇을 썼는지 적는다. Adaptive ECE나 calibration plot도 함께 보면 bin 선택 의존성을 줄일 수 있다.

7.3 영상에서의 주의점

인접 프레임은 서로 매우 비슷하므로 1,000프레임이 1,000개의 독립 표본은 아니다. reliability diagram은 frame-level로 그릴 수 있지만, confidence interval과 calibration split은 sequence 단위로 처리해야 한다. 한 긴 영상이 결과를 지배하는지도 함께 점검한다.

8. ECE, Brier score, NLL

Calibration은 하나의 지표만으로 판단하지 않는다. ECE는 직관적이지만 bin에 의존하고, Brier와 NLL은 개별 확률을 직접 평가한다.

8.1 ECE — Expected Calibration Error

ECE=Σ_b=1^B(n_b)/(N)

|acc(b)-conf(b)|

앞 예시에서는 모든 bin 크기가 2로 같다.

ECE=(0.625+0.225+0.175+0.425)/(4)=0.3625

0에 가까울수록 좋다. 그러나 모든 예측을 단순히 0.5 근처로 모으면 ECE가 좋아 보이면서 discrimination이 나쁠 수 있다. AUROC·AUPRC와 함께 봐야 한다.

8.2 Brier score

이진 사건에서 다음 평균 제곱오차를 계산한다.

Brier=(1)/(N)Σ_i=1^N(p_i-y_i)^2

앞의 8개 예시에서는 제곱오차 합이 2.31이므로 다음과 같다.

Brier=2.31/8=0.28875

0에 가까울수록 좋다. Brier는 calibration과 discrimination이 모두 반영된 proper scoring rule이다.

8.3 NLL — Negative Log-Likelihood

NLL=-(1)/(N)Σ_i

[y_ilog p_i+(1-y_i)log(1-p_i)]

앞 예시의 NLL은 약 0.806이다. 틀린 사례에 0.99처럼 극단적으로 자신하면 매우 큰 벌점을 받는다. 계산에서는 p=0이나 1 때문에 로그가 무한대가 되지 않도록 작은 ε으로 clipping한다.

8.4 무엇을 함께 보고할까

권장 조합은 다음과 같다.

- reliability diagram
- ECE
- Brier score 또는 NLL
- AUROC와 AUPRC
- 선택 시스템이라면 risk–coverage curve

9. AUROC와 AUPRC: 순위를 얼마나 잘 매기는가

Calibration은 숫자의 절대 의미를 보고, AUROC·AUPRC는 위험한 사례가 안전한 사례보다 높은 점수를 받는지 본다.

9.1 작은 순위 예시

미래 실패를 positive로 놓고 5개의 위험 점수가 다음과 같다고 하자.

- **0.90** — 실제 실패: 1
- **0.80** — 실제 실패: 0
- **0.70** — 실제 실패: 1
- **0.60** — 실제 실패: 0
- **0.10** — 실제 실패: 0

positive 2개와 negative 3개의 모든 쌍은 6개다. 위험 점수 0.90인 positive는 세 negative보다 모두 높다. 0.70인 positive는 0.60과 0.10보다 높지만 0.80보다 낮다. 따라서 6쌍 중 5쌍의 순서가 맞다.

AUROC=5/6≈0.833

9.2 AUPRC가 희귀 실패에 중요한 이유

실패가 전체의 2%뿐이면 모든 frame을 정상이라고 해도 accuracy가 98%다. AUROC도 많은 easy negative 때문에 낙관적으로 보일 수 있다. Precision–recall curve는 높은 위험으로 선택한 사례 중 진짜 실패가 얼마나 되는지 직접 보여 준다.

AUPRC의 무작위 기준선은 대략 positive prevalence다. 실패율이 2%라면 기준선도 약 0.02다. 반드시 실패 prevalence와 함께 보고한다.

`AUPRC`라는 이름 아래 trapezoidal PR-AUC와 average precision이 혼용되므로 구현을 명시한다. scikit-learn의 `average_precision_score`를 썼다면 그대로 적는다.

9.3 높은 AUROC가 확률을 보장하지 않는다

모든 위험 점수에 10을 곱해도 순서는 같아 AUROC는 변하지 않는다. 그러나 0~1 확률 해석은 완전히 깨진다. 그래서 다음 두 질문을 분리한다.

1. 위험한 frame을 먼저 찾는가? → AUROC, AUPRC
2. 위험도 0.1이 실제 10% 실패를 뜻하는가? → ECE, Brier, NLL, reliability diagram

10. 사후 보정 방법

모델과 위험 점수 계산기를 학습한 뒤, 별도의 calibration set에서 score를 확률로 변환한다. Calibration set을 test set과 공유하면 안 된다.

10.1 Temperature scaling

logit z를 온도 T>0로 나눈 뒤 sigmoid 또는 softmax를 적용한다.

$p=\sigma(z/T)$

- T>1: 확률을 0.5 쪽으로 부드럽게 만들어 과신을 줄인다.
- T<1: 확률을 더 극단적으로 만든다.
- 순위를 바꾸지 않으므로 AUROC는 일반적으로 그대로다.
- parameter가 적어 과적합 위험이 낮지만 복잡한 calibration 오류는 고치지 못한다.

10.2 Logistic/Platt scaling

원래 score s에 대해 다음을 적합한다.

$p=\sigma(as+b)$

기울기와 절편을 함께 학습하므로 temperature scaling보다 유연하다. 위험 feature 여러 개를 logistic regression에 넣는 것은 단순 score 보정을 넘어 작은 failure predictor를 학습하는 것으로 구분해 써야 한다.

10.3 Isotonic regression

score가 커질수록 확률이 감소하지 않는다는 단조성만 가정하고 계단형 함수를 학습한다.

- 장점: 비선형 calibration 오류를 고칠 수 있다.
- 단점: calibration 표본이 적으면 계단이 거칠고 과적합하기 쉽다.
- sequence 수가 적은 연구에서는 bootstrap으로 불안정성을 확인한다.

10.4 보정법 비교 원칙

- 동일한 raw score와 동일한 calibration sequences를 사용한다.
- 보정 방법과 threshold를 calibration set에서만 선택한다.
- test에서는 ECE, Brier, NLL, AUROC/AUPRC를 한 번 평가한다.
- 보정 전후 AUROC가 같아도 calibration 개선은 의미가 있다.
- domain shift가 크면 source에서 잘 보정된 확률이 target domain에서 다시 틀어질 수 있다. dataset별 reliability diagram을 따로 그린다.

11. Conformal risk control

11.1 직관

일반 calibration은 “0.1이라고 한 사건이 약 10% 발생하는가”를 본다. Conformal risk control은 한 단계 더 나아가 허용 위험 이하가 되도록 행동 **threshold**를 **calibration data**로 선택하는 방법이다.

예를 들어 연구자가 다음 위험 예산을 정했다고 하자.

> memory에 기록하도록 허용한 frame 중, 실제로 나쁜 frame의 비율을 평균 5% 이하로 제한하고 싶다.

각 sequence i와 threshold λ에 대해 bounded loss $L_i(\lambda) \in [0,1]$를 정의한다. 예를 들면 선택된 memory write 중 이후 실패와 연결된 비율이다. Calibration sequences에서 여러 threshold를 시험하고, 위험의 one-sided upper confidence bound가 목표 α=0.05 이하인 범위에서 coverage가 가장 큰 threshold를 선택한다.

개념적으로 다음 조건을 만족하는 λ를 찾는다.

$$U_\delta\left(\frac{1}{n}\sum_{i=1}^{n} L_i(\lambda)\right) \le \alpha$$

여기서 U_δ는 신뢰수준 1-δ의 상한이다.

11.2 보장 범위

Conformal 방법의 보장은 calibration과 test 사례가 exchangeable하다는 가정에 의존한다. 쉽게 말해 순서를 바꿔도 같은 분포에서 나온 사례처럼 취급할 수 있어야 한다.

다음은 자동으로 보장되지 않는다.

- 전혀 다른 카메라·날씨·도메인
- 모든 개별 sequence에서 각각 5% 이하

- 매우 작은 subgroup의 성능
- frame을 독립 표본으로 잘못 간주한 경우

따라서 본 연구에서는 frame이 아니라 sequence를 calibration 단위로 삼고, cross-dataset 성능은 별도로 보고한다. Conformal이라는 이름만 붙였다고 arbitrary domain shift까지 안전해지는 것은 아니다.

11.3 초보자용 구현 순서

1. 실패 사건과 loss를 먼저 정의한다.
2. train sequence로 raw failure score를 만든다.
3. 별도 calibration sequence에서 threshold 후보마다 sequence-level loss를 계산한다.
4. 목표 risk budget과 신뢰수준을 정한다.
5. 조건을 만족하는 threshold 중 coverage가 가장 큰 것을 고른다.
6. threshold를 고정한 채 test와 cross-dataset test를 수행한다.

12. Abstention, selective prediction, risk budget

12.1 Abstention — 모르면 강제로 답하지 않기

일반 모델은 모든 frame에서 mask를 출력한다. Selective system은 위험할 때 다음 행동을 할 수 있다.

- mask를 확정하지 않고 `SUSPECT`로 표시
- memory write를 생략
- 빈 mask 또는 `target absent` 출력
- 추가 detector·retriever를 호출
- 사용자 correction을 요청

이것이 abstention이다. 영상 추적에서는 틀린 mask를 memory에 넣는 것보다 잠시 보류하는 편이 장기 성능에 유리할 수 있다.

12.2 Coverage와 selective risk

Coverage=(자동으로 수용한 사례 수)/(전체 사례 수)

Selective Risk

=(수용한 사례 중 실패 수)/(수용한 사례 수)

threshold를 엄격하게 하면 selective risk는 낮아지지만 coverage도 낮아져 detector 호출이나 사용자 개입이 많아진다. 따라서 risk–coverage curve 전체를 보고한다.

12.3 Risk budget 예시

한 가지 budget만 정하는 것보다 실제 사용 비용을 나눠 적는 편이 좋다.

- 잘못된 memory write 비율 ≤ 5%
- false recovery ≤ 3%
- 추가 검색 호출 ≤ 20회/1,000 frames

- 사용자 correction ≤ 2회/1,000 frames
- 평균 처리속도 ≥ vanilla SAM 2의 70%

이 수치는 예시다. 소규모 pilot 뒤 test를 보기 전에 고정해야 한다. 결과를 본 뒤 달성 가능한 기준으로 바꾸면 성공 기준의 의미가 사라진다.

PART E — 공정한 실험, 통계, 데이터셋과 계산 비용

13. 좋은 연구 질문과 가설 만들기

13.1 연구 질문, RQ

연구 질문은 측정 가능한 형태여야 한다.

- **RQ1:** predicted IoU 하나보다 다중 신호로 만든 보정 위험도가 앞으로 10프레임 내 추적 실패를 더 잘 예측하는가?
- **RQ2:** 위험도가 높은 frame의 memory write를 막으면 장기 가림 이후의 비가역적 drift가 줄어드는가?
- **RQ3:** TRACK–SUSPECT–LOST 상태와 anchor recovery가 재등장 복구율을 높이면서 false reacquisition을 허용 범위 안에 유지하는가?
- **RQ4:** 개선 효과가 DAVIS뿐 아니라 MOSE, LVOS, RobustPVOS에서도 유지되는가?
- **RQ5:** 정확도 향상에 필요한 latency·VRAM·추가 호출 비용은 얼마이며 Pareto-efficient한가?

13.2 가설, hypothesis

가설은 방향과 비교 대상을 포함한다.

- **H1:** 다중 신호 calibrated risk는 predicted IoU baseline보다 미래 실패 AUPRC가 높다.
- **H2:** selective memory write는 전체 frame의 memory write보다 LVOS의 sequence-level J&F와 recovery success를 높인다.
- **H3:** anchor recovery는 motion-only recovery보다 유사 distractor 상황의 false reacquisition을 줄인다.
- **H4:** 전체 시스템은 vanilla SAM 2 대비 LVOS와 MOSE에서 양의 paired improvement를 보이며, 평균 FPS는 baseline의 70% 이상을 유지한다.

"성능이 좋아질 것이다"만 쓰면 무엇을 측정할지 알 수 없다. 실패 horizon, dataset, baseline, metric을 포함한다.

14. Baseline과 공정한 비교

14.1 Baseline의 역할

Baseline은 새 방법이 없을 때의 기준이다. 최소한 다음을 포함한다.

- 공식 vanilla SAM 2 또는 SAM 2.1
- predicted IoU 또는 object score 하나로 trigger하는 단순 방법
- 고정 간격 every-N-frame 재프롬프트

- 가능하면 관련 선행연구의 공개 구현

복잡한 새 방법을 약하게 구현한 baseline과 비교하면 공정하지 않다. 공식 checkpoint와 공식 preprocessing을 먼저 재현한다.

14.2 Fair comparison 체크리스트

비교 방법 사이에서 다음을 같게 유지한다.

- SAM 2 모델 크기와 checkpoint
- 입력 해상도와 resize/interpolation
- 첫 prompt 형태: point, box, mask
- 같은 GT first-frame prompt
- propagation 방향과 처리 frame 수
- memory 최대 크기
- 외부 detector와 feature encoder 사용 여부
- threshold를 고른 validation data
- mixed precision과 batch size
- hardware, CUDA/PyTorch 환경
- metric 공식 코드와 absent-frame 처리

외부 Grounding DINO를 새 방법만 사용한다면, “SAM 2 내부 개선”이 아니라 “추가 모델을 포함한 시스템 개선”으로 써야 한다. parameter와 FLOPs도 전체 시스템 기준으로 보고한다.

15. Data split과 video leakage

15.1 프레임 무작위 분할이 왜 누수인가

한 영상의 100번째 frame을 train에, 101번째 frame을 test에 넣으면 두 장은 거의 같다. 배경, 대상 identity, 카메라가 모두 공유된다. 높은 test 성능은 일반화가 아니라 같은 영상을 기억한 결과일 수 있다.

따라서 split의 최소 단위는 frame이 아니라 원본 **video sequence**다.

15.2 자주 발생하는 video leakage

- 같은 원본 YouTube 영상에서 잘라낸 clip이 train과 test에 나뉨
- 같은 촬영 세션·카메라·장소의 연속 clip이 양쪽에 존재
- 한 sequence의 augmentation이 다른 split에 들어감
- 같은 객체 identity가 거의 동일한 배경에서 반복 등장
- calibration threshold를 test sequence를 보며 조절
- test GT로 실패 시점을 정의한 뒤, 그 정보를 online trigger 입력에 실수로 사용
- SAM 2가 학습한 SA-V를 독립 zero-shot test라고 주장

가능하면 source video ID, 촬영 세션, near-duplicate perceptual hash를 이용해 누수를 검사한다.

15.3 권장 분할

1. **Train:** failure predictor 또는 추가 모듈 학습
2. **Validation:** hyperparameter와 모델 선택
3. **Calibration:** 확률 보정 및 conformal threshold 결정
4. **Test:** 마지막 한 번의 성능 평가
5. **Cross-dataset test:** domain shift 평가

데이터가 작다면 validation 안에서 calibration subset을 sequence 단위로 분리한다. Test를 calibration에 사용해서는 안 된다.

16. Domain shift와 cross-dataset 평가

16.1 Domain shift 예시

- 맑은 야외 → 안개·야간·눈
- 짧고 중심 객체가 뚜렷한 DAVIS → crowded MOSE
- 수 초 영상 → 수 분 LVOS
- 일반 객체 → camouflage 또는 작은 대상
- 영상 mask benchmark → box 중심 tracking benchmark

source dataset에서 calibration이 좋아도 target dataset에서 과신할 수 있다. 전체 ECE 하나만 보고하지 말고 dataset별, 조건별 reliability diagram을 그린다.

16.2 Cross-dataset protocol

예를 들어 MOSE train으로 failure predictor를 학습하고 MOSE의 held-out sequence에서 calibrate한 뒤 다음을 수행한다.

- in-domain: MOSE test 또는 frozen validation split
- short clean shift: DAVIS 2017
- long-term shift: LVOS
- corruption shift: RobustPVOS
- distractor stress: DiDi

Target test에 맞춰 threshold를 다시 조절하면 strict cross-dataset 결과가 아니다. 재보정을 허용한 결과는 `target-calibrated`로 별도 표기한다.

17. Ablation과 controlled variable

17.1 Ablation study

전체 방법에서 구성요소를 하나씩 제거하거나 추가해 어느 부분이 효과를 냈는지 확인한다.

권장 순서는 다음과 같다.

1. E0: vanilla SAM 2
2. E1: predicted IoU threshold trigger
3. E2: 다중 신호 raw failure score

4. E3: E2 + calibration
5. E4: E3 + selective memory write
6. E5: E4 + TRACK/SUSPECT/LOST
7. E6: E5 + positive anchor recovery
8. E7: E6 + negative distractor memory

17.2 Controlled variable

한 번에 하나의 원인만 비교한다.

- **calibration** 효과 — 바꿀 것: raw vs calibrated risk · 고정할 것: feature, classifier, threshold 선택 규칙 외 전부
- **memory gate** 효과 — 바꿀 것: write gate on/off · 고정할 것: 같은 mask prediction과 recovery
- **anchor** 효과 — 바꿀 것: anchor scoring on/off · 고정할 것: 같은 candidate proposals
- **motion vs appearance** — 바꿀 것: candidate score 항 · 고정할 것: candidate 수, 호출 시점, SAM 2
- **event trigger** 효과 — 바꿀 것: fixed interval vs risk trigger · 고정할 것: 총 허용 호출 수 또는 계산 budget

호출 횟수가 다른 두 시스템을 그대로 비교하면 정확도 향상이 단순히 계산을 더 많이 쓴 결과일 수 있다. 같은 호출 budget 비교와 unconstrained 비교를 둘 다 제시하면 좋다.

18. Hyperparameter와 재현성

18.1 주요 hyperparameter

- 미래 실패 horizon H
- 실패 IoU threshold τ_fail
- TRACK→SUSPECT, SUSPECT→LOST threshold
- hysteresis 연속 frame 수
- memory write risk threshold
- anchor similarity와 distractor margin
- LOST 상태 검색 간격
- candidate 수와 NMS threshold
- calibration bin 수와 방법

18.2 선택 절차

- coarse grid를 validation에서 비교한다.
- 가장 좋은 값 하나만 남기지 말고 sensitivity plot을 제공한다.
- 모든 dataset에 같은 threshold를 적용하는 결과와 dataset-specific threshold를 구분한다.
- test 성능을 본 뒤 값을 수정하지 않는다.

18.3 재현성 기록

- Git commit hash

- SAM 2 checkpoint 파일명과 checksum
- Python, PyTorch, CUDA, driver 버전
- 모든 package lock 또는 environment 파일
- random seed와 deterministic 설정
- 데이터 다운로드 날짜·버전·split 목록
- 실행 command와 config
- 전처리 해상도·색공간·interpolation
- metric code version
- raw per-frame, per-object, per-sequence 결과
- 실패 로그와 예외 처리 규칙

평균 점수 표만 저장하면 나중에 통계 검정을 다시 할 수 없다. 최소한 sequence별 결과를 CSV/JSON으로 남긴다.

19. Sequence-level paired bootstrap 95% CI

19.1 왜 frame bootstrap이 아닌가

영상의 인접 frame은 강하게 상관되어 있다. frame 10,000개를 독립 표본처럼 bootstrap하면 표본 수가 과장되어 CI가 지나치게 좁아진다.

독립 단위에 더 가까운 sequence를 cluster로 사용한다. 한 sequence 안의 여러 객체와 frame은 함께 움직인다.

19.2 Paired bootstrap 절차

baseline A와 새 방법 B를 같은 N개 sequence에서 평가했다고 하자.

1. 각 sequence에서 두 방법의 metric을 계산한다.
2. sequence별 차이 $d_i = M_{B,i} - M_{A,i}$를 만든다.
3. N개 sequence ID를 복원추출로 다시 뽑는다.
4. 선택된 sequence들의 평균 차이를 계산한다.
5. 이 과정을 10,000번 반복한다.
6. bootstrap 차이 분포의 2.5 percentile과 97.5 percentile을 95% CI로 사용한다.

예를 들어 평균 향상이 +2.4 J&F point이고 95% CI가 [+0.8, +4.0]이면 대부분의 bootstrap 표본에서 양수다. 반면 [+2.4]라도 CI가 [-0.7, +5.1]이면 평균 향상은 관찰됐지만 불확실성이 커서 일관된 개선이라고 강하게 결론내리기 어렵다.

19.3 Pairing의 의미

두 방법은 반드시 같은 sequence 쌍으로 비교한다. 어려운 영상과 쉬운 영상 구성이 동일하므로 between-sequence 변동을 제거할 수 있다.

한 sequence에 객체가 여러 개라면 두 가지 estimand가 가능하다.

- sequence를 동등 가중: sequence별 객체 평균을 만든 뒤 bootstrap
- 객체를 동등 가중하되 sequence cluster를 함께 resample

어느 것을 주요 분석으로 쓸지 사전에 정하고 공식 benchmark aggregation과의 차이를 설명한다.

20. Wilcoxon signed-rank test

20.1 무엇을 검정하는가

같은 sequence에서 얻은 차이 d_i를 사용한다. 차이의 절댓값에 순위를 매기고 부호를 반영해, 차이 분포가 0을 중심으로 하는지 검정한다.

- 귀무가설: 두 방법의 paired difference가 0을 중심으로 한다.
- 대립가설: 차이가 0이 아니다. 방향 가설을 미리 정했다면 one-sided도 가능하다.

20.2 주의점

- frame별 점수에 Wilcoxon을 적용하지 않는다.
- 0 차이는 통상 제외되며 사용한 구현을 적는다.
- 차이 분포가 매우 비대칭이면 sign test도 보조적으로 고려한다.
- p-value는 효과 크기가 아니다. 평균·중앙값 차이와 95% CI를 함께 보고한다.
- 여러 dataset, metric, ablation에 검정을 반복하면 Holm 등의 multiple-comparison correction을 고려한다.
- “p>0.05이므로 두 방법이 같다”라고 결론내리면 안 된다. 증거가 부족한 것과 동등함은 다르다.

권장 보고 예시는 다음과 같다.

> LVOS 50개 validation sequence에서 새 방법은 SAM 2.1 대비 sequence-level J&F를 평균 2.4 point 향상시켰다. Paired bootstrap 95% CI는 [0.8, 4.0]이었으며, 양측 Wilcoxon signed-rank test의 Holm-adjusted p는 0.012였다.

21. 희귀 실패와 event-based recovery 평가

평균 J&F는 정상 frame이 많으면 가림·재등장 실패를 숨긴다. 연구 주제가 복구라면 event를 별도로 정의한다.

21.1 Failure event 정의 예시

대상이 보이는 frame에서는 다음 중 하나를 failure로 정할 수 있다.

- J_t<0.3이 2프레임 이상 지속
- target 대신 distractor와 더 크게 겹치는 ID switch
- mask area가 GT target과 무관하게 급증하며 IoU가 붕괴

대상이 GT상 완전히 보이지 않는 frame에서는 빈 mask가 정답일 수 있다. 따라서 `target absent인데 mask가 없음`을 실패로 세면 안 된다. Presence annotation 또는 visibility rule을 명시한다.

21.2 미래 실패 label

현재 시점 t의 위험 모델을 학습하려면 미래 horizon H를 정한다.

$y_t^{(H)}$=

1[frame t+1… t+H 안에 failure 발생]

예를 들어 H=10이면 현재까지 관찰한 정보로 향후 10프레임의 실패를 예측한다. 입력 feature에 미래 frame이나 GT를 넣으면 leakage다.

21.3 Detection metrics

- event precision, recall, F1
- frame-level AUROC와 AUPRC
- false alarms per 1,000 frames
- event당 중복 trigger 수
- detection lead time: 실패 몇 frame 전에 경고했는가
- missed-event rate

Frame-level AUPRC와 event-level recall을 둘 다 보고한다. 한 failure 주변에서 경고가 20번 발생해도 event 하나를 잡은 것으로만 세는 matching window가 필요하다.

21.4 Recovery metrics

- **Recovery success@H:** 재등장 후 H프레임 안에 IoU threshold를 회복한 event 비율
- **Time-to-recovery, TTR:** 재등장부터 처음 J≥0.5를 연속 k프레임 달성할 때까지의 frame 수
- **Post-recovery IoU:** 복구 후 일정 window의 평균 IoU
- **Recovery AUC:** 재등장 후 시간에 따른 IoU curve의 면적
- **False reacquisition:** target이 아닌 distractor를 target으로 확정한 비율
- **Relapse rate:** 복구 뒤 짧은 시간 안에 다시 실패한 비율
- **Never-recovered rate:** 영상 종료까지 복구하지 못한 event 비율

영상 종료 때문에 관찰 window가 부족한 event는 censored case가 될 수 있다. 단순히 제외하면 짧은 영상에 유리해지므로 별도 표시하거나 survival analysis를 고려한다.

21.5 통계 단위

Event는 같은 sequence 안에서 상관될 수 있다. CI를 구할 때 event만 독립 resampling하지 말고 sequence cluster를 resample한 뒤 그 안의 event를 함께 포함한다. Event 수와 이를 포함한 sequence 수도 함께 보고한다.

22. 효율성: FPS, latency, VRAM, FLOPs, parameter, Pareto

22.1 FPS

FPS=(처리한 frame 수)/(걸린 초)

다음 조건을 함께 적는다.

- GPU와 CPU 모델
- 입력 해상도
- batch size, 보통 streaming은 1

- FP32, FP16, BF16 여부
- warm-up frame 수
- disk I/O와 video decoding 포함 여부
- CUDA synchronization 여부
- normal path와 recovery path 호출 비율

22.2 Latency

평균 latency만으로 event-driven 시스템을 설명하기 어렵다.

- p50: 평소 frame의 체감 지연
- p95, p99: recovery가 호출된 느린 frame 포함
- normal-path latency
- trigger-path latency
- end-to-end latency: decoding과 후처리 포함

실시간 시스템에서는 평균 30 FPS여도 p99가 1초라면 사용이 불편할 수 있다.

22.3 VRAM

- peak allocated VRAM
- peak reserved VRAM
- sequence 길이에 따른 증가
- 객체 수에 따른 증가
- memory bank와 anchor 저장량

측정 직전 peak statistic을 reset하고 동일 sequence로 비교한다. 다른 process가 GPU를 쓰지 않는 상태를 기록한다.

22.4 FLOPs와 parameters

- total parameters와 trainable parameters를 구분한다.
- 외부 DINO, detector, optical-flow model도 전체 시스템 parameter에 포함한다.
- 동적 시스템은 normal FLOPs, triggered FLOPs, 실제 trigger rate를 이용한 expected FLOPs를 모두 보고한다.

개념적으로 평균 frame 비용은 다음과 같다.

C_avg=C_normal+r_triggerC_extra

22.5 Pareto frontier

방법 A가 방법 B보다 정확도도 높고 속도도 빠르면 B는 dominated다. 어느 한쪽만 좋은 방법들이 Pareto frontier를 이룬다.

다음 plot을 권장한다.

- x축: average latency 또는 expected FLOPs
- y축: J&F 또는 recovery success
- 점 크기: peak VRAM
- 점 색: false reacquisition 또는 selective risk

단일 “최고 모델”보다 accuracy–cost trade-off를 명확히 보여 준다.

23. 데이터셋 이해와 역할 분담

23.1 DAVIS 2017

DAVIS 2017은 고품질 dense mask가 있는 대표적인 다중 객체 VOS benchmark다. Semi-supervised, 더 정확히는 human-guided 설정에서는 첫 frame의 대상 mask가 제공된다. 공식 평가는 region J와 contour F를 사용한다.

장점

- 선명한 mask와 널리 쓰이는 공식 평가 코드
- 빠른 sanity check와 선행연구 비교에 좋음
- 경계 품질을 확인하기 좋음

한계

- 비교적 짧고 잘 촬영된 영상이 많다.
- 중심 대상이 뚜렷해 실제 crowded·장기 재등장 문제를 충분히 대표하지 못한다.
- DAVIS만 향상되면 장기 recovery 개선이라고 주장하기 어렵다.

DAVIS 공식 사이트

23.2 MOSE

MOSEv1은 2,149 clips, 5,200 objects, 431,725개 고품질 mask를 포함하며 crowded scene, heavy occlusion, 유사 객체, disappearance–reappearance를 의도적으로 많이 담았다.

적합한 질문

- distractor 때문에 target identity가 바뀌는가?
- 가림 중 memory가 오염되는가?
- 작은 객체와 복잡한 움직임에서 trigger가 작동하는가?

MOSEv2를 사용할 경우 버전을 명시한다. MOSEv2는 adverse weather, low light, multi-shot, camouflage, non-physical target과 knowledge-dependent 장면까지 확장한다. v1과 v2 결과를 같은 split처럼 섞지 않는다.

MOSE 공식 프로젝트, MOSEv1 ICCV 원문

23.3 LVOS

LVOS는 220개 장기 영상, 총 약 421분으로 구성되며 평균 영상 길이는 약 1.59분, 평균 574 frames다. Long-term reappearance는 적어도 100 frames 동안 사라졌다 다시 나타나는 속성으로 정의된다. 시간상 동시에 등장하지 않는 유사 객체로 인한 cross-temporal confusion도 포함한다.

적합한 질문

- 짧은 FIFO memory가 오래된 target identity를 잊는가?
- target이 화면 밖으로 나간 뒤 다시 찾는가?

- immutable anchor가 장기 drift를 줄이는가?
- recovery time이 얼마나 줄어드는가?

LVOS 공식 프로젝트, ICCV 원문

23.4 DiDi

DiDi는 기존 tracking benchmark에서 distractor가 뚜렷한 sequence를 반자동으로 추출한 distractor-distilled dataset이다. visually similar object가 있을 때 target을 유지하는 능력을 집중 평가한다.

중요한 한계

- 본래 visual object tracking 평가가 중심이며 bounding box와 VOT toolkit 지표를 사용한다.
- DiDi의 quality/accuracy/robustness를 VOS의 J&F와 동일하게 해석하면 안 된다.
- mask 품질 주장을 하려면 별도의 pixel annotation 또는 VOS 평가가 필요하다.

본 연구에서는 negative distractor memory와 false reacquisition stress test에 사용하는 것이 적절하다.

DAM4SAM·DiDi 공식 저장소

23.5 RobustPVOS

RobustPVOS는 adverse-condition promptable VOS를 위한 CVPR 2026 benchmark다. 두 실제 평가셋에 총 351 clips와 2,500개 이상의 object mask가 있다.

- ACDC-Video: 149 clips, 3,259 frames, 613 objects
- MVSeg 계열: 202 clips, 13,581 frames, 1,930 objects
- 조건: 안개, 비, 눈, 야간, 저조도, motion blur 등
- synthetic train/evaluation: MOSE-C, DAVIS-C, YouTube-VOS-C와 같이 시간에 따라 변하는 corruption을 적용

적합한 질문

- source에서 보정한 risk가 날씨·저조도 shift에서도 calibration을 유지하는가?
- blur에서 predicted IoU가 과신하는가?
- recovery 호출 증가가 latency tail을 얼마나 키우는가?

CVPR 원문, 공식 프로젝트

23.6 SA-V

Meta의 SA-V는 약 51K 영상과 643K class-agnostic masklet을 포함한다. 약 191K masklet은 SAM 2-assisted manual annotation이고, 약 452K는 SAM 2가 자동 생성한 뒤 annotator가 검증했다. 평균 영상당 12.61 masklet이며 class label은 제공하지 않는다.

장점

- 다양하고 대규모인 open-world video segmentation 자료
- failure predictor의 추가 학습 또는 self-supervised 분석에 활용 가능

주의점

- SAM 2 학습에 사용된 데이터이므로 SA-V 성능을 완전히 독립적인 zero-shot generalization으로 주장하면 안 된다.
- annotation의 상당 부분이 SAM 2 생성·인간 검증 방식이므로 모델 고유 bias가 반영될 수 있다.
- 규모가 매우 커 학기 연구에서는 curated subset과 명확한 sampling rule이 필요하다.

Meta SA-V 데이터 카드, SAM 2 ICLR 원문

23.7 데이터셋 역할 요약

- **DAVIS 2017** — 주 역할: 기본 mask·경계 성능 · 강한 실패 유형: 변형, 일부 가림, 다중 객체 · 주의점: 짧고 비교적 깨끗함
- **MOSE** — 주 역할: 복잡 장면 · 강한 실패 유형: crowd, occlusion, distractor, 재등장 · 주의점: v1/v2 구분
- **LVOS** — 주 역할: 장기 평가 · 강한 실패 유형: 긴 부재, out-of-view, cross-temporal confusion · 주의점: 긴 sequence 비용
- **DiDi** — 주 역할: distractor stress · 강한 실패 유형: 유사 객체, identity drift · 주의점: tracking 지표 중심
- **RobustPVOS** — 주 역할: domain/corruption shift · 강한 실패 유형: 날씨, 저조도, blur · 주의점: calibration shift를 별도 분석
- **SA-V** — 주 역할: 대규모 학습·분석 · 강한 실패 유형: open-world 다양성 · 주의점: SAM 2 train data와 순환성

PART F — 제안 연구를 처음부터 다시 조립하기

24. 제안 연구**:** 미래 실패위험 보정 기반 **SAM 2**

24.1 핵심 문제

SAM 2는 현재 mask를 memory에 저장하고 다음 frame 예측에 사용한다. 잘못된 mask가 memory에 들어가면 다음 예측도 틀리고, 그 결과가 다시 memory에 들어가는 self-reinforcing drift가 생길 수 있다.

따라서 현재 mask가 얼마나 좋아 보이는지만 판단하지 않고 다음을 물어야 한다.

> **“**현재 상태를 그대로 믿고 **memory**에 기록하면 앞으로 몇 프레임 안에 추적이 무너질 위험은 얼마인가**?”**

24.2 예측 대상

미래 horizon을 H=10 frames로 둔 예를 생각하자.

r_t=P(y_t^(H)=1| x_1:t,SAM2 outputs_1:t)

여기서 r_t는 미래 failure probability이고 추론 시점에는 현재와 과거 정보만 사용한다.

Failure label은 visibility를 고려해 정의한다.

- target visible: 향후 H 안에 J<τ_fail이 연속 k frames 발생
- target absent: 큰 false-positive mask를 출력하거나 distractor를 target으로 확정
- identity annotation 가능: 다른 객체와의 overlap이 target보다 커지는 ID switch

24.3 입력 feature

GT 없이 추론 중 얻을 수 있는 값만 사용한다.

- **SAM 2** 내부 — feature 예시: predicted IoU, object score, object pointer distance · 의미: 모델 자체 품질·존재 신호
- **mask geometry** — feature 예시: area ratio, centroid velocity, aspect ratio, fragmentation · 의미: 갑작스러운 모양 변화
- **appearance** — feature 예시: DINOv2 anchor similarity, positive–negative margin · 의미: 대상 identity 유지 여부
- **motion** — feature 예시: Kalman residual, optical-flow consistency · 의미: 위치 예측과 관측 불일치
- **temporal** — feature 예시: forward–backward cycle error, mask overlap history · 의미: 시간 일관성
- **context** — feature 예시: blur, brightness, corruption estimate · 의미: 위험 장면 여부

처음에는 logistic regression이나 작은 MLP로 시작한다. 큰 모델보다 feature와 evaluation protocol의 타당성을 먼저 입증한다.

24.4 보정

1. Train sequences로 failure predictor를 학습한다.
2. 별도 calibration sequences에서 temperature, logistic, isotonic을 비교한다.
3. ECE, Brier, NLL로 최종 calibrator를 선택한다.
4. Memory write와 LOST transition threshold는 validation/calibration에서 고정한다.
5. Test와 cross-dataset에서는 수정하지 않는다.

Predicted IoU 단독 score를 반드시 baseline으로 둔다. 그래야 다중 신호와 calibration의 실제 기여를 알 수 있다.

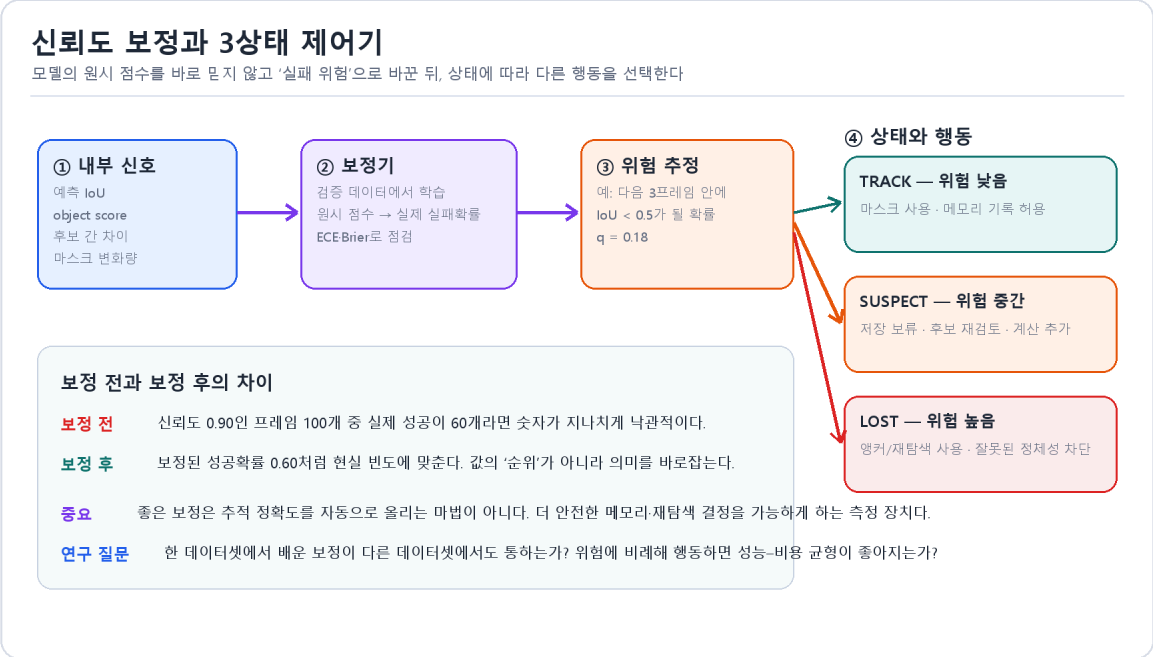


그림 *4.* 내부 *score*를 바로 *threshold*하지 않고 실제 미래 실패위험으로 *calibration*한 뒤 *TRACK·SUSPECT·LOST* 상태별 행동으로 연결하는 구조다.

25. TRACK–SUSPECT–LOST 상태기계

25.1 TRACK

대상을 정상적으로 추적한다고 판단한 상태다.

- mask를 정상 출력
- risk가 충분히 낮을 때만 short-term memory write
- 신뢰도 높은 일부 frame만 long-term anchor 후보로 승격
- 외부 detector/retriever는 호출하지 않음

25.2 SUSPECT

오류 가능성은 있지만 즉시 LOST로 확정하기 어려운 상태다.

- memory write를 일시 중지
- 현재 mask를 provisional output으로 표시
- motion box, appearance proposal, negative point 등 소수 후보 생성
- 후보 간 consistency 확인
- 위험이 낮아지면 TRACK으로 복귀
- 모든 후보가 약하거나 위험이 더 높아지면 LOST로 전환

SUSPECT 상태가 없으면 점수 한 번의 흔들림만으로 비싼 재탐색을 호출하거나 target을 잘못 초기화할 수 있다.

25.3 LOST

현재 target identity를 신뢰할 수 없거나 target이 화면에 없다고 보는 상태다.

- 잘못된 mask를 강제로 memory에 기록하지 않음
- 필요하면 empty mask와 abstention 상태를 출력
- 일정 간격 또는 target-presence 신호가 생길 때 candidate search
- immutable anchor와 negative distractor memory로 후보 검증
- 여러 조건을 연속 m frames 통과할 때만 TRACK으로 복귀

25.4 Hysteresis

한 threshold만 사용하면 TRACK과 SUSPECT가 매 frame 진동할 수 있다. 진입·복귀 threshold를 다르게 둔다.

- TRACK→SUSPECT: $r_t \geq \tau_s$가 2 frames 지속
- SUSPECT→TRACK: $r_t \leq \tau_{back} < \tau_s$이고 anchor 검증 통과
- SUSPECT→LOST: $r_t \geq \tau_l$ 또는 후보 검증 연속 실패
- LOST→TRACK: recovery score가 $\tau_{recover}$ 이상으로 2 frames 지속

이 숫자는 예시이며 validation에서 정한다.

26. Selective memory write

26.1 왜 필요한가

최근 frame을 무조건 memory에 넣으면 최신 appearance를 잘 따라갈 수 있지만, 가림·distractor frame도 함께 저장된다. 반대로 너무 적게 저장하면 appearance change에 적응하지 못한다.

26.2 Write gate 예시

다음 조건을 모두 만족할 때만 현재 frame을 memory에 쓴다.

$$WRITE_t = \mathbb{1}[S_t = TRACK,\ r_t \leq \tau_w,\ s^{anchor}_t \geq \theta_a,\ m^{distractor}_t \geq \theta_m]$$

- S_t: 현재 상태
- r_t: calibrated future failure risk
- s^{anchor}_t: positive anchor similarity
- $m^{distractor}_t$: target score와 가장 유사한 distractor score의 margin

26.3 메모리 종류

- **Short-term memory:** 최근 appearance 적응용. FIFO로 교체 가능.
- **Long-term positive anchors:** 최초 prompt frame과 매우 신뢰도 높은 corrected frame. 쉽게 삭제하지 않음.
- **Negative distractor memory:** 잘못 선택되기 쉬운 유사 객체 특징. 후보 감점에 사용.

첫 prompt anchor는 immutable하게 유지한다. 다만 대상이 장기간 변형되면 최초 모습만으로 부족하므로, 검증된 새로운 anchor를 소수 추가한다.

26.4 무엇을 측정할까

- 전체 memory write 수
- 나중에 실제 저품질로 판명된 contaminated write 비율
- contaminated write 후 첫 failure까지의 시간
- gate로 막은 frame 중 사실 안전했던 비율
- coverage와 selective risk

27. Anchor recovery

27.1 Candidate 생성

SUSPECT 또는 LOST에서 다음 후보를 만든다.

- motion prediction box
- DINO similarity heatmap의 높은 영역 box
- SAM 2 automatic mask proposals
- 선택적으로 open-vocabulary detector의 text box
- 이전 mask 주변의 positive/negative point 조합

모든 frame에서 모든 후보를 만들지 않고 event-trigger 방식으로 호출한다.

27.2 Candidate 점수

예시 점수는 다음과 같다.

$$S(c) = w_a \, sim_{pos}(c) - w_n \, sim_{neg}(c) + w_m \, motion(c) + w_q \, quality(c) - w_r \, risk(c)$$

- positive anchor와 닮을수록 가점
- known distractor와 닮을수록 감점
- 가능한 motion 범위와 맞으면 가점
- SAM predicted quality가 높으면 가점
- 미래 failure risk가 높으면 감점

최고 후보가 threshold를 넘지 못하면 억지로 선택하지 않고 LOST를 유지한다.

27.3 Recovery 확인

한 frame에서만 좋은 후보는 우연일 수 있다. 다음 중 일부를 요구한다.

- 2개 연속 frame에서 높은 anchor similarity
- forward로 추적한 뒤 backward 했을 때 원래 후보와 일치
- negative distractor margin 확보
- 예상 object size와 큰 모순이 없음
- calibrated recovery confidence가 threshold 이상

검증을 통과하면 box 또는 mask prompt로 SAM 2를 재초기화하고 TRACK으로 돌아간다.

28. 전체 알고리즘 의사코드

```
입력: video frames, initial prompt, SAM 2
초기화:
 state = TRACK
 immutable positive anchor 저장
 short-term memory 초기화
 negative distractor memory = empty
각 frame t에 대해:
 SAM 2 mask와 내부 quality 신호 계산
 geometry, appearance, motion, cycle feature 계산
 calibrated future failure risk r_t 계산
 if state == TRACK:
 if r_t가 낮고 anchor 검증 통과:
 mask 출력
 조건부로 memory write
 else if suspect 조건 지속:
 state = SUSPECT
 memory write 중지
 if state == SUSPECT:
 소수 candidate prompt 생성
 candidate를 anchor/distractor/motion/quality로 평가
 if 안전한 후보가 연속 검증 통과:
 SAM 2 재프롬프트
 state = TRACK
 else if lost 조건 충족:
 state = LOST
 else:
 provisional mask 또는 abstention 출력
 if state == LOST:
 memory write 금지
 empty/abstain 출력
 검색 간격이 되었으면 candidate search
 if anchor recovery 검증 통과:
 SAM 2 재초기화
 state = TRACK
```

29. 권장 실험 계획

29.1 단계별 개발

단계 ***A —*** 평가 파이프라인 검증

- vanilla SAM 2.1을 DAVIS와 LVOS 일부에서 재현
- per-frame J, F, object presence, predicted IoU 저장
- 공식 점수와 직접 집계 점수가 일치하는지 확인

단계 ***B — failure predictor***

- future failure label 생성
- predicted IoU 단독 baseline
- geometry, appearance, temporal feature를 순서대로 추가
- AUROC/AUPRC와 lead time 평가

단계 ***C — calibration***과 ***selective write***

- temperature/logistic/isotonic 비교
- reliability, ECE, Brier, NLL 평가
- risk–coverage curve와 memory contamination 측정

단계 ***D —*** 상태기계와 ***anchor recovery***

- TRACK/SUSPECT/LOST 전환
- motion-only, positive-anchor, positive+negative anchor 비교
- event-based recovery metric 추가

단계 ***E — cross-dataset·***효율성

- DAVIS, MOSE, LVOS, RobustPVOS, DiDi 역할별 평가
- FPS, p95 latency, peak VRAM, expected FLOPs
- sequence paired bootstrap와 Wilcoxon

29.2 Main result 표 양식

- **SAM 2.1**
- **Fixed-N re-prompt**
- **Pred-IoU trigger**
- **Proposed full**

29.3 Failure prediction·calibration 표

- **Predicted IoU**
- **+ geometry**
- **+ appearance**
- **+ temporal**
- **+ calibration**

29.4 Ablation 표

- **E0**
- **E1** — Calibrated risk: ✓
- **E2** — Calibrated risk: ✓ · Selective write: ✓
- **E3** — Calibrated risk: ✓ · Selective write: ✓ · State machine: ✓
- **E4** — Calibrated risk: ✓ · Selective write: ✓ · State machine: ✓ · Positive anchor: ✓
- **E5** — Calibrated risk: ✓ · Selective write: ✓ · State machine: ✓ · Positive anchor: ✓ · Negative memory: ✓

29.5 통계 보고 표

- **DAVIS** — 95% paired bootstrap CI: [,]
- **MOSE** — 95% paired bootstrap CI: [,]
- **LVOS** — 95% paired bootstrap CI: [,]
- **RobustPVOS** — 95% paired bootstrap CI: [,]

30. 성공 기준 예시

성공 기준은 pilot을 마친 뒤 test 결과를 보기 전에 고정한다. 다음은 과제연구용 예시다.

30.1 Primary success criterion

- 주 데이터셋 LVOS에서 vanilla SAM 2.1 대비 sequence-level J&F가 최소 +2.0 point 향상
- paired bootstrap 95% CI의 하한이 0보다 큼

둘 중 하나만 만족했다고 표현하지 말고 primary 기준을 사전에 `AND` 또는 `OR`로 명시한다.
엄격한 예시는 둘 다 만족하는 것이다.

30.2 Failure prediction

- predicted IoU 단독 대비 미래 실패 AUPRC 상대 향상 ≥ 20%
- 보정 후 ECE ≤ 0.05 또는 raw score 대비 30% 이상 감소
- Brier와 NLL 중 적어도 하나가 악화되지 않음
- median detection lead time ≥ 3 frames

30.3 Recovery

- Recovery success@30이 baseline보다 ≥ 10 percentage points 향상
- median TTR이 ≥ 20% 감소
- false reacquisition ≤ 3% 또는 baseline보다 증가하지 않음
- never-recovered rate 감소

30.4 Selective memory

- contaminated memory write 비율 ≤ 5%
- memory write coverage ≥ 70%
- 같은 recovery 호출 budget에서 gate-off보다 J&F 향상

30.5 효율성

- 평균 FPS가 vanilla baseline의 70% 이상
- peak VRAM 증가 ≤ 25%
- 추가 검색 호출 ≤ 전체 frames의 10%
- p95 latency와 worst-case recovery latency를 별도 공개

30.6 일반화

- LVOS 외 MOSE 또는 RobustPVOS 중 적어도 한 곳에서 양의 평균 ΔJ&F
- dataset별 calibration plot을 제시
- target-specific 재보정 없이 얻은 strict cross-dataset 결과를 포함

위 숫자는 연구를 시작하기 위한 예시이지 보편적 기준이 아니다. 데이터 규모와 응용 위험에 따라 조정한다.

31. 결과 해석에서 피해야 할 문장

잘못된 예

> 우리 방법은 J&F가 1.2 높으므로 모든 상황에서 우수하다.

문제점: CI, sequence 분포, 계산량, 특정 dataset 의존성을 모른다.

개선된 예

> LVOS validation sequence에서 평균 J&F는 1.2 point 향상했으나 paired bootstrap 95% CI가 -0.3에서 2.7이어서 일관된 개선이라고 단정하기 어렵다. 이득은 long-term reappearance subset에 집중됐고 DAVIS에서는 차이가 작았다.

잘못된 예

> 위험 예측 AUROC가 0.9이므로 0.9 score는 90% 실패 확률이다.

문제점: AUROC는 순위 지표이지 calibration 지표가 아니다.

개선된 예

> 위험 score의 AUROC는 0.9였지만 high-risk bin에서 과신이 관찰됐다. Logistic calibration 후 AUROC는 유지됐고 ECE가 0.12에서 0.04로 감소했다.

32. 연구 실행 전 최종 체크리스트

문제·지표

- [ ] foreground와 absent frame 정의가 명확한가?
- [ ] primary metric 하나를 test 전에 정했는가?
- [ ] J, F, J&F, mIoU의 평균 단위를 기록했는가?
- [ ] failure와 recovery event matching window가 정해졌는가?

- [ ] AUPRC와 failure prevalence를 함께 보고하는가?

데이터

- [ ] frame이 아니라 source sequence 단위로 split했는가?
- [ ] 원본 영상이 같은 clip의 누수를 확인했는가?
- [ ] validation, calibration, test 역할이 분리됐는가?
- [ ] SA-V가 SAM 2 학습 데이터라는 점을 해석에 반영했는가?
- [ ] MOSE v1/v2, LVOS 버전과 split을 명시했는가?

비교

- [ ] 동일 checkpoint, prompt, 해상도, memory budget인가?
- [ ] 추가 detector와 encoder 비용을 포함했는가?
- [ ] fixed compute/call budget 비교가 있는가?
- [ ] 공식 metric code를 사용했는가?

Calibration·선택

- [ ] score가 어떤 사건의 확률인지 정의했는가?
- [ ] reliability diagram, ECE, Brier/NLL을 함께 확인했는가?
- [ ] conformal 가정과 risk 단위를 sequence 수준으로 설명했는가?
- [ ] coverage와 selective risk를 함께 보고했는가?

통계·효율성

- [ ] sequence-level paired bootstrap CI가 있는가?
- [ ] Wilcoxon을 frame이 아니라 sequence에 적용했는가?
- [ ] multiple testing 보정을 고려했는가?
- [ ] FPS 측정 조건과 p95 latency를 공개했는가?
- [ ] peak VRAM, total/trainable parameters, 외부 모델 FLOPs를 포함했는가?

재현성

- [ ] commit, 환경, seed, checkpoint checksum을 기록했는가?
- [ ] sequence별 raw 결과를 저장했는가?
- [ ] 실패 사례와 성공 사례를 함께 시각화했는가?
- [ ] test를 본 뒤 threshold를 바꾸지 않았는가?

33. 자가점검 문제

문제 **1**

100픽셀 이미지에서 GT 객체가 20픽셀이다. 모델은 25픽셀을 객체로 예측했고, 그중 15픽셀이 GT와 겹쳤다. TP, FP, FN, TN을 구하라.

문제 **2**

문제 1의 precision, recall, IoU, Dice를 계산하라.

문제 **3**

문제 1의 pixel accuracy가 높더라도 모델이 좋다고 단정할 수 없는 이유를 설명하라.

문제 **4**

IoU가 0.6인 mask의 Dice를 계산하라.

문제 **5**

두 모델의 region IoU가 모두 0.8인데 모델 A의 boundary F가 0.9, 모델 B가 0.6이다. 어떤 차이를 의심할 수 있는가?

문제 **6**

foreground IoU만 평균한 값과 background까지 포함한 binary mIoU가 왜 다를 수 있는가?

문제 **7**

모델이 confidence 0.9를 출력한 frame 100개 중 65개만 실제 성공했다. 이 점수는 잘 보정되었는가? 어떤 방향의 오류인가?

문제 **8**

높은 AUROC와 나쁜 ECE가 동시에 가능하다는 것을 설명하라.

문제 **9**

실패율이 1%인 데이터에서 accuracy보다 AUPRC가 중요한 이유와 무작위 AUPRC 기준선을 설명하라.

문제 **10**

Temperature scaling, logistic scaling, isotonic regression의 유연성과 데이터 요구량을 비교하라.

문제 **11**

Conformal risk control로 calibration sequence의 memory-write 위험을 5% 이하로 맞췄다. 전혀 다른 열화상 카메라 영상에서도 반드시 5% 이하라고 말할 수 있는가?

문제 **12**

같은 영상의 홀수 frame을 train, 짝수 frame을 test로 나누면 왜 안 되는가?

문제 **13**

10개 영상에서 각 1,000 frame을 평가했다. 10,000 frame을 독립 표본으로 bootstrap하는 대신 10개 sequence를 resample해야 하는 이유를 설명하라.

문제 **14**

새 방법의 평균 ΔJ&F가 +2.0이고 95% paired bootstrap CI가 [-0.5, 4.4]다. 어떻게 해석해야 하는가?

문제 **15**

Wilcoxon p-value가 0.03이면 새 방법이 실용적으로 크게 향상되었다는 뜻인가?

문제 **16**

TRACK 상태에서 failure risk가 높아졌지만 anchor similarity는 아직 높다. 바로 LOST로 가지 않고 SUSPECT를 거치는 이유는 무엇인가?

문제 **17**

대상이 완전히 가려진 동안 예측 mask를 memory에 계속 기록하면 어떤 문제가 생길 수 있는가?

문제 **18**

Anchor recovery가 motion-only recovery보다 유사 distractor에 강할 수 있는 이유를 설명하라.

문제 **19**

방법 A는 J&F 75, 30 FPS이고 방법 B는 J&F 74, 20 FPS다. 정확도와 속도만 고려할 때 어느 방법이 Pareto dominated인가?

문제 **20**

Recovery@30은 높아졌지만 false reacquisition도 크게 증가했다. 이 시스템을 성공으로 볼 수 있는가? 어떤 추가 기준이 필요한가?

34. 자가점검 정답과 해설

정답 **1**

- TP = 겹친 픽셀 = 15
- FP = 예측 객체 25 중 틀린 것 = 25-15=10
- FN = GT 객체 20 중 놓친 것 = 20-15=5
- TN = 전체 100에서 나머지 = 100-15-10-5=70

정답 **2**

Precision=15/(15+10)=0.6

Recall=15/(15+5)=0.75

IoU=15/(15+10+5)=0.5

Dice=30/(30+10+5)=30/45≈0.667

Precision보다 recall이 높으므로 실제 객체를 비교적 많이 찾았지만 배경으로 번진 FP가 더 큰 문제다.

정답 **3**

Accuracy는 (15+70)/100=0.85다. 배경 TN 70개가 점수를 높였기 때문에 객체 영역 오류가 가려진다. 분할 대상이 작을수록 이 현상이 심하다. IoU와 Dice처럼 TN을 제외한 지표가 객체 품질을 더 직접적으로 보여 준다.

정답 **4**

Dice=(2×0.6)/(1+0.6)=1.2/1.6=0.75

정답 **5**

두 모델의 전체 겹침 면적은 비슷하지만 B는 윤곽선이 거칠거나 위치가 더 어긋났을 가능성이 크다. 얇은 구조를 놓치거나 object 주변으로 부풀린 mask일 수도 있다. Region metric 하나로 fine detail을 판단하면 안 된다.

정답 **6**

Foreground IoU는 지정 객체만 계산한다. Binary mIoU에 background를 포함하면 넓고 쉬운 배경 IoU가 평균에 더해진다. 따라서 foreground IoU가 낮아도 mIoU가 더 높게 나올 수 있다. 어떤 class를 평균했는지 밝혀야 한다.

정답 **7**

잘 보정되지 않았다. 0.9라고 예측했지만 실제 성공률은 0.65이므로 0.25만큼 과신했다. Reliability diagram에서 해당 점은 대각선 아래에 놓인다.

정답 **8**

AUROC는 positive가 negative보다 높은 score를 받는 순서만 본다. 모든 score를 과도하게 높은 값으로 바꾸어도 순서가 유지되면 AUROC는 같다. 그러나 score의 절대값과 실제 빈도가 맞지 않아 ECE는 나쁠 수 있다.

정답 **9**

모두 정상이라고 예측해도 accuracy가 99%이므로 실패 탐지 성능을 알 수 없다. AUPRC는 높은 위험으로 선택한 사례 중 실제 실패 비율과 전체 실패를 찾은 비율을 본다. 무작위 ranking의 기대 AUPRC 기준은 대략 positive prevalence인 0.01이다.

정답 **10**

- Temperature scaling: parameter가 매우 적고 안정적이지만 단순한 sharpness 오류만 주로 고친다.
- Logistic/Platt scaling: 기울기와 절편을 학습해 더 유연하다.
- Isotonic: 단조 비선형 함수를 학습해 가장 유연하지만 calibration sequence가 적으면 과적합하기 쉽다.

보정법은 calibration data에서 선택하고 test에서는 고정해야 한다.

정답 **11**

말할 수 없다. Conformal 보장은 calibration과 test의 exchangeability에 의존한다. 열화상이라는 큰 domain shift가 있으면 가정이 깨질 수 있다. 열화상 test 결과와 calibration curve를 별도로 평가하거나, target calibration을 허용한 프로토콜을 분리해야 한다.

정답 **12**

홀수와 짝수 frame은 배경, 대상 identity, motion이 거의 동일하다. Test가 새로운 영상이 아니라 train frame의 근접 복사본이 되어 성능이 과대평가된다. 원본 sequence 또는 촬영 세션 단위로 나눠야 한다.

정답 **13**

같은 sequence의 frame들은 독립이 아니다. 10,000개로 취급하면 effective sample size를 크게 과장해 CI가 지나치게 좁아진다. Sequence를 cluster로 resample하면 영상 내부 상관을 보존한다. 단, sequence가 10개뿐이면 CI 자체가 불안정하다는 한계도 보고해야 한다.

정답 **14**

관찰 평균은 +2.0이지만 CI가 0을 포함한다. 새 방법이 평균적으로 좋아 보인다는 신호는 있으나, sequence sampling 변동을 고려하면 일관된 양의 개선이라고 강하게 결론내리기 어렵다. 더 많은 sequence나 failure-specific 분석이 필요하다.

정답 **15**

아니다. p=0.03은 귀무가설 아래 현재와 같은 순위 차이가 드물다는 통계적 증거다. 향상 크기를 뜻하지 않는다. 평균/중앙값 Δ, 95% CI, recovery benefit, 계산비용을 함께 봐야 한다. 여러 검정을 했다면 보정 p-value도 확인한다.

정답 **16**

Risk spike가 blur 한 frame이나 일시적 motion 때문에 생겼을 수 있다. Anchor similarity가 높다면 identity는 아직 유지됐을 가능성이 있다. SUSPECT에서 memory write만 막고 소수 후보를 검증하면 불필요한 LOST 전환과 비싼 전체 재검색을 줄일 수 있다.

정답 **17**

가림 물체나 배경, distractor의 특징이 target memory로 들어간다. 이후 SAM 2가 잘못된 특징에 attention하고, 그 예측이 다시 기록되는 memory poisoning과 self-reinforcing drift가 생긴다. Selective write와 absent 상태가 필요한 이유다.

정답 **18**

Motion-only 방식은 target과 distractor가 비슷한 경로를 지나거나 카메라가 급격히 움직이면 잘못된 후보를 선택할 수 있다. Anchor는 초기 target의 appearance와 identity 정보를 제공하며, negative distractor memory를 함께 쓰면 비슷한 후보도 margin으로 구분할 수 있다. 반대로 appearance가 크게 변하면 motion cue가 필요하므로 둘을 결합하는 것이 좋다.

정답 **19**

B는 A보다 J&F가 낮고 FPS도 낮으므로, 두 축만 보면 A가 B를 지배한다. 따라서 B가 Pareto dominated다. 다만 B가 VRAM이나 false recovery에서 더 좋다면 그 축까지 포함한 Pareto 판단은 달라질 수 있다.

정답 **20**

Recovery@30만으로 성공이라고 할 수 없다. 잘못된 객체를 빠르게 선택해도 recovery로 잘못 집계될 수 있다. False reacquisition 상한, anchor identity 검증, post-recovery IoU, relapse rate, 사용자·검색 호출 비용을 공동 성공 기준으로 둬야 한다.

35. 한 페이지 요약

1. TP·FP·FN·TN은 모든 분할 지표의 출발점이다.
2. IoU와 Dice는 영역, Boundary F와 BIoU는 윤곽 품질을 본다.
3. DAVIS의 J는 IoU, F는 contour F, J&F는 두 값의 평균이다.
4. mIoU와 mBIoU는 어떤 클래스를 평균했는지 반드시 밝혀야 한다.
5. Confidence는 아직 probability가 아니며 calibration으로 실제 빈도와 맞춰야 한다.
6. ECE·Brier·NLL은 확률 품질, AUROC·AUPRC는 위험 순위를 본다.
7. 실패가 희귀하면 AUPRC와 event metric이 중요하다.
8. Selective prediction은 위험할 때 강제로 답하지 않고 memory write나 mask 확정을 보류한다.
9. Train·validation·calibration·test는 sequence 단위로 분리한다.
10. 통계는 frame이 아니라 sequence-level paired bootstrap과 Wilcoxon을 사용한다.
11. 평균 J&F만 보지 말고 recovery time, false reacquisition, contamination, latency를 함께 본다.
12. 제안 시스템의 핵심은 미래 실패위험을 보정해 TRACK–SUSPECT–LOST를 전환하고, 안전한 frame만 memory에 기록하며, 검증된 anchor로 target을 다시 찾는 것이다.

PART G — 원문을 읽는 법과 핵심 용어사전

원문 구간별로 이 가이드에서 찾아볼 곳

- 원문 1장 '연구 범위'가 헷갈리면 PART A의 task taxonomy와 VOS/PVOS/VOT/MOT/RVOS/VRS 구분을 다시 본다.
- 원문 2장 '구조적 실패 이유'가 헷갈리면 PART C의 SAM 2 inference loop와 memory contamination을 다시 본다.
- 원문 3.1 '장기 memory, 가림, 재등장'은 PART C의 admission·retention·read·pruning과 PART D의 drift 사례에 대응한다.
- 원문 3.2 'prompt, 언어, 의미 기반 재집중'은 prompt, visual anchor, text embedding, VLM, open-vocabulary 항목에 대응한다.
- 원문 3.3 '다중 객체 identity'는 identity switch, association, mask ownership, bipartite matching 항목에 대응한다.
- 원문 3.4 '경계와 작은 객체'는 downsampling, high-resolution feature, boundary F, mBIoU, temporal flicker 항목에 대응한다.
- 원문 3.5 '도메인 적응과 강건성'은 domain shift, corruption, fine-tuning, PEFT, adapter, LoRA 항목에 대응한다.
- 원문 3.6 '속도·memory 효율'은 FPS, latency, VRAM, parameter, FLOPs, token pruning, quantization, distillation, Pareto frontier 항목에 대응한다.
- 원문 4장 '연구 공백'은 PART E의 calibration과 PART F의 연구가설을 연결해서 읽는다.
- 원문 5장 '최우선 연구안'은 미래 실패 label, calibrator, selective write, 3상태 controller, anchor recovery 순으로 읽는다.
- 원문 6장 '실험 설계'는 baseline, data split, ablation, event metric, bootstrap, Wilcoxon을 확인하며 읽는다.

논문 표를 읽는 **7**단계

1. task를 확인한다. VOS, VOT, MOT, RVOS는 숫자를 바로 비교할 수 없는 서로 다른 문제다.
2. input prompt를 확인한다. 첫 frame GT mask, box, point, 자동 prompt는 난이도가 다르다.
3. backbone과 checkpoint를 확인한다. SAM 2와 SAM 2.1, Tiny·Base+·Large를 섞으면 안 된다.
4. dataset과 split을 확인한다. validation을 조정에 쓰고 같은 set을 test라고 부르면 안 된다.
5. metric과 단위를 확인한다. J&F 0.80과 80은 같은 값일 수 있고, AUC와 J&F는 의미가 다르다.
6. 추가 계산과 supervision을 확인한다. detector, optical flow, VLM, GT box, test-time training이 있으면 비용과 정보 조건이 달라진다.
7. 평균과 함께 실패 유형별 결과, confidence interval, 속도, VRAM을 확인한다.

핵심 용어사전

A–C

Ablation study: 제안 방법의 부품을 하나씩 빼거나 바꾸어 어느 요소가 실제 향상을 만들었는지 확인하는 실험.

Anchor memory: 대상이 확실히 보였던 신뢰 frame이나 feature를 오래 보존해 소실 뒤 복구에 사용하는 기준 기억.

Annotation: 사람이 데이터에 붙인 정답 표시. class, box, point, mask 등이 있다.

Appearance: 물체의 색, 질감, 모양처럼 보이는 특성. motion과 대비되는 단서다.

Association: 현재 frame의 후보와 과거 track 중 무엇이 같은 객체인지 연결하는 과정.

Attention: query가 여러 key와의 관련성을 계산해 필요한 value를 가중합하는 정보 선택 연산.

AUROC: 실패와 성공을 구분하는 score의 임계값을 전부 움직였을 때 true-positive rate와 false-positive rate의 관계를 요약한 면적.

AUPRC: precision–recall 곡선 아래 면적. 실패가 드문 불균형 문제에서는 AUROC보다 실용적일 수 있다.

Backbone: 입력에서 일반적인 feature를 뽑는 모델의 중심 encoder.

Baseline: 제안 방법의 효과를 판단하기 위해 같은 조건에서 비교하는 기준 방법.

Batch: 한 번의 forward/backward에서 함께 처리하는 데이터 묶음.

Bipartite matching: 두 집합의 후보를 비용이 최소가 되도록 일대일 연결하는 방법. 다중 객체 ID association에 쓰인다.

Bounding box: 객체를 둘러싼 직사각형 위치 표현.

Brier score: 예측확률과 실제 0/1 결과의 제곱오차 평균. 낮을수록 확률이 실제 결과와 잘 맞는다.

Calibration: 0.8이라고 예측한 사건이 실제로도 약 80% 일어나도록 score의 확률적 의미를 맞추는 것.

Candidate mask: decoder가 한 prompt에 대해 내놓는 여러 가능한 mask 중 하나.

Checkpoint: 학습된 parameter와 설정을 저장한 파일 또는 상태.

Class: 모델이 구분하려는 의미 범주. instance ID와는 다르다.

CLIP: image와 text를 같은 embedding 공간에 정렬해 의미 유사도를 비교하도록 학습한 모델 계열.

Confidence: 모델이 자기 예측을 얼마나 믿는지를 나타내는 score. 자동으로 정확한 probability가 되는 것은 아니다.

Conformal risk control: calibration data를 이용해 제한된 가정 아래 선택 결과의 위험이나 오류율을 통제하려는 방법 계열.

Confounder: 원인으로 생각한 변수와 결과 사이를 함께 바꾸어 잘못된 결론을 만들 수 있는 숨은 요인.

Convolution: 작은 kernel을 영상 위로 이동시키며 주변 패턴을 검출하는 연산.

Corruption: blur, noise, rain, fog, low light처럼 입력을 손상시키는 변화.

Cross-dataset evaluation: 한 dataset에서 정한 방법이나 calibrator를 다른 dataset에 적용해 일반화를 보는 평가.

D–H

Data leakage: test 정보가 학습, threshold 선택, feature 생성 과정에 흘러 들어가 성능이 실제보다 높아지는 문제.

Decoder: 압축된 feature와 prompt를 최종 mask나 label로 변환하는 부분.

Deep supervision: 중간 decoder 단계에도 loss를 주어 여러 해상도에서 학습을 돕는 방법.

DINO/DINOv2: label 없이도 강한 visual feature를 학습하는 self-supervised vision model 계열.

Dice coefficient: 두 mask의 겹침을 2|교집합|/(|정답|+|예측|)으로 측정하는 값.

Disappearance: target이 화면 밖으로 나가거나 완전히 가려져 관측되지 않는 상태.

Distractor: target과 비슷해 모델을 헷갈리게 하는 다른 객체.

Distillation: 큰 teacher의 출력이나 feature를 작은 student가 따라 배우게 하는 압축 학습.

Domain: 데이터가 만들어지는 환경과 분포. 일반 사진, 의료, 수중, 항공, 야간 영상은 서로 다른 domain이다.

Domain adaptation: source domain에서 학습한 모델을 target domain에 맞게 조정하는 방법.

Domain shift: 학습 환경과 실제 평가 환경의 밝기, 센서, 대상, 배경 분포가 달라지는 현상.

Downsampling: feature의 가로·세로 해상도를 줄여 계산을 아끼는 연산. 작은 객체와 가는 경계가 사라질 수 있다.

Drift: 작은 추적 오류가 시간에 따라 누적되어 mask가 target에서 벗어나는 현상.

ECE: confidence 구간별 평균 confidence와 실제 accuracy의 차이를 가중 평균한 calibration 오차.

Embedding: 이미지, prompt, text, object를 의미나 관계가 보존되도록 숫자 vector로 표현한 것.

Encoder: 원시 픽셀이나 prompt를 task에 유용한 feature로 바꾸는 부분.

Epoch: 학습 dataset 전체를 한 번 순회한 단위.

Event-triggered: 매 N frame처럼 고정 주기로 동작하지 않고 위험·가림·score 변화 같은 사건이 생길 때 동작하는 방식.

Feature: 입력에서 추출한 유용한 숫자 표현. edge, texture, object part, semantics 등이 암묵적으로 담긴다.

Fine-tuning: pretrained parameter의 일부 또는 전체를 새 데이터로 추가 학습하는 것.

Flicker: 영상 mask의 경계나 영역이 frame마다 불안정하게 떨리는 현상.

FLOPs: 모델이 수행하는 부동소수점 연산량의 근사치. 실제 latency와 항상 일치하지는 않는다.

Foundation model: 매우 넓은 데이터로 학습해 여러 task와 domain으로 확장할 수 있도록 만든 대형 기반 모델.

FPS: 초당 처리 frame 수. batch, 해상도, 장치, 측정 구간을 함께 밝혀야 비교할 수 있다.

Frozen: parameter를 학습 중 업데이트하지 않는 상태.

Ground truth, GT: 학습과 평가에서 정답으로 취급하는 annotation.

Hiera: SAM 2 image encoder에 쓰인 계층형 vision Transformer 계열. 여러 해상도의 visual feature를 만든다.

Hyperparameter: 학습률, threshold, memory size처럼 학습 전에 사람이 정하거나 validation으로 선택하는 설정.

I–P

Identity: 여러 frame에서 같은 실제 객체임을 나타내는 지속적인 구분.

Identity switch: tracker가 target의 ID를 다른 객체와 바꾸는 오류.

Image encoder: RGB pixels를 reusable visual feature로 변환하는 encoder.

Inference: 학습이 끝난 모델로 새로운 입력의 결과를 계산하는 과정.

Instance: 같은 class 안에서도 서로 구별되는 개별 객체.

IoU/Jaccard: 정답과 예측 mask의 교집합 크기를 합집합 크기로 나눈 영역 겹침 지표.

Isotonic regression: score의 순서를 유지하면서 단조로운 비선형 함수로 probability를 보정하는 방법.

Latency: 입력 하나 또는 frame 하나의 결과를 얻는 데 걸리는 시간.

LoRA: 큰 weight는 고정하고 낮은 rank의 작은 행렬만 학습해 parameter와 memory를 줄이는 PEFT 방법.

Logistic regression: 입력 feature의 가중합을 sigmoid에 넣어 0–1 probability를 만드는 단순한 분류·calibration 모델.

Long-term memory: 오래전의 중요 정보를 보존해 긴 가림이나 재등장에 사용하는 memory.

Loss: 예측이 정답과 얼마나 다른지 나타내며 학습이 줄이려는 목적함수.

Mask: target인 pixel과 background pixel을 구분하는 지도.

mBIoU: 여러 sample이나 class의 boundary IoU 평균.

Memory admission: 현재 정보를 memory에 새로 기록할지 결정하는 단계.

Memory attention: 현재 feature가 과거 memory의 어느 부분을 참고할지 attention으로 선택하는 부분.

Memory bank: 과거 frame의 feature, mask, pointer 등을 저장하는 제한된 공간.

Memory contamination: 틀린 mask나 잘못된 identity 정보가 memory에 저장되어 이후 예측을 해치는 현상.

Memory encoder: mask와 visual feature를 다음 frame에서 사용할 memory representation으로 변환하는 부분.

Memory pruning: 덜 중요하거나 중복된 memory item을 제거하는 것.

Memory read: 현재 추론에 사용할 memory를 선택하고 가져오는 단계.

Memory retention: 기록된 항목을 얼마나 오래 보존할지 정하는 정책.

mIoU: 여러 sample 또는 class IoU의 평균.

Motion: frame 사이 위치와 모양의 시간적 변화.

Multi-hypothesis: 한 순간에 후보 경로 하나만 남기지 않고 여러 가능성을 잠시 유지하는 방식.

NLL: 실제 결과에 부여한 probability의 negative log likelihood. 확신하고 틀린 예측을 크게 벌준다.

Object pointer: SAM 2에서 추적 객체의 고수준 identity와 의미를 압축해 전달하는 token 성격의 표현.

Object score/presence score: 현재 frame에 target이 존재하는지를 나타내려는 score.

Occlusion: target의 일부 또는 전부가 다른 물체에 가려진 상태.

Online learning: test 또는 사용 도중 새 data·수정 신호로 모델 일부를 업데이트하는 학습.

Open-vocabulary: 학습 때 고정한 class 목록 밖의 개념도 text나 exemplar로 지정할 수 있는 능력.

Optical flow: 인접 frame 사이 각 pixel의 이동을 추정한 vector field.

Overfitting: 학습 data에는 잘 맞지만 새로운 data에는 성능이 떨어지는 현상.

Parameter: 학습으로 값이 바뀌는 weight와 bias.

Pareto frontier: 한 목표를 더 좋게 하려면 다른 목표를 희생해야 하는 최적 trade-off 경계. 예: J&F–FPS–VRAM.

PEFT: 전체 모델 대신 작은 일부 parameter만 학습하는 parameter-efficient fine-tuning 방법군.

Post-hoc calibration: 원 모델의 학습을 바꾸지 않고 완성된 score를 validation data로 뒤에서 보정하는 방법.

Precision: positive라고 예측한 것 중 실제 positive의 비율.

Predicted IoU: mask decoder가 자기 mask와 GT의 IoU를 추정하려고 출력하는 품질 score. 실제 IoU나 보정된 확률과 동일하지 않다.

Pretraining: 넓은 데이터로 일반 feature를 먼저 학습하는 단계.

Prompt: 모델이 무엇을 처리해야 하는지 알려 주는 입력 지시. point, box, mask, text, exemplar가 될 수 있다.

Prompt encoder: point·box·mask prompt를 model이 사용할 embedding으로 바꾸는 부분.

Q–Z

Quantization: float weight와 activation을 int8/int4 같은 낮은 bit 표현으로 바꾸어 memory·연산을 줄이는 기술.

Query–Key–Value: attention에서 query가 질문, key가 검색표, value가 실제 가져올 내용에 해당하는 세 표현.

Recall: 실제 positive 중 모델이 positive로 찾아낸 비율.

Reappearance: 가려지거나 화면 밖으로 나간 target이 다시 보이는 사건.

Reliability diagram: confidence bin마다 평균 confidence와 실제 accuracy를 비교하는 calibration 그래프.

Re-prompt: 모델이 실패했다고 판단했을 때 point, box, mask 등의 prompt를 자동 또는 사용자 입력으로 다시 주는 것.

Reproducibility: 다른 사람이 같은 code, data, seed, configuration으로 비슷한 결과를 얻을 수 있는 성질.

Risk budget: 허용할 평균 오류, 실패율, 계산량, 사용자 개입 횟수 같은 자원·위험 한도.

Robustness: noise, blur, low light, domain shift, prompt error가 있어도 성능이 크게 무너지지 않는 성질.

Selective prediction/abstention: 불확실할 때 무리하게 답하지 않고 보류, 추가 계산, 사용자 요청을 선택하는 방식.

Short-term memory: 최근 appearance와 motion을 빠르게 반영하는 memory.

SOTA: 특정 공개 조건에서 당시 보고된 최고 수준 결과. task와 조건이 다르면 직접 비교할 수 없다.

Streaming: 미래 frame을 보지 않고 들어오는 순서대로 제한된 memory와 시간 안에 처리하는 설정.

Temperature scaling: logit을 하나의 양수 temperature로 나눈 뒤 probability를 다시 계산하는 단순 calibration 방법.

Temporal consistency: 같은 객체의 mask와 identity가 시간에 따라 자연스럽고 안정적으로 이어지는 성질.

Tensor: 여러 축을 가진 숫자 배열. image, video, feature, mask의 기본 자료구조.

Test-time adaptation: 배포 또는 test data를 보면서 parameter나 statistic을 현장에서 조정하는 방법.

Token: Transformer가 처리하는 기본 단위. image patch, prompt, object pointer, memory item 등이 token이 될 수 있다.

Token pruning: 중요도가 낮은 token을 attention과 memory 계산에서 제거하는 효율화 방법.

Tracking-by-detection: 매 frame detection을 만든 뒤 시간에 따라 같은 객체를 연결하는 tracking 방식.

Transformer: token 사이 관계를 attention으로 학습하는 신경망 구조.

Uncertainty: 예측이 틀릴 가능성이나 결과의 모호함에 대한 정보. 하나의 confidence 숫자와 동일하지 않다.

Upsampling: 낮은 해상도 feature나 mask를 높은 해상도로 키우는 과정.

Validation set: 학습 자체가 아니라 model·threshold·hyperparameter 선택에 쓰는 data.

Visual prompt: point, box, mask, exemplar image처럼 시각적 위치나 대상을 지정하는 prompt.

VLM/LVLM/MLLM: image/video와 language를 함께 이해·생성하는 vision-language model 계열. large 또는 multimodal을 강조할 때 약어가 달라진다.

VRAM: GPU가 model weight, activation, memory feature를 저장하는 전용 memory.

Wavelet: image를 여러 주파수·scale 성분으로 분해하는 수학적 표현. 미세 경계와 저주파 구조를 따로 다루는 데 쓰인다.

Zero-shot: target task의 labeled example로 추가 학습하지 않고 pretrained 능력과 prompt만으로 수행하는 설정.

마지막 **10**문장 요약

- Image는 pixel tensor이고 video는 시간축으로 image가 이어진 tensor다.
- Segmentation은 pixel마다 target 여부를 맞추고 VOS는 같은 target mask를 시간축에서 유지한다.
- SAM 2는 image encoder, prompt encoder, memory attention, mask decoder, memory encoder가 순환한다.
- Memory는 과거를 돕지만 틀린 결과를 저장하면 drift를 증폭시키는 통로가 된다.
- 가림과 비슷한 distractor는 target 존재 판단과 identity 유지 문제를 동시에 만든다.
- Predicted IoU와 object score는 유용한 내부 신호지만 실제 probability라고 가정하면 안 된다.
- Calibration은 score를 실제 실패 빈도와 맞추고, policy는 그 risk에 따라 write·보류·복구를 결정한다.
- 평균 J&F만으로는 memory 오염, 실패 시점, 가림 후 recovery delay를 충분히 설명할 수 없다.
- 공정한 연구는 같은 checkpoint·prompt·dataset split에서 baseline과 ablation을 비교하고 통계적 uncertainty를 보고한다.
- 좋은 결론은 '점수가 올랐다'가 아니라 어떤 failure mechanism이 줄었고 어떤 비용을 지불했는지를 말한다.